FPT SOFTWARE

Điều khiển robot hình người G1 gắp–đặt vật bằng NVIDIA GR00T



Nghiên cứu khả thi về mô hình NVIDIA GR00T Foundation

BÁO CÁO DỰ ÁN · UNITREE G1

Khách hàng
Shimizu Corporation

Thời gian
24/08/2026 – 06/11/2026

Phiên bản · Phát hành
v1.0 · 06/11/2026

MỤC LỤC

Nghiên cứu khả thi về mô hình NVIDIA GR00T Foundation

PHẦN I BỐI CẢNH & NỀN TẢNG 5

I.01 Tổng quan & kết quả 5
- I.01.1 Bài toán nghiên cứu 5
- I.01.2 Kịch bản thực nghiệm 6
- I.01.3 Mục tiêu nghiên cứu 7
- I.01.4 Phạm vi nghiên cứu 7
- I.01.5 Số liệu nghiên cứu 8
- I.01.6 Quy trình thực hiện 8
- I.01.7 Kế hoạch thực hiện 9

I.02 Hạ tầng & hiệu năng 11
- I.02.1 Thiết bị & kết nối 11
- I.02.2 Phần mềm & phiên bản 12
- I.02.3 Hiệu năng suy luận 12
- I.02.4 Hiệu năng thu dữ liệu 13
- I.02.5 Hiệu năng huấn luyện 13
- I.02.6 Phân tích hiệu năng 14

PHẦN II NGHIÊN CỨU BÀI TOÁN 15

II.01 Nghiên cứu mô hình 15
- II.01.1 Thuật ngữ nhanh 15
- II.01.2 Cơ chế vận hành 15
- II.01.3 Dữ liệu đầu vào 16
- II.01.4 Dữ liệu đầu ra 17
- II.01.5 Kiến trúc mô hình 18
- II.01.6 Kết quả công bố 19
- II.01.7 Đối chiếu phiên bản 20
- II.01.8 Checkpoint thực nghiệm 20
- II.01.9 Kết luận nghiên cứu 21

II.02 Nghiên cứu pipeline 22
- II.02.1 Luồng tổng thể 22
- II.02.2 Khởi tạo môi trường 24
- II.02.3 Tạo kết nối 24
- II.02.4 Lấy quan sát 24
- II.02.5 Gửi quan sát 25
- II.02.6 Nhận chuỗi action 25
- II.02.7 Xử lý chuỗi action 26
- II.02.8 Chạy robot 26
- II.02.9 Chạy vòng kín 27
- II.02.10 Danh mục 12 notebook và vị trí trong báo cáo 27

II.03 Nghiên cứu chuyển động 29
- II.03.1 Cấu trúc khớp G1 29
- II.03.2 Động học thuận 30
- II.03.3 Động học ngược 31
- II.03.4 Động lực học 31
- II.03.5 Giới hạn tốc độ khớp 33
- II.03.6 Bước vật lý: chuyển động được tính thế nào 33
- II.03.7 Mô phỏng: chiếc xe qua từng bước vật lý 35
- II.03.8 Tick điều khiển và tick vật lý 36
- II.03.9 Bên trong một tick điều khiển 37
- II.03.10 Thời gian mô phỏng và thời gian thực 38

II.04 Nghiên cứu dữ liệu 39
II.04.1 Problem 1: Visual Gap 39
II.04.2 Solution 1: Domain Randomization 39
II.04.3 Problem 2: Sim Data Scarcity 41
II.04.4 Solution 2: Real2sim 41
II.04.5 Problem 3: Teleop Bottleneck 42
II.04.6 Solution 3: Demo Generation 42
II.04.7 Kết hợp và kết luận 43

PHẦN III KỊCH BẢN TRIỂN KHAI 44

III.01 Kịch bản Apple & Plate 44
III.01.1 Video Showcase 44
III.01.2 Bối cảnh kịch bản 45
III.01.3 Mục tiêu kịch bản 46
III.01.4 Dữ liệu kịch bản 46
III.01.5 Vấn đề gặp phải 46
III.01.6 Phương pháp đề xuất 47
III.01.7 Chu trình thực hiện 48
III.01.8 Kết quả đạt được 49
III.01.9 Phân tích kết quả 50
III.01.10 Kết luận kịch bản 52

III.02 Kịch bản Multi Fruit 53
III.02.1 Video Showcase 53
III.02.2 Bối cảnh kịch bản 54
III.02.3 Mục tiêu kịch bản 55
III.02.4 Dữ liệu kịch bản 55
III.02.5 Vấn đề gặp phải 55
III.02.6 Phương pháp đề xuất 56
III.02.7 Chu trình thực hiện 58
III.02.8 Fine-tune 59
III.02.9 Kết quả đạt được 59
III.02.10 Phân tích kết quả 61
III.02.11 Kết luận kịch bản 62

III.03 Kịch bản Hammer & Bin 63
III.03.1 Video Showcase 63
III.03.2 Bối cảnh kịch bản 64
III.03.3 Mục tiêu kịch bản 65
III.03.4 Dữ liệu kịch bản 65
III.03.5 Vấn đề gặp phải 65
III.03.6 Phương pháp đề xuất 66
III.03.7 Chu trình thực hiện 67
III.03.8 Fine-tune 68
III.03.9 Kết quả đạt được 68
III.03.10 Phân tích kết quả 70
III.03.11 Kết luận kịch bản 72

PHẦN IV BÀN GIAO & KIỂM CHỨNG 73

IV.01 Sản phẩm đầu ra 73
IV.01.1 Cấu trúc gói bàn giao 73
IV.01.2 Đối tượng sử dụng theo vai trò 73
IV.01.3 Danh mục nội dung chi tiết 74
IV.01.4 Dữ liệu gốc bàn giao riêng 76
IV.01.5 Giấy phép và điều khoản sử dụng 77

IV.02 Cài đặt môi trường 78
IV.02.1 Kiến trúc kết nối 78
IV.02.2 Yêu cầu phần cứng và phần mềm 79

IV.02.3 Đặt gói sản phẩm trên máy trạm 79
IV.02.4 Cài client trên máy trạm 80
IV.02.5 Cài server trên máy chủ GPU 81
IV.02.6 Đưa checkpoint bàn giao lên server 83
IV.02.7 Thiết lập kết nối 84

IV.03 Chạy server GR00T 86
IV.03.1 Cấu hình server theo kịch bản 86
IV.03.2 Khởi động server 87
IV.03.3 Kiểm tra kết nối từ máy trạm 88

IV.04 Chạy kiểm chứng 90
IV.04.1 Quy trình chung 90
IV.04.2 Kiểm tra nền không cần server 91
IV.04.3 Lệnh kiểm chứng từng kịch bản 92
IV.04.4 Đọc và so sánh kết quả 93
IV.04.5 Chạy tương tác trong Jupyter 93

IV.05 Đánh giá video bằng VSS 95
IV.05.1 VSS làm gì trong dự án 95
IV.05.2 Yêu cầu máy chủ 96
IV.05.3 Cài đặt VSS 3.2.1 97
IV.05.4 Cấu hình sau cài đặt và kiểm tra sức khoẻ 98
IV.05.5 Ba cách chấm và kết quả trên 99 episode 99
IV.05.6 Chạy lại ba cách chấm trên 2 episode (video bước 9–11) 101
IV.05.7 Giao diện web 102
IV.05.8 Phát hiện và việc cần làm 103
IV.05.9 Vận hành hằng ngày 104

PHẦN V ĐÁNH GIÁ & PHỤ LỤC 105

V.01 Đánh giá tổng hợp 105
V.01.1 Kết quả tổng hợp 105
V.01.2 Nhận định chung 106
V.01.3 Bài học rút ra 107
V.01.4 Định hướng tương lai 109

V.02 Phụ lục 111
V.02.1 Thuật ngữ 111
V.02.2 Tài liệu tham khảo 112

I.01 Tổng quan & kết quả

Điều khiển robot hình người G1 gắp–đặt vật bằng NVIDIA GR00T.

Hơn hai tháng nghiên cứu và thực nghiệm (24/08 – 06/11/2026): đọc hiểu mô hình nền tảng GR00T, **tái hiện độc lập** pipeline của NVIDIA Isaac Lab Arena, tạo dữ liệu **real2sim** từ robot G1 thật, và hoàn thành trọn chu trình cho một **tác vụ mới**, từ teleop thu dữ liệu đến fine-tune và đánh giá.

KHÁCH HÀNG
Shimizu Corporation

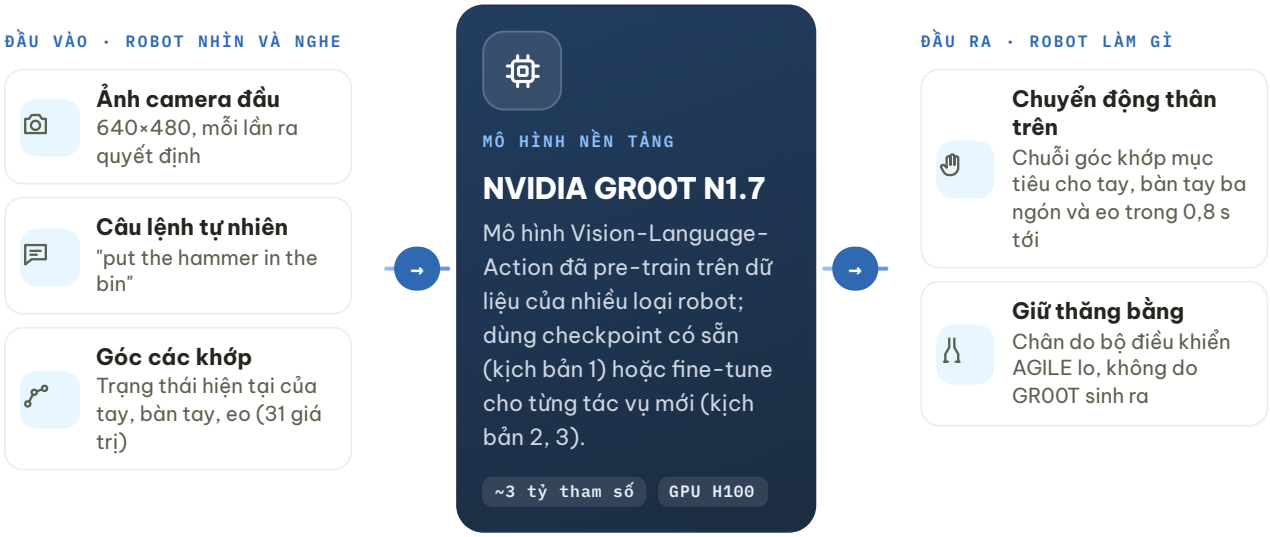
THỰC HIỆN
FPT Software

THỜI GIAN
24/08/2026 – 06/11/2026

MÔI TRƯỜNG
Mô phỏng Isaac Sim / Isaac Lab

1 Bài toán nghiên cứu

Robot hình người **Unitree G1** cần tự gắp một vật và đặt vào đúng chỗ chỉ từ **ảnh camera** và một **câu lệnh**, không lập trình sẵn từng chuyển động. Dự án nghiên cứu cách giao việc này cho mô hình nền tảng **NVIDIA GR00T**: model nhận ảnh, câu lệnh và trạng thái khớp, rồi sinh chuyển động cho thân trên. Toàn bộ thực hiện trong mô phỏng Isaac Sim / Isaac Lab.



Bốn câu hỏi nghiên cứu mà ba kịch bản thực nghiệm cần trả lời:

1 CÂU HỎI

GR00T có điều khiển được G1 gắp–đặt đồ vật không?

Có: 92 % · 66 % · 73 %

Ba kịch bản →

2 CÂU HỎI

Dạy một tác vụ mới cần bao nhiêu dữ liệu và công sức?

200 demo, ~1 giờ fine-tune

Kịch bản 3 →

3 CÂU HỎI

Kết quả thay đổi thế nào khi môi trường khác lúc ghi dữ liệu?

92 % → 56 % → 0 %

Kịch bản 1 →

4 CÂU HỎI

Dữ liệu robot thật có đi vòng qua mô phỏng rồi quay lại robot thật được không (real2sim2real)?

Real2sim: 66 % · sim2real: giai đoạn 2

Kịch bản 2 →

2 Kịch bản thực nghiệm

Ba kịch bản tăng dần độ khó: từ **tái hiện** pipeline có sẵn của NVIDIA, tới **dữ liệu robot thật** chuyển vào mô phỏng, và cuối cùng là một **tác vụ hoàn toàn mới** do dự án tự xây dựng.

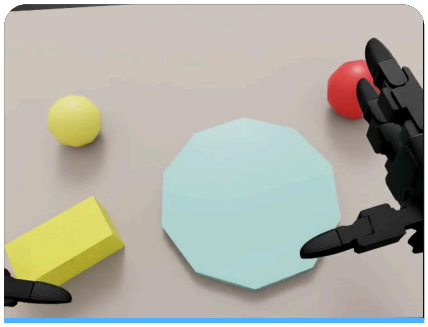


KỊCH BẢN 1

Táo → đĩa

`"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."`

MỤC ĐÍCH	Tái hiện độc lập pipeline mẫu của NVIDIA
DỮ LIỆU	208 demo của NVIDIA, model có sẵn
THÀNH CÔNG	Táo nằm trên đĩa, cách tâm < 8 cm, đứng yên



KỊCH BẢN 2

4 loại quả

`"Pick up the green grapes and place it on the plate"`

MỤC ĐÍCH	Chọn đúng quả theo câu lệnh giữa các quả gây nhiễu
DỮ LIỆU	790 episode real2sim từ dữ liệu G1 thật
THÀNH CÔNG	Quả được nhấc, nằm trên đĩa, không xô lệch quả khác



KỊCH BẢN 3

Búa → thùng

`"put the hammer in the bin"`

MỤC ĐÍCH	Dạy một tác vụ hoàn toàn mới, từ đầu đến cuối
DỮ LIỆU	Tự thu bằng teleop Meta Quest: 200–400 episode
THÀNH CÔNG	Búa trong thùng, đứng yên, tay đã buông

3 Mục tiêu nghiên cứu

MỤC TIÊU CHUNG

Đánh giá khả năng dùng **GR00T** điều khiển robot **G1** gắp–đặt vật (câu hỏi 1), và xác định cần bao nhiêu dữ liệu, công sức để đưa model vào một tác vụ mới. Mục tiêu chung được chia thành 6 mục tiêu cụ thể dưới đây.

1 NỀN TẢNG

Làm chủ GR00T và pipeline điều khiển

Hiểu kiến trúc và quy trình fine-tune từ GR00T N1 tới N1.7; tự xây lại luồng GR00T ↔ G1 (client, lớp an toàn, AGILE giữ thăng bằng, backend Isaac Lab), không phụ thuộc khung có sẵn của NVIDIA.

II.01–II.03

2 KỊCH BẢN 1

Tái hiện kết quả của NVIDIA

Chạy checkpoint táo → đĩa trên pipeline tự xây, đối chiếu với môi trường gốc để kiểm chứng pipeline trước khi dùng cho tác vụ mới.

III.01

3 KỊCH BẢN 2

Tận dụng dữ liệu robot thật

Chuyển dữ liệu teleop G1 thật (chỉ góc khớp và video) thành dataset mô phỏng bằng real2sim; fine-tune model chọn đúng quả theo câu lệnh.

III.02 Câu hỏi 4

4 KỊCH BẢN 3

Dạy một tác vụ hoàn toàn mới

Trọn chu trình "búa → thùng": dựng môi trường, teleop bằng kính thực tế ảo, sinh thêm dữ liệu, fine-tune, đánh giá; ghi lại số demo và thời gian cần.

III.03 Câu hỏi 2

5 VISUAL GAP

Thử nghiệm và phân tích khác biệt hình ảnh

Đo tỉ lệ thành công thay đổi ra sao khi ảnh model nhận khác lúc ghi dữ liệu: giữa các khung cảnh mô phỏng và giữa ảnh thật với ảnh render; tìm yếu tố gây sụt giảm, đề xuất cách thu hẹp như domain randomization.

Kịch bản 1, 2 Câu hỏi 3, 4

6 XUYÊN SUỐT

Đánh giá định lượng, lặp lại được

Chấm vòng kín bằng điều kiện thành công rõ ràng, ≥ 50 episode mỗi cấu hình, khoảng tin cậy Wilson 95 %, phân tích thất bại theo pha; mỗi lần chạy chỉ đổi một yếu tố.

III.01–III.03

4 Phạm vi nghiên cứu

Toàn bộ thực nghiệm diễn ra trong mô phỏng; pipeline được thiết kế để thay backend robot thật ở giai đoạn sau.

TRONG PHẠM VI

- ✓ Mô phỏng Isaac Sim 6.0 / Isaac Lab 3.0, Isaac Lab Arena 0.2.1

NGOÀI PHẠM VI · GIAI ĐOẠN SAU

- – Triển khai trên robot G1 thật
- – Di chuyển kết hợp thao tác

- ✓ G1 đứng tại chỗ, hai tay + bàn tay Dex3
- ✓ GR00T N1.7-3B: checkpoint NVIDIA và tự fine-tune
- ✓ Dữ liệu: demo NVIDIA, dataset G1 thật công khai (real2sim), teleop tự thu trong mô phỏng
- ✓ Fine-tune trên 1 GPU H100

- – Huấn luyện mô hình từ đầu
- – Thu dữ liệu trên robot thật
- – Huấn luyện đa GPU quy mô lớn

5 Số liệu nghiên cứu

Kết quả tốt nhất của từng kịch bản, kèm các lần chạy dẫn tới con số đó. Bấm vào từng kịch bản để xem chi tiết.

KỊCH BẢN 1

Táo → đĩa

92%

môi trường gốc NVIDIA

Client NVIDIA Arena	**92%**
Chỉ lệch ánh sáng	**56%**
Lệch toàn bộ môi trường	**0%**

Model rất nhạy với hình ảnh: cùng checkpoint, chỉ đổi môi trường là kết quả thay đổi từ 92 % xuống 0 %.

KỊCH BẢN 2

4 loại quả

66%

33/50 · 95 %: 52–78 %

Checkpoint táo, chưa tune	**0%**
Fine-tune real2sim	**66%**

Fine-tune trên dữ liệu real2sim là bắt buộc; model chọn đúng quả theo câu lệnh trong mọi lần thử.

KỊCH BẢN 3

Búa → thùng

73%

73/100 · 95 %: 64–81 %

Chưa fine-tune	**0%**
Fine-tune 200 episode	**66%**
Fine-tune 400 episode	**63%**
Nới ngưỡng đứng yên	**73%**

Tác vụ mới từ 0 % lên 73 % chỉ với dữ liệu tự thu thập và 1–2 giờ huấn luyện mỗi lần (61 phút với mix200, 2 giờ 15 với mix400).

6 Quy trình thực hiện

Dự án đi theo ba bước: xây **nền tảng chung**, chạy **ba kịch bản** trên nền tảng đó, rồi dùng kết quả đánh giá để **cải tiến**. Mỗi kịch bản đi qua cùng bốn chặng (nguồn dữ liệu → phần việc trọng tâm → model → đánh giá vòng kín) và khác nhau ở phần việc trọng tâm: kịch bản 1 dùng lại dữ liệu và model của NVIDIA, trọng tâm là

nghiên cứu và viết lại luồng GR00T ↔ G1; kịch bản 2 tự tạo dữ liệu mô phỏng từ dữ liệu robot thật, trọng tâm là **real2sim**; kịch bản 3 tự làm một tác vụ mới từ đầu, trọng tâm là **tăng cường dữ liệu** từ demo teleop. Ô nền tối trong sơ đồ đánh dấu phần việc trọng tâm đó.

NỀN TẢNG CHUNG — Nghiên cứu GR00T N1 → N1.7 · Isaac Lab Arena · pipeline độc lập GR00T ↔ G1 (client, lớp an toàn, AGILE, backend Isaac Lab)

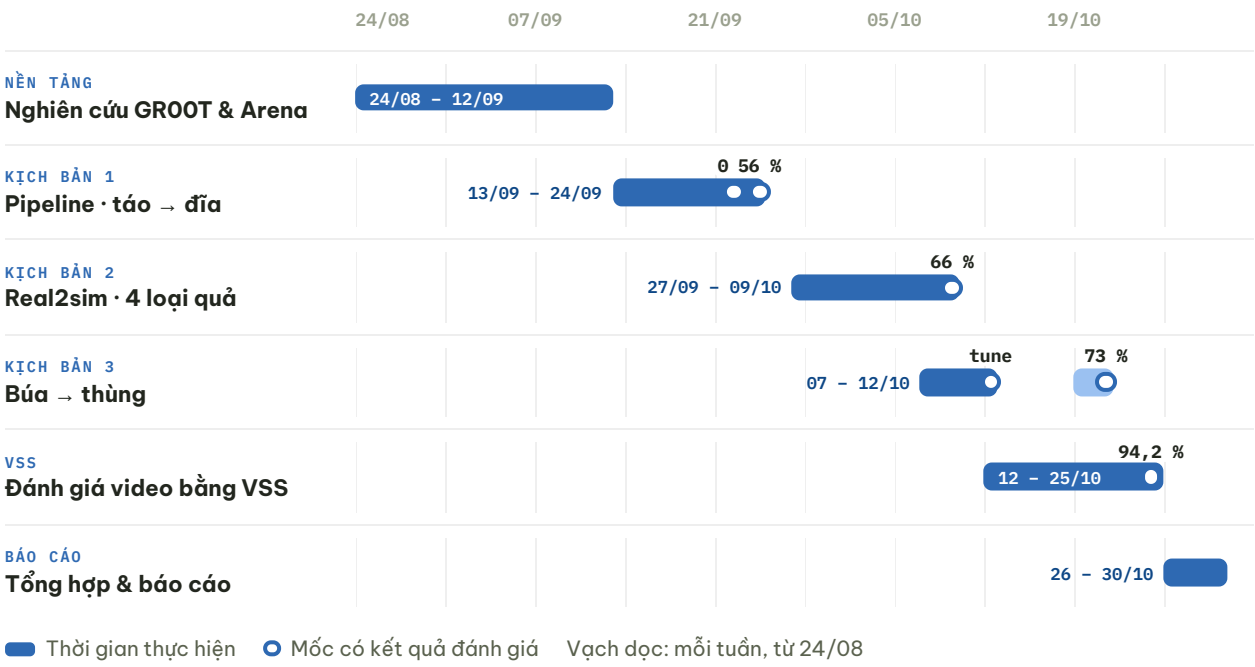
	NGUỒN DỮ LIỆU	TRỌNG TÂM	MODEL GR00T N1.7	KẾT QUẢ VÒNG KÍN
KỊCH BẢN 1 **Nghiên cứu & viết lại luồng · táo → đĩa**	NVIDIA **Dataset static apple** 208 episode teleop XR trong mô phỏng	VIẾT LẠI LUỒNG **12 notebook → pipeline độc lập** Kiểm chứng open-loop, phát lại demo	CÓ SẴN **Checkpoint của NVIDIA** Fine-tune sẵn cho táo → đĩa	MÔI TRƯỜNG GỐC ARENA **92 %** Galileo dựng lại 56 % · khung cảnh tự dựng 0 %
KỊCH BẢN 2 **Real2sim · 4 loại quả**	ROBOT G1 THẬT **1.095 quỹ đạo teleop** Chỉ có góc khớp + video	REAL2SIM **790 episode mô phỏng** FK → khung cảnh, vật nhiễu, phát lại, lọc	FINE-TUNE · H100 **Model đa nhiệm 4 quả** 20.000 bước · 2 giờ 34 phút	50 EPISODE **66 %** Checkpoint táo 0 % · 0 lần gắp nhầm
KỊCH BẢN 3 **Tăng cường dữ liệu · búa → thùng**	TELEOP META QUEST **100 demo teleop** Thu trong Isaac Lab, môi trường tự xây dựng	TĂNG CƯỜNG DỮ LIỆU **mix200 · mix400** 100 teleop + 100 sinh thêm; mix400 thêm 200 teleop	FINE-TUNE · H100 **Model búa → thùng** mix200 ~1 giờ · mix400 ~2 giờ 15 phút	100 EPISODE **73 %** Chưa tune 0 % → 66 % → 73 %

↺ **VÒNG CẢI TIẾN** — Phân tích thất bại theo pha → sửa môi trường hoặc cách chấm, bổ sung dữ liệu → huấn luyện và đánh giá lại. Ví dụ: dựng lại khung cảnh Galileo (kịch bản 1), sửa lỗi camera không render mỗi bước (kịch bản 2), mix200 → mix400 và ngưỡng đứng yên 0,01 → 0,02 (kịch bản 3).

7 Kế hoạch thực hiện

Dự án chạy theo **ba kịch bản nối tiếp** trên cùng một nền tảng, mỗi kịch bản khoảng hai tuần. Hai tuần gần cuối (12–25/10) đánh giá video episode bằng VSS (mục IV.05), song song với phần cuối kịch bản 3; tuần cuối (26–30/10) tổng hợp và viết báo cáo. Biểu đồ dưới đây cho thấy thời gian thực hiện và các mốc có kết

quả; chi tiết từng kịch bản nằm ở các mục III.



I.02 Hạ tầng & hiệu năng

Hai máy, một kính VR — và những con số giới hạn hệ thống.

Mô phỏng, teleop và đánh giá chạy trên **một máy trạm**; huấn luyện và chạy model trên **một máy chủ GPU H100** ở xa. Mục này mô tả cấu hình đã dùng, phần mềm cần cài trên từng máy, và các chỉ số hiệu năng đo được — đặc biệt là **độ trễ model**, giới hạn quan trọng nhất khi chuyển sang robot thật.

MÁY TRẠM
RTX 5090 · Ryzen 7 9800X3D · 64 GB

MÁY CHỦ
1× H100 80 GB · 188 GB

MÔ PHỎNG
Isaac Sim 6.0.1 · Isaac Lab 3.0-beta2

1 Thiết bị & kết nối

Máy trạm là trung tâm: nhận thao tác từ kính VR, chạy mô phỏng robot, và gửi quan sát tới máy chủ GPU để nhận lại chuỗi chuyển động.

TELEOP
Kính Meta Quest

Người điều khiển robot trong mô phỏng để tạo demo.

THIẾT BỊ	Kính + 2 tay cầm
NỐI MÁY	Cáp USB 3
GIAO THỨC	OpenXR · CloudXR

USB 3 · CloudXR

MÁY TRẠM · TRUNG TÂM
Mô phỏng & điều khiển

Chạy Isaac Sim, Arena, client điều khiển, bộ giữ thăng bằng, đánh giá và ghi dữ liệu.

GPU	RTX 5090 · 32 GB
CPU	Ryzen 7 9800X3D · 8 nhân
RAM / SSD	64 GB · NVMe 1 TB
HĐH	Ubuntu 24.04 LTS

GIAI ĐOẠN SAU
Robot Unitree G1 thật

Nhận lệnh khớp từ client qua Unitree SDK2; hiện robot chỉ chạy trong mô phỏng.

KHỚP	43 · bàn tay Dex3
CAMERA	Đầu robot, 640×480

SSH tunnel · ZeroMQ

MÁY CHỦ GPU · TỪ XA
Huấn luyện & chạy model

Fine-tune GR00T và chạy policy server trả chuỗi chuyển động cho máy trạm.

GPU	1× H100 · 80 GB
RAM	188 GB
DRIVER	575.57
SỬ DỤNG	Dùng chung nhiều dự án

2 Phần mềm & phiên bản

Mỗi cột là các lớp phần mềm của một máy, xếp từ nền tảng (dưới) lên ứng dụng của dự án (trên, tô đậm).

Máy trạm — MÔ PHỎNG

Client g1gr00t + AGILE — Python 3.10+
numpy, pyzmq, msgpack, opencv, onnxruntime

Isaac Lab Arena — 0.2.1
Chạy trong Docker 29; nhánh riêng cho bài búa

Isaac Lab — 3.0.0-beta2
Cặp chính thức với Isaac Sim 6.0

Isaac Sim — 6.0.1
Python 3.12, cài cục bộ

Driver NVIDIA + CUDA — 580 · 12.6
Bản 580 đủ cho Isaac Sim 6.0; Isaac Sim 6.1 sẽ cần ≥ 595

Ubuntu 24.04 LTS — LTS
Hệ điều hành

Thêm: Jupyter, pytest, pandas, matplotlib cho notebook và kiểm thử.

Máy chủ GPU — MODEL

GR00T N1.7-3B + checkpoint — 3B
Policy server ZeroMQ; checkpoint mỗi bản 12,6 GB

Cosmos-Reason2-2B — 2B
Backbone thị giác–ngôn ngữ, tải từ Hugging Face

Isaac-GR00T — 4b1dca9
Môi trường Python bằng uv; đúng commit Arena đã kiểm thử

Linux + Driver NVIDIA — 575.57

Cần chấp nhận giấy phép model của NVIDIA trên Hugging Face.

Robot thật — GIAI ĐOẠN SAU

Client g1gr00t — Python 3.10+
Dùng lại nguyên vẹn; chỉ thêm backend robot thật

Unitree SDK2 — –
Gói unitree_sdk2py, đã khai báo trong cấu hình

Máy tính trên robot
Hoặc máy tính đặt cạnh robot

Chưa triển khai trong giai đoạn này.

3 Hiệu năng suy luận

Robot nhận lệnh bao nhiêu lần mỗi giây, mỗi lần gọi model cho bao nhiêu chuyển động, và model trả lời mất bao lâu.

TẦN SỐ ĐIỀU KHIỂN
50 Hz
Vật lý 200 Hz, 4 bước mỗi tick · real2sim 20 Hz

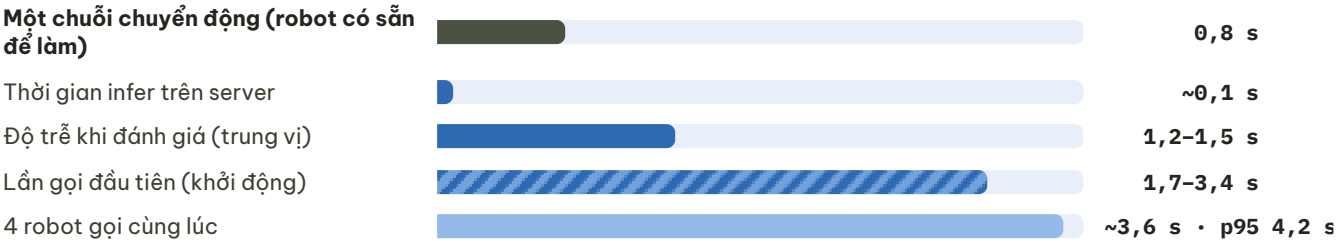
MỖI LẦN GỌI MODEL
40 bước
= 0,8 s chuyển động · real2sim 16 bước, thực thi 8

INFER TRÊN SERVER
~100 ms
Model tính một chuỗi 40 bước trên H100

ĐỘ TRỄ MODEL
~1,2 s
Trung vị mỗi lần gọi khi đánh giá, qua SSH tunnel tới H100

Ngân sách thời gian: chuyển động so với độ trễ model

giây · thang 0–4 s



Model trả lời **chậm hơn** thời gian robot thực hiện xong một chuỗi. Trong mô phỏng không sao vì mô phỏng dừng chờ; trên robot thật, robot sẽ khựng giữa các chuỗi nếu không gọi trước và không đặt GPU gần robot. Bản thân model chỉ mất ~100 ms; phần lớn độ trễ còn lại nằm ở đường truyền qua SSH tunnel và đóng gói dữ liệu.

4 Hiệu năng thu dữ liệu

Đo khi ghi demo bài búa bằng kính Meta Quest qua CloudXR.

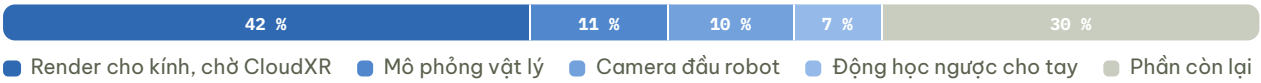
RENDER CHO KÍNH
42 %
Phần lớn nhất của một vòng lặp, gồm chờ CloudXR

MÔ PHỎNG VẬT LÝ
11 %
Camera đầu robot 10 %, động học ngược 7 %

BỘ NHỚ RAM
~12 GB
Tăng ~15 MB/s khi ghi, giải phóng khi lưu demo

Một vòng lặp teleop dành thời gian cho việc gì

đo khi ghi demo bài búa



Nút thắt khi teleop là đường truyền hình ảnh tới kính, không phải mô phỏng. Tiến trình dùng ~12 GB RAM và tăng ~15 MB/s trong lúc ghi (bộ đệm ảnh camera, giải phóng khi lưu demo).

5 Hiệu năng huấn luyện

Fine-tune GR00T N1.7 trên một GPU H100 và chuẩn bị dữ liệu huấn luyện.

THỜI GIAN FINE-TUNE
61 phút
10.000 bước · 0,36 s/bước · batch 32

BỘ NHỚ GPU
≈ 53 GB
LLM đóng băng; tune thị giác, projector, diffusion

CHECKPOINT
12,6 GB
Chỉ trọng số; ≈ 27 GB nếu kèm optimizer

CHỈ SỐ	GIÁ TRỊ	GHI CHÚ
Tốc độ huấn luyện	0,36 s / bước	Batch 32, 1× H100 (2,9 s/bước khi GPU bị job khác chiếm)
Bộ nhớ GPU khi huấn luyện	≈ 53 GB	LLM đóng băng; tune thị giác, projector, diffusion
10.000 bước huấn luyện	61 phút	Loss 1,27 → 0,020

CHỈ SỐ	GIÁ TRỊ	GHI CHÚ
Kích thước checkpoint	12,6 GB	Chỉ trọng số; ≈ 27 GB nếu kèm trạng thái optimizer
Sinh dữ liệu tăng cường	100 demo / 17 phút	133 lượt chạy, giữ 100 lượt thành công
Dung lượng dữ liệu	131 MB / 13 GB	200 episode búa dạng LeRobot / dạng HDF5 gốc có ảnh
Kiểm thử tự động	84 unit test	Test trong mô phỏng: ~3,5 phút

6 Phân tích hiệu năng

Đọc chung ba card trên: ở mỗi khâu, thời gian và tài nguyên thực sự bị tiêu tốn ở đâu, và điều đó có ý nghĩa gì cho giai đoạn sau.

SUY LUẬN

Nút thắt là đường truyền, không phải model

Model chỉ tính ~100 ms, nhưng mỗi lần gọi mất ~1,05–1,3 s, lâu hơn một chuỗi chuyển động (0,8 s). Phần chênh chủ yếu nằm ở SSH tunnel và đóng gói dữ liệu. Khi 4 robot gọi cùng lúc, độ trễ lên ~3,6 s (p95 4,2 s) vì server xử lý lần lượt.

THU DỮ LIỆU

Nút thắt là hình ảnh tới kính

Render và chờ CloudXR chiếm 42 % mỗi vòng lặp, gấp gần 4 lần mô phỏng vật lý (11 %). RAM tăng ~15 MB/s trong lúc ghi, nên mỗi demo cần lưu đều đặn thay vì ghi liền một phiên dài.

HUẤN LUYỆN

Một H100 là đủ, nếu không bị chia sẻ

10.000 bước fine-tune mất 61 phút và dùng ≈ 53 GB bộ nhớ GPU, vừa một H100. Khi GPU bị job khác chiếm, tốc độ chậm gần 8 lần (2,9 s/bước so với 0,36 s/bước).

Hướng cải thiện: đặt máy chủ GPU gần robot hoặc trong cùng mạng nội bộ và gọi trước chuỗi kế tiếp để robot không khựng; dùng kết nối có dây cho kính khi thu dữ liệu; dành riêng GPU cho các đợt fine-tune.

II.01 Nghiên cứu mô hình

GR00T N1 và phiên bản N1.7 dùng trong dự án.

GR00T N1 là mô hình VLA mở cho robot hình người, gồm một mô hình thị giác–ngôn ngữ để **hiểu** và một Diffusion Transformer để **hành động**. Dự án dùng bản kế nhiệm **GR00T N1.7-3B**. Mục này đi lần lượt: model làm gì, gồm những gì, có hiệu quả không, rồi dự án áp dụng thế nào.

BÀI BÁO	MODEL DỰ ÁN	TỪ KHÓA
arXiv:2503.14734 (03/2025)	**GR00T N1.7-3B**	**VLA · flow matching · data pyramid**

1 Thuật ngữ nhanh

Token
Mảnh nhỏ của ảnh hoặc câu chữ sau khi mã hoá; model đọc chuỗi token thay vì ảnh, chữ gốc.

VLA (Vision-Language-Action)
Model nhìn ảnh, đọc câu lệnh và xuất ra hành động cho robot.

System 2 / System 1
Phần hiểu (chậm) và phần hành động (nhanh) của GR00T.

Action / chuỗi action
Một action là bộ góc khớp mục tiêu cho một bước; chuỗi action là đoạn chuyển động sắp tới.

Flow matching
Cách sinh chuỗi action: bắt đầu từ nhiễu ngẫu nhiên rồi chỉnh dần thành chuyển động.

Embodiment
Một loại thân robot (G1, GR-1…); mỗi loại có bộ mã hoá riêng.

Pre-train / fine-tune
Học trước trên dữ liệu lớn chung / tinh chỉnh thêm cho một robot, một tác vụ cụ thể.

Checkpoint
Bộ trọng số của model đã huấn luyện, lưu thành file để dùng lại.

Teleop
Người điều khiển robot từ xa (ở đây bằng kính thực tế ảo) để ghi demo.

2 Cơ chế vận hành

Video dưới đây minh hoạ cách một mô hình **Vision-Language-Action** như GR00T biến ảnh camera và câu lệnh thành chuyển động. Bấm vào từng bước bên phải để tua video tới đoạn tương ứng; mỗi bước có ghi chú

cách dự án áp dụng.



7 BƯỚC · BẤM ĐỂ TUA VIDEO

Từ ảnh và câu lệnh tới chuyển động

0:00 IMAGE TOKENS

Ảnh camera thành token hình ảnh

Ảnh được chia thành các ô nhỏ, qua bộ mã hoá thị giác (học từ cặp ảnh–chú thích) rồi chiếu sang không gian của mô hình ngôn ngữ.

> **Dự án:** ảnh camera đầu G1 640×480; GR00T N1.7 dùng bộ mã hoá SigLIP-2.

0:12 TEXT TOKENS

Câu lệnh thành token văn bản

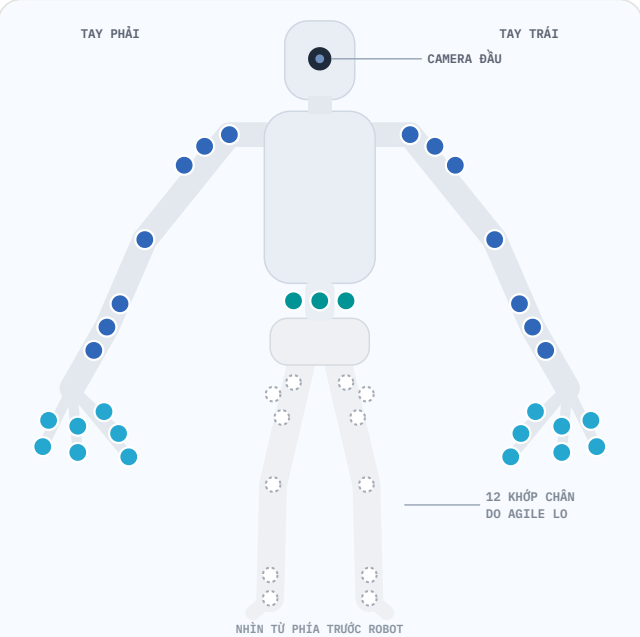
Câu lệnh như "pick up the dessert item" được tách thành từng mảnh từ.

> **Dự án:** ví dụ "put the hammer in the bin"; câu lệnh phải trùng nguyên văn lúc ghi dữ liệu.

3 Dữ liệu đầu vào

Mỗi lần gọi, model nhận ba loại dữ liệu của **đúng thời điểm hiện tại**: một ảnh camera, góc các khớp thân trên và câu lệnh. Model không nhận lịch sử các khung hình trước, cũng không thấy 12 khớp chân (phần chân do AGILE lo).

1 ảnh camera đầu 640×480, khung hình hiện tại	**31** góc khớp thân trên tay, ngón tay, eo	**1** câu lệnh giữ nguyên suốt episode



TRẠNG THÁI · ROBOT G1

Chấm xanh: 31 khớp được gửi lên model · chấm trắng: 12 khớp chân do AGILE điều khiển



GÓC NHÌN CỦA ROBOT · CAMERA ĐẦU

Khung đầu tiên của kịch bản 1

CÂU LỆNH GỬI ĐI

```
"Pick up the apple from the shelf and place
it onto the plate on the same shelf next
to it."
```

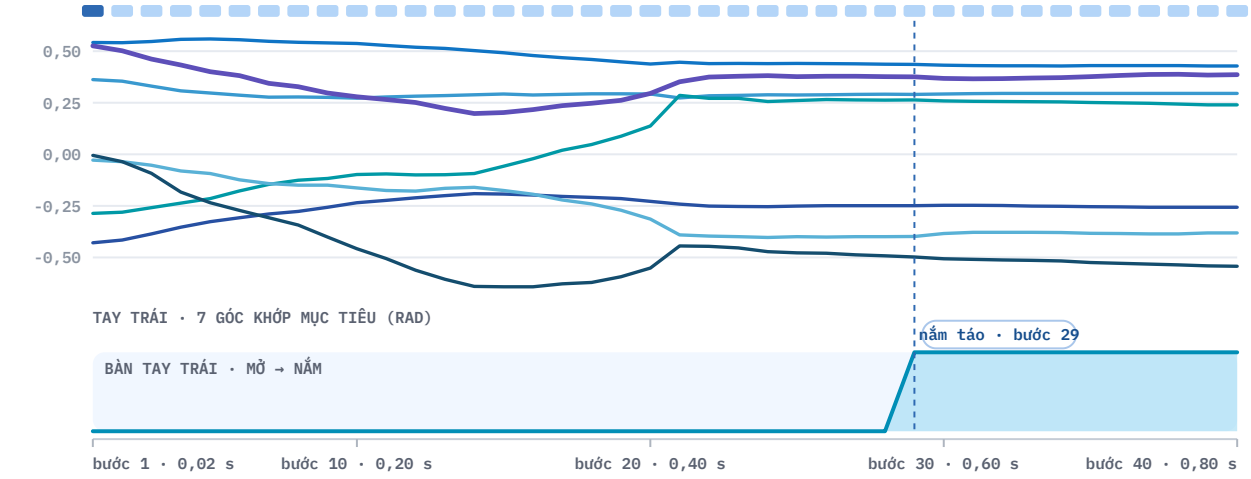
> **Trong dự án:** kịch bản 1 và 3 dùng camera `ego_view` như trên; kịch bản 2 (real2sim) dùng camera `rs_view` với cùng 31 khớp.

4 Dữ liệu đầu ra

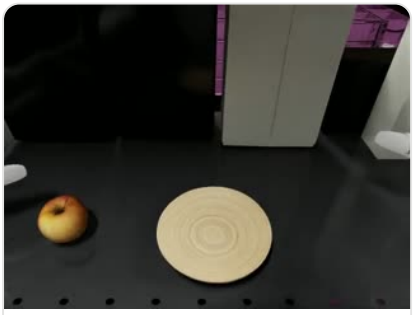
Mỗi lần gọi, model trả về **một chuỗi 40 bước** cho tương lai gần, mỗi bước 35 giá trị. Robot thực hiện một bước mỗi tick 20 ms, nên một chuỗi đủ cho 0,8 s chuyển động trước khi cần chuỗi mới.

40 **bước mỗi chuỗi** 20 ms mỗi bước = 0,8 s chuyển động	**35** **giá trị mỗi bước** 31 góc khớp + 4 lệnh cho chân	**1** **chuỗi mỗi lần gọi** gọi lại khi chuỗi sắp hết

VÍ DỤ · MỘT CHUỖI 40 BƯỚC TRONG DEMO TÁO → ĐĨA (EPISODE 0, TỪ GIÂY 0,4)



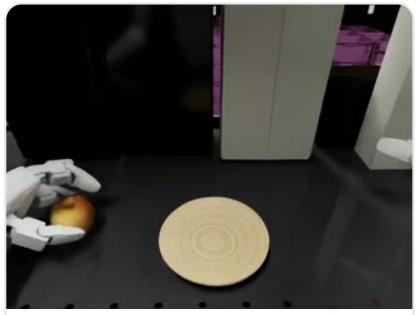
Vai pitch · Vai roll · Vai yaw · Khuỷu · Cổ tay roll · Cổ tay pitch · Cổ tay yaw · Độ nắm bàn tay trái



Bước 1 · tay tiến tới quả táo



Bước 21 · tay tới sát quả



Bước 40 · đã nắm quả

Đây là chuỗi action ghi sẵn trong dữ liệu demo, cùng dạng với chuỗi model trả về. Khi kiểm chứng open-loop, chuỗi model dự đoán cho cùng thời điểm lệch so với chuỗi này nhỏ hơn biên độ chuyển động 10–70 lần.

MỘT BƯỚC · 35 GIÁ TRỊ

Tay trái	Tay phải	Bàn tay trái	Bàn tay phải	Eo	Di chuyển	Độ cao
7	7	7	7	3	3	1

31 góc khớp mục tiêu → đưa thẳng xuống khớp · 4 lệnh → chuyển cho AGILE

GÓC TUYỆT ĐỐI

31 giá trị khớp là góc mục tiêu, không phải độ thay đổi so với bước trước. Trước khi dùng, client vẫn kẹp theo giới hạn khớp và giới hạn tốc độ mỗi tick (II.02, card 7).

LỆNH CHO CHÂN

Model không điều khiển trực tiếp 12 khớp chân. Nó chỉ đưa lệnh di chuyển và chiều cao hông (lệnh 0,72 m); AGILE dựa vào đó để giữ thăng bằng. Ba kịch bản của dự án đều đứng yên tại chỗ.

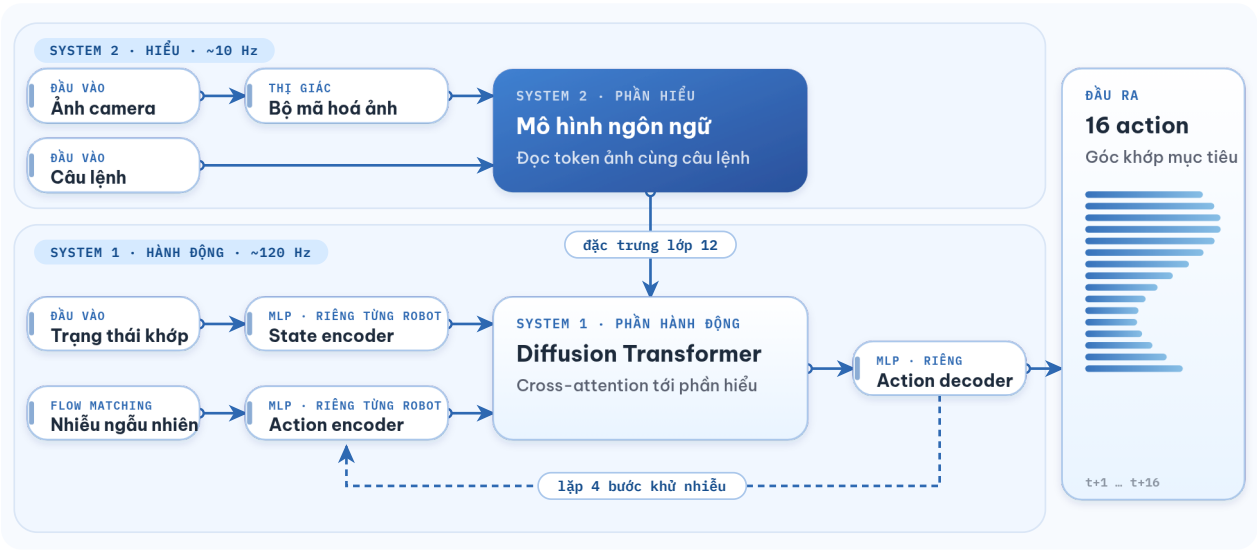
Trong dự án: kịch bản 1 và 3 dùng đúng chuỗi 40 bước ở 50 Hz như trên. Kịch bản 2 cố định hông nên không có lệnh cho chân: chuỗi 16 bước ở 20 Hz (cũng 0,8 s), mỗi bước 31 giá trị, mỗi lần gọi thực thi 8 bước.

5 Kiến trúc mô hình

Phần hiểu (bài báo gọi là **System 2**) là một mô hình thị giác–ngôn ngữ, chạy chậm, khoảng 10 lần mỗi giây. Phần hành động (**System 1**, trong video gọi là "action expert") nhận kết quả của phần hiểu cùng góc khớp hiện tại của robot, và chạy nhanh hơn nhiều. Bấm vào từng khối trong sơ đồ để xem nó làm gì.

LUỒNG DỮ LIỆU TRONG GR00T N1

SƠ ĐỒ · SỐ LIỆU TỪ BÀI BÁO



SYSTEM 2 · PHẦN HIỂU

Mô hình ngôn ngữ

Đọc cùng lúc token ảnh và câu lệnh để hiểu khung cảnh và yêu cầu. Đặc trưng ở lớp 12 được gửi sang phần hành động. Chạy khoảng 10 lần mỗi giây.

Trong dự án: N1.7 dùng Cosmos-Reason2-2B; phần này được đóng băng khi fine-tune.

Trong dự án: GR00T N1.7 giữ nguyên cấu trúc hai phần này; phần hiểu được thay bằng model mới hơn (card 7). Đầu vào là ảnh camera đầu G1 và 31 góc khớp thân trên. Ba khối MLP (state encoder, action encoder, action decoder) là riêng cho từng loại robot: đây là phần phải học khi đưa G1 vào model.

TỔNG THAM SỐ

2,2 tỷ

GR00T N1, bản trong bài báo

PHẦN HIỂU

1,34 tỷ

Phần lớn tham số nằm ở mô hình thị giác–ngôn ngữ

TỐC ĐỘ

10 · 120 Hz

Phần hiểu ~10 Hz, vòng điều khiển ~120 Hz

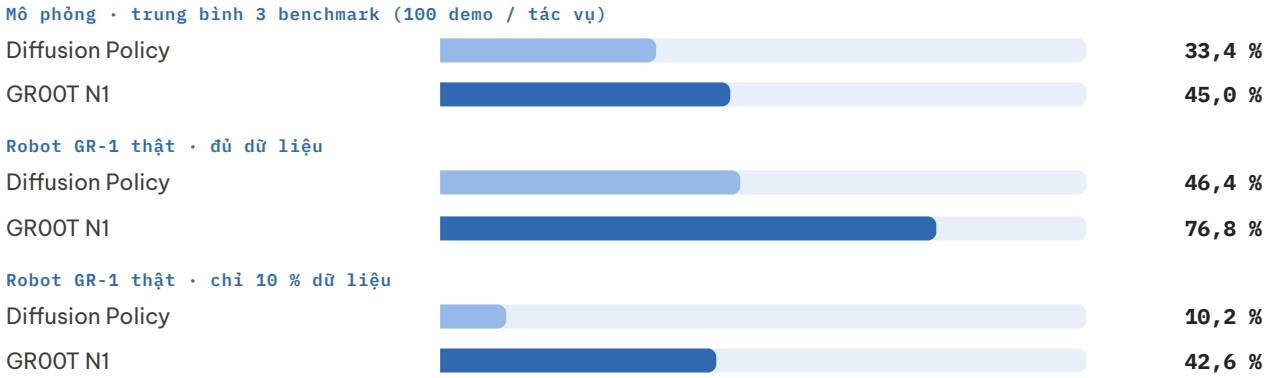
Chi tiết kỹ thuật · bài báo –

- Phần hiểu: Eagle-2, gồm bộ mã hoá ảnh SigLIP-2 và mô hình ngôn ngữ SmolLM2. Ảnh 224×224 được nén còn 64 token (pixel shuffle).
- Đặc trưng lấy ở lớp 12 của mô hình ngôn ngữ thay vì lớp cuối: bài báo ghi nhận vừa nhanh hơn vừa thành công cao hơn.
- Phần hành động: Diffusion Transformer (DiT), xen kẽ cross-attention (nhìn token của phần hiểu) và self-attention (giữa token trạng thái và token action).
- Trạng thái và action đi qua bộ mã hoá/giải mã MLP riêng cho từng loại robot (embodiment).

6 Kết quả công bố

Bài báo so GR00T N1 với Diffusion Policy, một model chỉ học từ dữ liệu của chính tác vụ, không học trước. GR00T N1 tốt hơn ở mọi cấu hình, cách biệt lớn nhất khi ít dữ liệu.

Tỉ lệ thành công: Diffusion Policy so với GR00T N1 số liệu bài báo



> **Trong dự án:** đây là lý do vài trăm demo đủ để fine-tune một tác vụ mới cho G1, và dữ liệu real2sim, dữ liệu sinh thêm là đủ dùng.

Bảng chi tiết theo benchmark mô phỏng · bảng 4 bài báo –

BENCHMARK	DIFFUSION POLICY 30 / 100 / 300 DEMO	GR00T-N1-2B 30 / 100 / 300 DEMO
RoboCasa (24 tác vụ)	14,7 / 25,6 / 43,2	**17,4 / 32,1 / 49,6**
DexMimicGen (9 tác vụ)	23,7 / 46,9 / 68,4	**29,6 / 58,5 / 74,2**
GR-1 Tabletop (24 tác vụ)	21,3 / 32,7 / 40,4	**43,2 / 50,0 / 49,3**

Tỉ lệ thành công trung bình (%). GR00T N1 dẫn ở mọi ô; riêng GR-1 Tabletop gần như không tăng thêm từ 100 lên 300 demo.

7 Đối chiếu phiên bản

Các số liệu ở card 6 là của N1 trên robot Fourier GR-1. Mọi thí nghiệm của dự án dùng N1.7 trên Unitree G1.

KHÍA CẠNH	GR00T N1 (BÀI BÁO)	GR00T N1.7 (DỰ ÁN)
Quy mô	~2,2 tỷ tham số	~3 tỷ tham số
Phần hiểu (System 2)	NVIDIA Eagle-2	Cosmos-Reason2-2B (kiến trúc Qwen3-VL), ảnh theo tỉ lệ gốc
Phần hành động (System 1)	DiT + flow matching	Giữ nguyên
Dùng cho nhiều robot	MLP riêng cho từng robot	Giữ nguyên; trạng thái được đệm tới độ dài cố định

8 Checkpoint thực nghiệm

Ba checkpoint dùng trong ba kịch bản đều xuất phát từ GR00T N1.7-3B; điểm khác nhau nằm ở ai fine-tune và fine-tune bằng dữ liệu gì.

CHECKPOINT	TÁC VỤ	NGUỒN	DỮ LIỆU FINE-TUNE	THÀNH CÔNG
Kịch bản 1 `GN1x-Tuned-Arena-G1-Static-PickNPlace`	Táo → đĩa	NVIDIA, dùng lại	208 demo teleop trong mô phỏng	92 % Arena gốc 56 % Galileo dựng lại 0 % khung cảnh tự dựng
Kịch bản 2 `g1_fruits_n17`	Chọn đúng 1 trong 4 loại quả	Dự án fine-tune	790 episode real2sim từ dữ liệu G1 thật	66 % (33/50)
Kịch bản 3 `hammer_mix200_n17`	Búa → thùng	Dự án fine-tune	200 episode: teleop + sinh thêm	66 % (66/100)
Kịch bản 3 `hammer_mix400_n17`	Búa → thùng	Dự án fine-tune	400 episode: mix200 + 200 demo teleop mới	63 % (63/100); 73 % khi nới ngưỡng đứng yên

> **Giấy phép:** N1.7 theo NVIDIA Open Model Agreement; checkpoint Arena ghi "non-commercial use".

9 Kết luận nghiên cứu

Những gì dự án rút ra sau khi dùng GR00T N1.7 trên cả ba kịch bản.

ĐIỂM MẠNH

Dạy tác vụ mới nhanh, ít dữ liệu

- Vài trăm demo là đủ: bài búa từ 0 % lên 66–73 % với 200–400 episode (mục III.03).
- Fine-tune ~1 giờ trên một H100, chạy được trên một GPU.
- Hiểu câu lệnh tự nhiên: model 4 loại quả chọn đúng quả theo lệnh, không lần nào gắp nhầm (mục III.02).
- Một model điều khiển cùng lúc tay, bàn tay và eo.

GIỚI HẠN

Nhạy với hình ảnh, phải tune từng tác vụ

- Phụ thuộc khung cảnh: bài táo từ 92 % xuống 56 % khi lệch ánh sáng, 0 % khi lệch cả khung cảnh (mục III.01).
- Không dùng lại được cho tác vụ khác: checkpoint táo đạt 0 % trên bài 4 loại quả.
- Mỗi lần gọi ~1,05–1,3 s qua mạng (model tính ~100 ms), lâu hơn một chuỗi chuyển động 0,8 s (mục I.02).
- Checkpoint Arena của NVIDIA ghi "non-commercial use".

Kết luận: GR00T phù hợp làm nền cho các tác vụ gắp–đặt của G1. Yếu tố quyết định kết quả không phải model mà là dữ liệu: thu đúng môi trường làm việc, hoặc đủ đa dạng về hình ảnh.

Tái hiện từng bước của luồng GR00T ↔ G1 bằng notebook nghiên cứu.

Để không phụ thuộc vào khung có sẵn của NVIDIA, dự án **tự xây lại toàn bộ luồng** từ ảnh camera tới lệnh khớp. Mỗi bước được nghiên cứu trong một **notebook** riêng — viết tay, quan sát dữ liệu, đối chiếu với tham chiếu — rồi mới đóng gói thành mã nguồn có kiểm thử. Nhãn `nbXX` trong ô XỬ LÝ của mỗi card là notebook (trong `codes/standalone_g1_gr00t/notebooks/`) nơi bước đó được viết và kiểm chứng. Gói bàn giao có 12 notebook: 7 notebook dựng pipeline được trình bày ở mục này, 5 notebook còn lại dùng pipeline đó để kiểm chứng và đánh giá các kịch bản ở phần III. Card 10 liệt kê đủ 12 notebook và vị trí của từng notebook trong báo cáo.

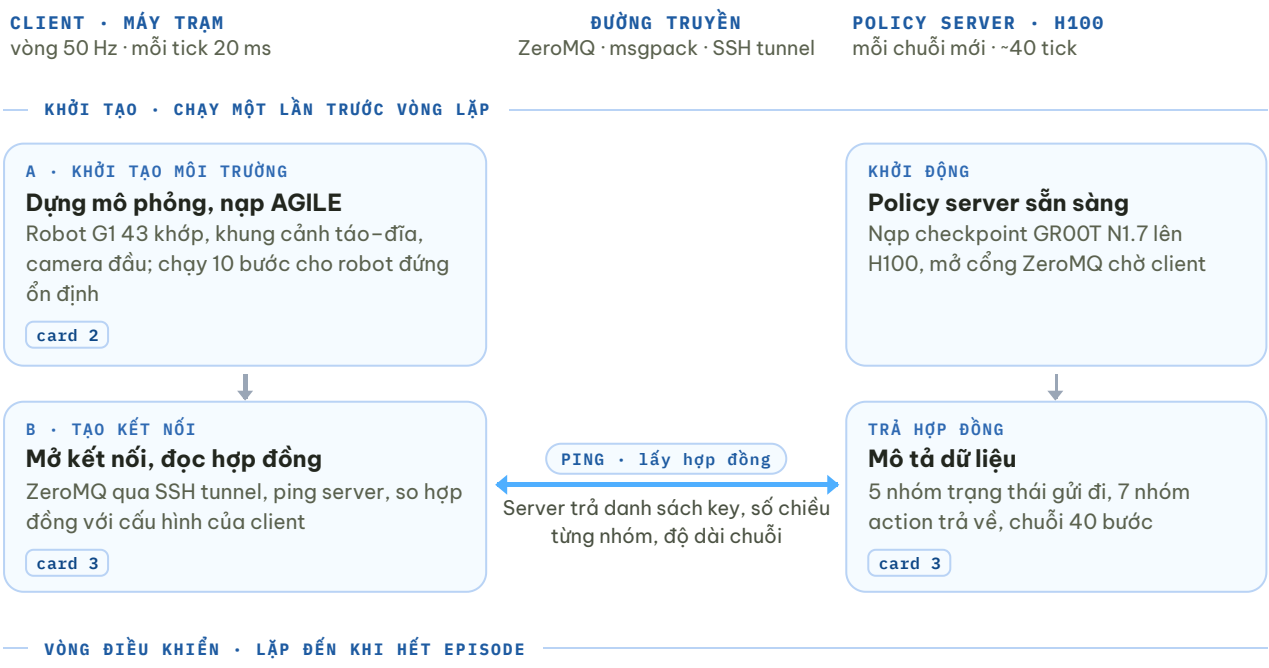
NOTEBOOK
12 notebook · 7 ở mục này, 5 ở phần III

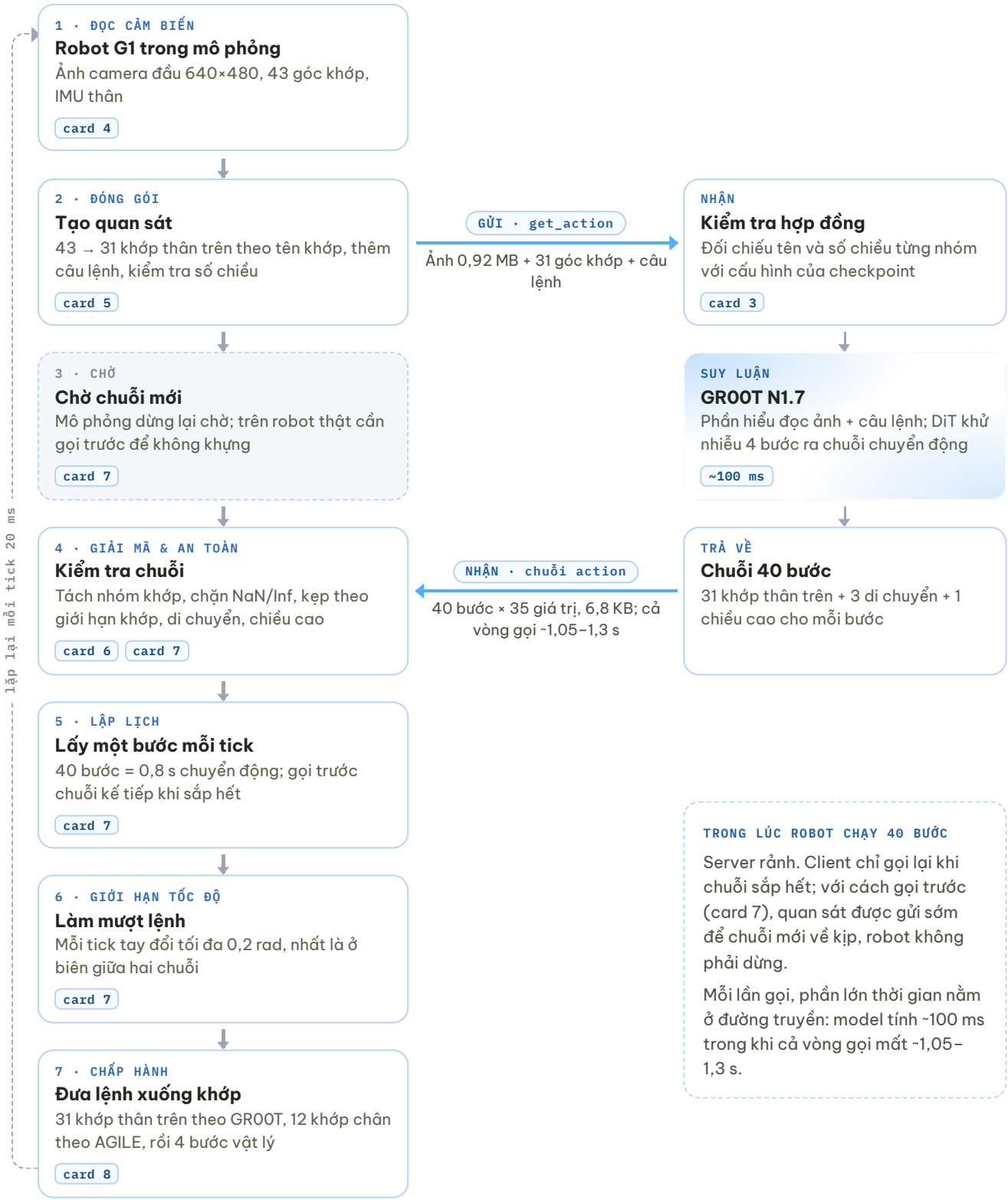
MÃ NGUỒN
Package g1gr00t · 84 unit test

TỪ KHÓA
notebook → package · kiểm chứng theo tầng

1 Luồng tổng thể

Trước khi chạy, client dựng môi trường mô phỏng và kết nối tới server một lần. Sau đó mỗi vòng điều khiển đi qua hai máy. **Client** trên máy trạm chạy mô phỏng và mọi bước xử lý dữ liệu: đọc cảm biến, đóng gói, giải mã, kiểm tra an toàn, đưa lệnh xuống khớp. **Policy server** trên GPU H100 chỉ chạy model. Client chạy mỗi 20 ms, nhưng chỉ gọi server khi cần chuỗi mới, tức khoảng 40 tick một lần. Nhãn card cho biết bước đó được trình bày ở card nào.




1 · ĐỌC CẢM BIẾN
Robot G1 trong mô phỏng
Ảnh camera đầu 640×480, 43 góc khớp, IMU thân
card 4
2 · ĐÓNG GÓI
Tạo quan sát
43 → 31 khớp thân trên theo tên khớp, thêm câu lệnh, kiểm tra số chiều
card 5
GỬI · get_action
Ảnh 0,92 MB + 31 góc khớp + câu lệnh
NHẬN
Kiểm tra hợp đồng
Đối chiếu tên và số chiều từng nhóm với cấu hình của checkpoint
card 3
3 · CHỜ
Chờ chuỗi mới
Mô phỏng dừng lại chờ; trên robot thật cần gọi trước để không khựng
card 7
SUY LUẬN
GR00T N1.7
Phần hiểu đọc ảnh + câu lệnh; DiT khử nhiễu 4 bước ra chuỗi chuyển động
~100 ms
4 · GIẢI MÃ & AN TOÀN
Kiểm tra chuỗi
Tách nhóm khớp, chặn NaN/Inf, kẹp theo giới hạn khớp, di chuyển, chiều cao
card 6
card 7
NHẬN · chuỗi action
40 bước × 35 giá trị, 6,8 KB; cả vòng gọi ~1,05–1,3 s
TRẢ VỀ
Chuỗi 40 bước
31 khớp thân trên + 3 di chuyển + 1 chiều cao cho mỗi bước
lặp lại mỗi tick 20 ms
5 · LẬP LỊCH
Lấy một bước mỗi tick
40 bước = 0,8 s chuyển động; gọi trước chuỗi kế tiếp khi sắp hết
card 7
6 · GIỚI HẠN TỐC ĐỘ
Làm mượt lệnh
Mỗi tick tay đổi tối đa 0,2 rad, nhất là ở biên giữa hai chuỗi
card 7
7 · CHẤP HÀNH
Đưa lệnh xuống khớp
31 khớp thân trên theo GR00T, 12 khớp chân theo AGILE, rồi 4 bước vật lý
card 8
TRONG LÚC ROBOT CHẠY 40 BƯỚC
Server rảnh. Client chỉ gọi lại khi chuỗi sắp hết; với cách gọi trước (card 7), quan sát được gửi sớm để chuỗi mới về kịp, robot không phải dừng.
Mỗi lần gọi, phần lớn thời gian nằm ở đường truyền: model tính ~100 ms trong khi cả vòng gọi mất ~1,05–1,3 s.
TRƯỚC ĐÓ · OFFLINE
Demo → dataset LeRobot → fine-tune trên H100 → checkpoint nạp vào policy server.
SAU MỖI EPISODE
Reset rồi chạy episode tiếp; gom kết quả N episode thành tỉ lệ thành công (card 9).

2 Khởi tạo môi trường

ĐẦU VÀO

- Robot G1 (USD, 43 khớp)
- Khung cảnh, táo, đĩa
- Policy AGILE (ONNX)

XỬ LÝ nb05 nb04

- Dựng mô phỏng: nền, ánh sáng → robot → camera đầu → vật thể → ma sát ngón tay
- Ghi trạng thái ban đầu, chạy 10 bước cho ổn định
- Nạp AGILE, reset trạng thái LSTM

ĐẦU RA

- Robot đứng ở tư thế mặc định
- Bước vật lý 5 ms, tick điều khiển 20 ms

KẾT QUẢ ĐẠT ĐƯỢC

ROBOT ỔN ĐỊNH
0,786 m
chiều cao hông sau 10 bước, rồi giữ yên

THỨ TỰ KHỚP
43 / 43
khớp của mô phỏng trùng bảng thứ tự đã lập

DỰNG KHUNG CẢNH
~4 s
sau khi tải sẵn 1,3 GB asset (lần đầu > 15 phút)

3 Tạo kết nối

ĐẦU VÀO

- Địa chỉ policy server (qua SSH tunnel)
- Checkpoint đã nạp trên H100

XỬ LÝ nb01

- Mở kết nối ZeroMQ, ping server
- Đọc hợp đồng dữ liệu từ server
- So với cấu hình của client trước khi chạy

ĐẦU RA

- Kết nối sẵn sàng
- Danh sách key, số chiều từng nhóm, độ dài chuỗi

KẾT QUẢ ĐẠT ĐƯỢC

HỢP ĐỒNG ĐỌC TỪ SERVER
5 → 7 nhóm
5 nhóm trạng thái gửi đi, 7 nhóm action trả về

ĐỘ DÀI CHUỖI
40 bước
= 0,8 s chuyển động ở 50 Hz

CỔNG KHÔNG PHẢN HỒI
0,5 s
trả lỗi sạch, tạo lại kết nối

4 Lấy quan sát

ĐẦU VÀO

- Trạng thái mô phỏng sau tick vừa chạy

XỬ LÝ nb05 nb02

- Render camera đầu
- Đọc 43 góc và vận tốc khớp, IMU thân
- Sắp lại khớp theo tên giữa 4 hệ thứ tự

ĐẦU RA

- Ảnh 640×480
- 43 góc khớp + IMU (cho AGILE)
- 31 góc khớp thân trên (cho GR00T)

KẾT QUẢ ĐẠT ĐƯỢC

HỆ THỨ TỰ KHỚP
4 hệ
mô phỏng, WBC, model, dataset

CHUYỂN ĐỔI QUA LẠI
0 sai lệch
mô phỏng → WBC → mô phỏng không mất giá trị

ẢNH MỚI
1 mỗi tick
render ở bước vật lý cuối; thiếu bước này model thấy ảnh đứng yên

5 Gửi quan sát

ĐẦU VÀO
- Ảnh 640×480
- 31 góc khớp thân trên
- Câu lệnh

XỬ LÝ nb02 nb01
- Đóng gói theo hợp đồng, kiểm tra số chiều
- Gửi get_action qua ZeroMQ (msgpack)
- Server chạy GR00T N1.7

ĐẦU RA
- Yêu cầu 0,92 MB lên server
- Server sinh chuỗi 40 bước

KẾT QUẢ ĐẠT ĐƯỢC

ĐỘ TRỄ MỖI LẦN GỌI
~1,05 s
1 robot · p95 1,39 s

MODEL TÍNH
~100 ms
thời gian tính trên server; phần lớn còn lại nằm ở đường truyền

4 ROBOT CÙNG LÚC
~3,6 s
gửi chung một lô · p95 4,2 s

Câu lệnh gửi đi ảnh hưởng tới chuỗi trả về — sai số tay trái, rad



Câu mặc định của Arena cho sai số lớn hơn cả câu vô nghĩa, nên client gửi nguyên văn câu lệnh của dataset.

6 Nhận chuỗi action

ĐẦU VÀO
- Trả lời của server: 7 key, 6,8 KB

XỬ LÝ nb01 nb02
- Giải mã msgpack
- Tách 31 góc khớp (40×31), di chuyển (40×3), chiều cao (40×1)
- Xếp về thứ tự khớp của client theo tên

ĐẦU RA
- Chuỗi 40 bước × 35 giá trị trong client

KẾT QUẢ ĐẠT ĐƯỢC

MỖI LẦN NHẬN
40 × 35 giá trị
40 bước, mỗi bước 35 giá trị

KÍCH THƯỚC
6,8 KB
nhỏ hơn yêu cầu gửi đi ~135 lần

NHÓM TRẢ VỀ
7 key

2 tay, 2 bàn tay, eo, di chuyển, chiều cao

7 Xử lý chuỗi action

ĐẦU VÀO

- Chuỗi 40 × 35 vừa nhận

XỬ LÝ nb08 nb02

- Chặn NaN/Inf, kẹp theo giới hạn khớp, di chuyển, chiều cao
- Lập lịch: mỗi tick lấy một bước, gọi trước khi sắp hết
- Giới hạn thay đổi mỗi tick

ĐẦU RA

- **Lệnh an toàn cho tick hiện tại: 31 góc khớp + 4 lệnh cho chân**

KẾT QUẢ ĐẠT ĐƯỢC

NGOÀI GIỚI HẠN
0 / 1.240
giá trị model trả về vượt giới hạn khớp, trong một chuỗi thử

PHÉP THỬ CỐ Ý
81 giá trị bị kẹp
1 góc khớp, 40 lệnh di chuyển, 40 lệnh chiều cao

THỜI GIAN CHỜ
0,28 s
khi gọi trước, so với 0,84 s gọi tuần tự

8 Chạy robot

ĐẦU VÀO

- 31 góc khớp mục tiêu
- Lệnh di chuyển + chiều cao
- Quan sát hiện tại (cho AGILE)

XỬ LÝ nb04 nb05

- AGILE tính 12 góc khớp chân từ vector 80 số
- Ghi 43 góc mục tiêu xuống mô phỏng
- Chạy 4 bước vật lý 5 ms

ĐẦU RA

- **Robot chuyển động một tick (20 ms)**
- **Trạng thái mới cho tick sau**

KẾT QUẢ ĐẠT ĐƯỢC

AGILE SO VỚI THAM CHIẾU
50 tick
trùng từng bit, cùng trạng thái LSTM

ĐỨNG VỮNG
10 s
500 tick, nghiêng tối đa 9,8°

PHÁT LẠI DEMO
121 tick
không dùng model: thành công, táo cách tâm đĩa 2,9 cm

9 Chạy vòng kín

ĐẦU VÀO

- Checkpoint, khung cảnh, câu lệnh
- Các bước ở card 4–8

XỬ LÝ nb06 nb07 nb08

- Mỗi tick: lấy quan sát → xử lý → chạy robot
- ~40 tick một lần: gửi quan sát, nhận chuỗi mới
- Kết thúc khi thành công, ngã hoặc hết giờ; reset rồi chạy episode tiếp

ĐẦU RA

- Kết quả từng episode
- Log, video, thống kê

KẾT QUẢ ĐẠT ĐƯỢC

Tỉ lệ thành công: cùng model, chỉ đổi khung cảnh — 50 episode



Một episode thành công khoảng 120 tick, tương ứng đúng 3 lần gọi server. Ở khung cảnh tự dựng robot vẫn nhấc được táo 11/50 lần nhưng không đặt lên đĩa; khoảng cách táo–đĩa nhỏ nhất (trung vị) giảm từ 20,6 cm xuống 3,8 cm khi dựng lại khung cảnh Galileo.

10 Danh mục 12 notebook và vị trí trong báo cáo

nb01, nb02 và nb04–nb08 dựng và kiểm chứng từng khối của pipeline ở card 2–9; nb03 kiểm chứng model với dữ liệu thật mà chưa cần mô phỏng; nb09–nb12 áp dụng pipeline cho kịch bản 2 và 3. Mỗi kịch bản có một notebook dựng khung cảnh và chạy thử một episode, kèm một notebook chạy N episode để tính tỉ lệ thành công.

NOTEBOOK	NỘI DUNG	TRÌNH BÀY Ở	MÔI TRƯỜNG
Khối của pipeline · mục II.02			
`nb01_server_wire`	Kết nối ZeroMQ + msgpack, đọc hợp đồng dữ liệu, đo độ trễ, xử lý mất kết nối	Card 3, 5, 6	`.venv` · cần server
`nb02_joints_obs_action`	4 hệ thứ tự khớp, đóng gói quan sát, giải mã chuỗi action, lập lịch chuỗi	Card 4–7	`.venv` · cần server
`nb04_agile_onnx`	Policy chân AGILE: vector quan sát 80 chiều, đối chiếu từng bit với tham chiếu	Card 2, 8	`.venv`
`nb05_backend_anatomy`	Dựng tay backend Isaac Sim: robot, camera đầu, táo/đĩa, ma sát, đọc và ghi khớp	Card 2, 4, 8	Isaac Sim
`nb08_runner_policy_safety`	Lớp an toàn (chặn NaN, kẹp giới hạn, giới hạn tốc độ), runner, ghi log	Card 7, 9	Isaac Sim

NOTEBOOK	NỘI DUNG	TRÌNH BÀY Ở	MÔI TRƯỜNG
Vòng kín · mục II.02 card 9 và kịch bản 1			
`nb06_full_pipeline`	Ghép đủ các khối thành vòng kín trên khung cảnh tự dựng: đứng vững, phát lại demo, chạy model	Card 9 · III.01	Isaac Sim · cần server
`nb07_full_pipeline_loop`	Dựng lại khung cảnh Galileo, chạy 50 episode: 0/50 → 28/50	Card 9 · III.01 · IV.04	Isaac Sim · cần server
Kiểm chứng open-loop · kịch bản 1			
`nb03_open_loop_dataset`	Gửi ảnh và trạng thái thật từ dataset lên server, so chuỗi trả về với action ghi sẵn	III.01 bước 3	`.venv` · cần server
Áp dụng cho kịch bản 2 và 3 · phần III			
`nb09_multi_object_task_pipeline`	Khung cảnh real2sim 4 loại quả, chọn câu lệnh, chạy một episode	III.02	Isaac Sim · cần server
`nb10_eval_multi_object`	Khung cảnh nb09 chạy N episode: 66 %	III.02 · IV.04	Isaac Sim · cần server
`nb11_hammer_to_bin`	Khung cảnh búa → thùng của Arena dựng lại trong Isaac Sim, chạy một episode	III.03	Isaac Sim · cần server
`nb12_eval_hammer_to_bin`	Khung cảnh nb11 chạy N episode, phân loại thất bại: 66 % (mix200), 73 % (mix400)	III.03 · IV.04	Isaac Sim · cần server

Phần lớn mã trong notebook đã được đóng gói vào package `g1gr00t`; các notebook khối (nb01, nb04, nb05, nb08) có cell đối chiếu phần viết tay với package. Cách chạy lại notebook: mục IV.04.

Từ góc khớp tới chuyển động và lực.

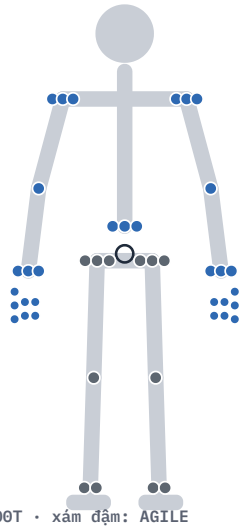
GR00T chỉ xuất ra **góc khớp mục tiêu**. Biến các con số đó thành chuyển động thật cần hiểu **động học** (góc khớp ↔ vị trí tay, camera) và **động lực học** (mô-men, quán tính, tiếp xúc, thăng bằng). Phần này tổng hợp các kiến thức đó và nơi chúng xuất hiện trong dự án; mỗi khái niệm có một khung minh hoạ để thử.

ROBOT	ÁP DỤNG	TỪ KHÓA
Unitree G1 · 43 khớp	**Real2sim · teleop · điều khiển khớp · thăng bằng**	**FK · IK · PD · tiếp xúc**

1 Cấu trúc khớp G1

G1 là một cây gồm các khâu nối với nhau bằng khớp quay, gốc là xương chậu (pelvis). Mỗi nhóm khớp do một thành phần khác nhau điều khiển. Bấm vào từng nhóm để xem.

43 KHỚP CỦA G1 · NHÌN TỪ PHÍA TRƯỚC — SƠ ĐỒ MINH HOẠ



xanh: GR00T · xám đậm: AGILE
vòng tròn: gốc (xương chậu)

12	**Chân** Hông (3), gối, cổ chân (2) mỗi bên	AGILE
3	**Eo** Xoay, nghiêng ngang, gập	GR00T
14	**Tay** Vai (3), khuỷu, cổ tay (3) mỗi bên	GR00T
14	**Bàn tay Dex3** Ngón trỏ (2), giữa (2), cái (3) mỗi bên	GR00T

GR00T điều khiển 31 khớp thân trên (eo gần như cố định trong các tác vụ); AGILE lo 12 khớp chân.

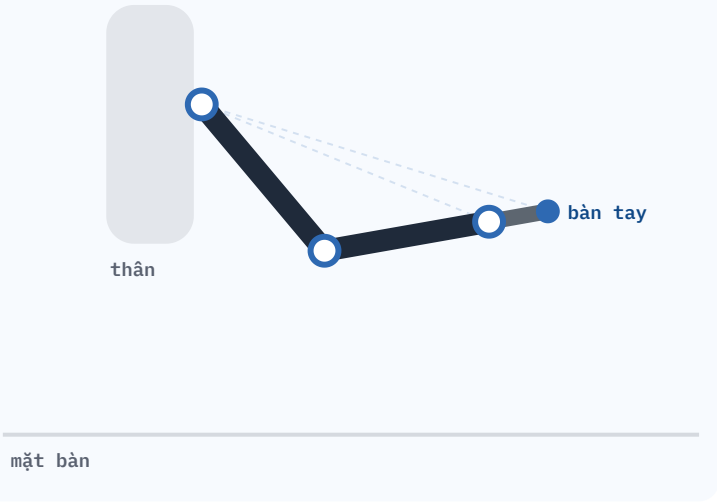
Bốn thứ tự khớp khác nhau cùng tồn tại: thứ tự trong mô phỏng (duyệt cây USD), thứ tự của bộ điều khiển toàn thân, thứ tự nhóm của GR00T và thứ tự đầu vào của AGILE. Chỉ cần lệch một cột là robot ngã hoặc model "mù", nên dự án ánh xạ mọi thứ theo **tên khớp** và kiểm thử tự động.

2 Động học thuận

Động học thuận (FK) tính vị trí của từng khâu (bàn tay, đầu ngón, camera) từ các góc khớp, bằng cách nhân lần lượt các phép biến đổi dọc cây khớp mô tả trong file URDF. Kéo các thanh trượt để thấy: biết góc khớp là biết chính xác bàn tay ở đâu.

CÁNH TAY NHÌN NGANG · 3 KHỚP

MINH HOẠ · ĐỘ DÀI KHÂU GẦN ĐÚNG



KẾT QUẢ FK

$T_{tay} = T_{vai}(0{,}70\ \text{rad}) \cdot T_{khuỷu}(1{,}05\ \text{rad}) \cdot T_{cổ\ tay}(0{,}00\ \text{rad})$

Bàn tay phía trước vai	**52,2 cm**
Bàn tay thấp hơn vai	**16,1 cm**

Mỗi khớp thêm một phép quay và một đoạn dịch; nhân lần lượt từ vai ra ngoài là ra vị trí bàn tay. Robot thật làm vậy trong 3D, cho mọi khâu.

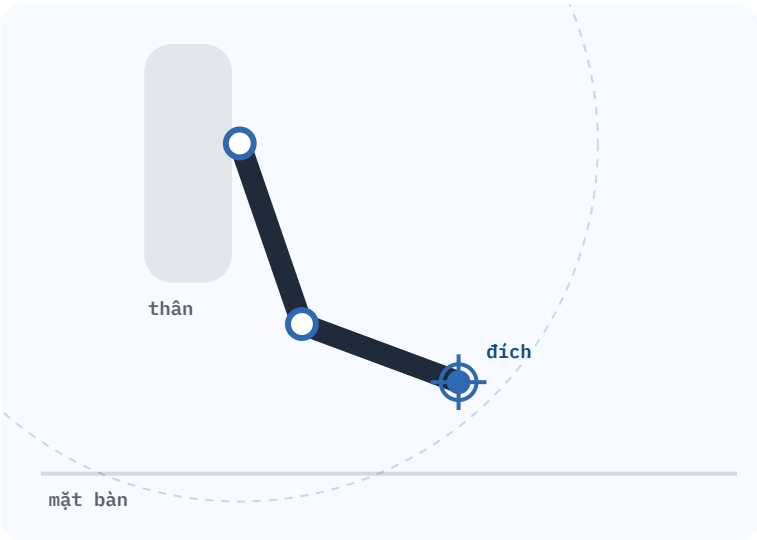
Ứng dụng trong real2sim: dataset G1 thật không có vị trí đồ vật. Bộ FK tự viết (chỉ dùng numpy) tính quỹ đạo lòng bàn tay từ góc khớp; tay khép ở đâu thì quả ở đó, tay mở ở đâu thì đĩa ở đó. Tư thế thân suy ra từ giả định hai bàn chân đặt phẳng trên sàn. Xem khung minh hoạ đầy đủ ở mục II.04.

3 Động học ngược

Động học ngược (IK) làm ngược lại: cho trước vị trí mong muốn của cổ tay, tìm góc khớp đạt được nó. Khi teleop bằng kính Meta Quest, tư thế tay cầm thành đích cổ tay robot, rồi bộ giải IK **Pink** tính góc khớp ở mỗi khung hình. Bấm hoặc kéo trong khung để đặt đích; thử đặt ngoài tầm với.

BẤM / KÉO ĐỂ ĐẶT ĐÍCH CỔ TAY

MINH HOẠ · IK 2 KHỚP



LỜI GIẢI IK

Góc vai	0,33 rad
Góc khuỷu	0,88 rad
Khoảng cách tới đích	48,8 cm

✓ Đạt đích: cổ tay trùng điểm đích

Ứng dụng trong dự án: IK chỉ dùng ở khâu **tạo dữ liệu** cho bài búa, không dùng khi model chạy.

- **Teleop bằng kính Meta Quest** (mục II.04, III.03): tư thế tay cầm được đổi thành đích cổ tay trong hệ toạ độ xương chậu, Pink giải ra góc khớp tay ở mỗi bước điều khiển (embodiment `g1_wbc_agile_pink`); IK chiếm ~7 % thời gian mỗi vòng lặp khi ghi (mục I.02). Đích ngoài tầm với thì IK kẹp ở giới hạn khớp và tay đứng im, không báo lỗi, đúng như khung minh hoạ ở trên.
- **Sinh thêm demo** (mục II.04): script dời các điểm xuất phát, cầm, thả của quỹ đạo cổ tay lấy từ demo teleop, rồi gửi đúng loại lệnh như kính; Pink đổi chúng thành góc khớp. 100 demo sinh thêm của mix200 đều đi qua bước này.
- **Khi đánh giá**, GR00T trả thẳng góc khớp (embodiment `g1_wbc_agile_joint`), nên không cần IK: dataset lưu góc khớp mà IK đã tính lúc ghi, và model học trực tiếp các góc đó. Kịch bản real2sim (mục III.02) cũng không dùng IK vì góc khớp có sẵn trong dữ liệu robot thật; ở đó dùng FK (card 2).

4 Động lực học

Động học chỉ nói **tay ở đâu** khi biết góc khớp. **Động lực học** trả lời câu hỏi tiếp theo: làm sao để khớp **thực sự quay tới** góc đó. GR00T chỉ gửi **góc mục tiêu**, không gửi lực; motor ở mỗi khớp phải tự tạo ra mô-men

xoắn đủ để kéo khâu (có khối lượng, quán tính) tới đích mà không rung.

1 · MỤC TIÊU
Góc khớp cần đạt
từ GR00T, mỗi 20 ms

2 · BỘ PD
Tính mô-men xoắn
dựa trên sai lệch góc và vận tốc

3 · VẬT LÝ
Khớp quay
mô-men ÷ quán tính = gia tốc góc

4 · ĐO LẠI
Góc mới
lặp lại mỗi bước vật lý 5 ms

P · ĐỘ CỨNG (K_p)

Như một lò xo

Lệch đích càng xa, lực kéo về càng mạnh. Lò xo càng cứng thì khớp bám đích càng nhanh, nhưng dễ vọt quá rồi rung qua lại.

D · GIẢM CHẤN (K_d)

Như một bộ giảm xóc

Khớp quay càng nhanh, lực hãm càng lớn. Đủ giảm chấn thì khớp tới đích êm, không vọt lố; quá nhiều thì tới đích chậm.

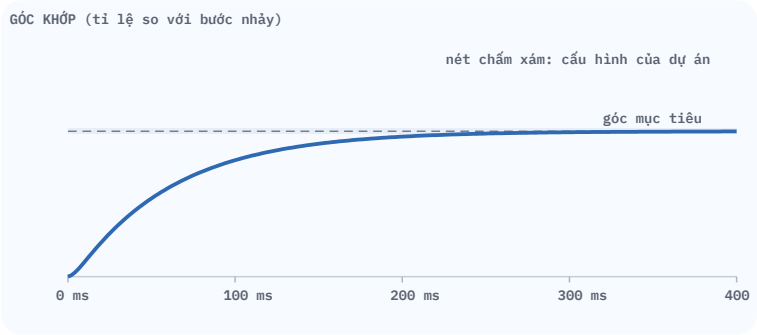
$$\text{mô-men} = K_p \times (\text{góc mục tiêu} - \text{góc hiện tại}) - K_d \times \text{vận tốc khớp}$$

(K_p × (góc mục tiêu − góc hiện tại): lò xo kéo về đích; K_d × vận tốc khớp: giảm xóc hãm lại)

Trong dự án: K_p, K_d không chọn tay mà suy ra từ quán tính motor theo hai thông số dễ hiểu: **tần số riêng 10 Hz** (độ nhanh) và **hệ số giảm chấn ζ = 2** (độ êm). Kết quả: khớp tới 90 % đích sau ~140 ms và không vọt lố. Kéo thanh trượt bên dưới để so sánh.

KHỚP NHẬN LỆNH NHẢY GÓC · GÓC THỰC TẾ THEO THỜI GIAN

MINH HOẠ · HỆ BẬC HAI LÝ TƯỞNG



ĐÁP ỨNG

Tới 90 % đích	**141 ms**
Vọt lố	**0 %**
Ổn định trong ±2 %	**237 ms**

ζ ≥ 1 · giảm xóc mạnh: tới đích êm, không vọt lố. Tần số riêng cao thì bám nhanh hơn nhưng motor phải tạo mô-men lớn hơn; dự án chọn ζ = 2 để tay không rung.

Ai tính mô-men PD trong mô phỏng?

CÁCH 1 · PD ẨN (IMPLICIT)

Bộ giải vật lý tự tính

PhysX tính lực PD ngay trong bước giải. Rất ổn định kể cả khi khớp cứng; là mặc định của Isaac Lab.

CÁCH 2 · PD TƯỜNG MINH (IDEAL PD)

Code tính rồi đặt mô-men vào

Dùng đúng công thức trên ở mỗi bước, giống driver của robot thật hơn. Arena dùng cách này cho G1.

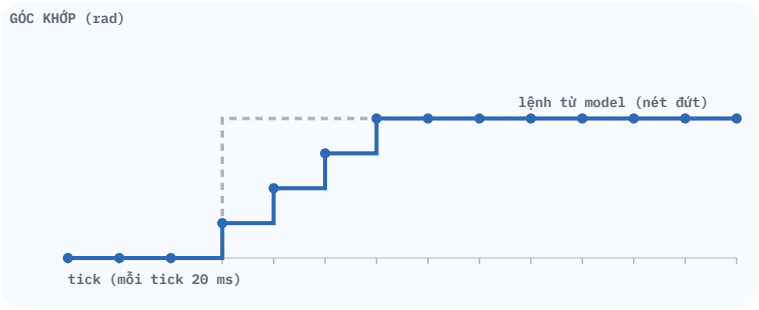
Vì sao cần để ý: cùng một chuỗi góc mục tiêu, hai cách cho chuyển động tay hơi khác nhau. Model học từ demo ghi ở cách nào thì khi đánh giá cũng nên dùng đúng cách đó.

5 Giới hạn tốc độ khớp

Lớp an toàn chặn bước nhảy góc khớp: mỗi tick (20 ms) góc tay đổi tối đa 0,2 rad, eo 0,1 rad, bàn tay 1,5 rad. Ngưỡng lấy từ dữ liệu thật (tay: 0,15 rad/tick ở phân vị 99,9 %). Thử một lệnh nhảy lớn:

LỆNH NHẢY GÓC VÀ GÓC SAU GIỚI HẠN · KHỚP TAY

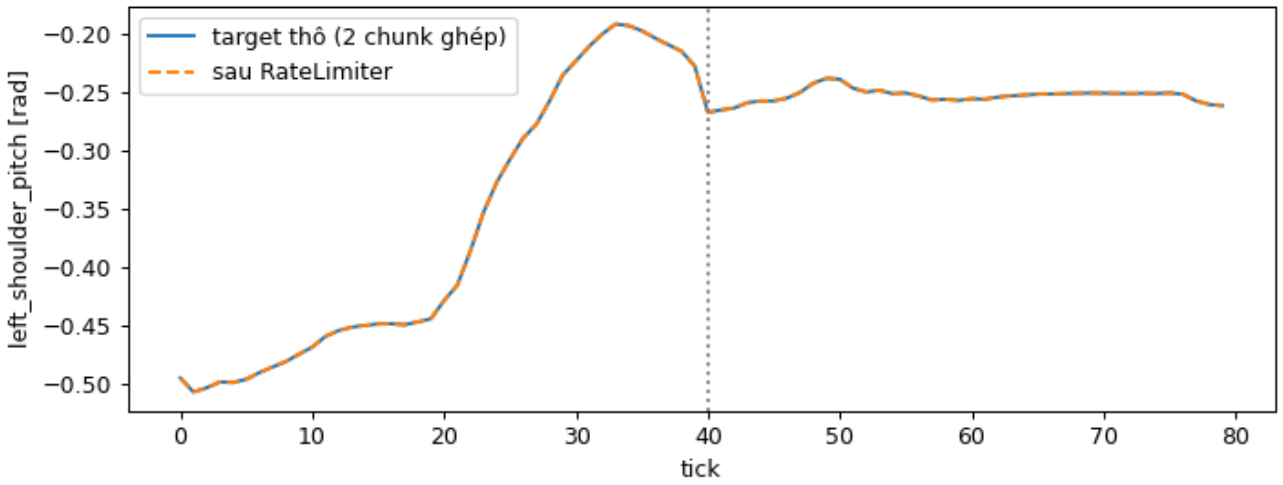
MINH HOẠ



KẾT QUẢ

Bước nhảy của lệnh	0,80 rad
Tối đa mỗi tick	0,2 rad
Số tick bị làm chậm	3

Lệnh bị chia thành 4 bước nhỏ, mất 80 ms thay vì 20 ms



Số liệu thật: lệnh khớp vai trái qua 80 tick (2 chuỗi 40 bước ghép lại). Với chuyển động bình thường, đường sau bộ giới hạn (nét đứt) trùng với lệnh gốc: lớp an toàn chỉ can thiệp khi có bước nhảy bất thường.

6 Bước vật lý: chuyển động được tính thế nào

Máy tính không mô phỏng chuyển động liên tục được. Nó chia thời gian thành các **bước vật lý** rất ngắn (5 ms) và ở mỗi bước tính lại lực → gia tốc → vận tốc → vị trí. Ví dụ dễ hình dung nhất là một chiếc xe chạy trên đường; khớp tay robot được tính đúng theo cách đó.

VÍ DỤ · CHIẾC XE

Mỗi 5 ms tính lại một lần

Ở mỗi bước: xem lực đang tác động (động cơ, ma sát) → gia tốc a = F / m → vận tốc mới v + a·Δt → vị trí mới x + v·Δt. Người lái thì không đạp ga 200 lần mỗi giây: cứ 20 ms mới quyết định một lần và giữ nguyên chân ga trong 20 ms đó, tức **một tick điều khiển = 4 bước vật lý**.

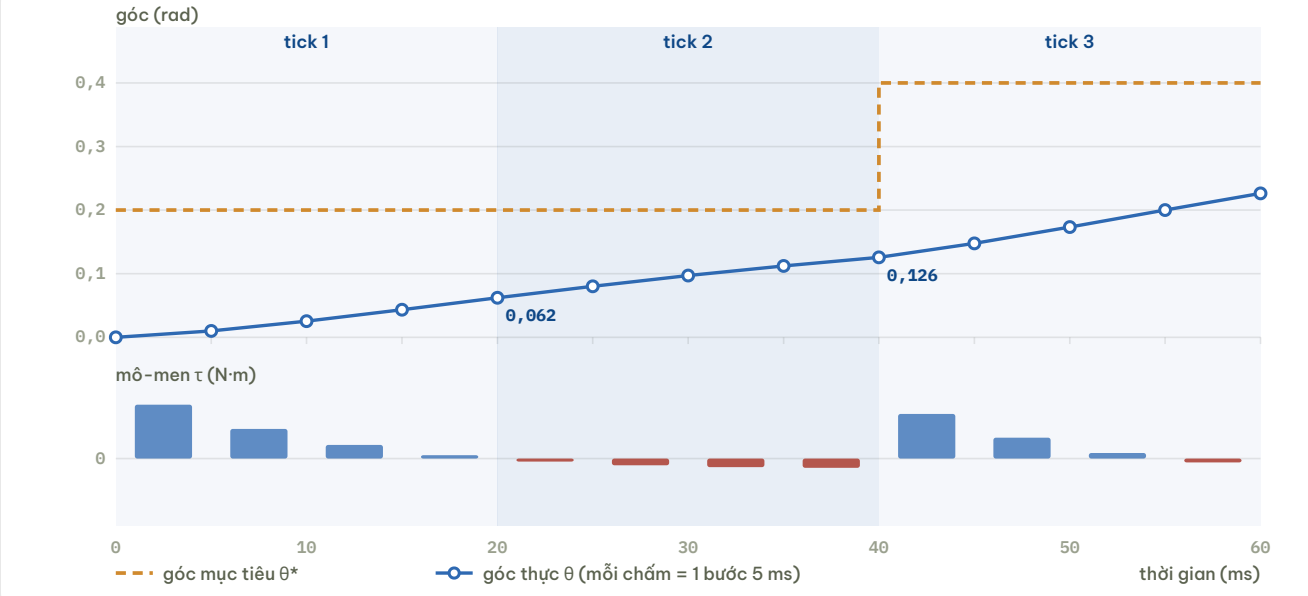
ÁP VÀO · KHỚP TAY

Lò xo kèm giảm chấn kéo khớp về góc mục tiêu

Model không đặt thẳng góc khớp mà chỉ cho **góc mục tiêu** θ*. Bộ PD (card 4) sinh mô-men như một lò xo kéo khớp về θ* và một giảm chấn hãm khi khớp quay nhanh. Mỗi bước vật lý, mô-men được tính lại theo góc và vận tốc góc lúc đó.

CHIẾC XE	KHỚP TAY	Ý NGHĨA
Vị trí x (m)	Góc khớp θ (rad)	Trạng thái cần tính
Vận tốc v (m/s)	Vận tốc góc ω (rad/s)	Trạng thái cần tính
Lực F (N)	Mô-men τ (N·m)	Tính lại mỗi bước vật lý
Khối lượng m	Quán tính I	Càng lớn càng khó tăng tốc
Người lái chọn mức ga	Pipeline chọn góc mục tiêu θ*	Chỉ đổi mỗi tick điều khiển (20 ms)

```
mỗi bước vật lý (Δt = 5 ms):
  τ = Kp·(θ* − θ) − Kd·ω     # lò xo kéo về θ*, giảm chấn hãm khi quay nhanh
  α = τ / I                  # gia tốc góc
  ω = ω + α·Δt               # vận tốc góc mới
  θ = θ + ω·Δt               # góc mới
```



Ví dụ minh hoạ (I = 0,05; K_p = 100; K_d = 4; không phải thông số thật của G1). Góc mục tiêu 0,2 rad ở tick 1–2 rồi 0,4 rad ở tick 3. Góc mục tiêu đổi theo bậc ở đầu tick; góc thực tăng dần qua từng bước 5 ms; mô-men lớn khi khớp còn xa đích, đổi dấu để hãm khi khớp đã quay nhanh.

TICK	BƯỚC	THỜI ĐIỂM	θ*	θ TRƯỚC	T	Ω SAU	θ SAU
1	1	5 ms	0,2	0,000	20,0	2,00	0,010
	2	10 ms	0,2	0,010	11,0	3,10	0,026
	3	15 ms	0,2	0,026	5,1	3,61	0,044

TICK	BƯỚC	THỜI ĐIỂM	θ*	θ TRƯỚC	T	Ω SAU	θ SAU
	4	20 ms	0,2	0,044	1,2	3,73	0,062
2	5	25 ms	0,2	0,062	-1,1	3,62	0,080
	6	30 ms	0,2	0,080	-2,5	3,37	0,097
	7	35 ms	0,2	0,097	-3,2	3,05	0,112
	8	40 ms	0,2	0,112	-3,4	2,71	0,126
3	**9**	**45 ms**	**0,4**	**0,126**	**16,6**	**4,37**	**0,148**

- **Góc mục tiêu chỉ đổi theo tick điều khiển**: giữ 0,2 rad suốt 8 bước đầu, nhảy lên 0,4 rad ở bước 9 (tick 3).
- **Mô-men tính lại ở từng bước vật lý**: 20 N·m ở bước 1 khi khớp còn xa đích, rồi giảm dần và đổi dấu từ bước 5 để hãm, không cho khớp vọt quá đích.
- **Góc tăng dần, không nhảy**: lệnh là 0,2 rad nhưng sau 20 ms khớp mới tới 0,062 rad, sau 40 ms tới 0,126 rad, giống xe không đạt ngay tốc độ khi vừa đạp ga.
- **Vì sao bước phải ngắn**: nếu chỉ tính mỗi 20 ms, mô-men 20 N·m ở bước đầu được áp suốt 20 ms không ai hãm, khớp vọt quá đích và dao động; va chạm ngón tay với vật cũng dễ sai.

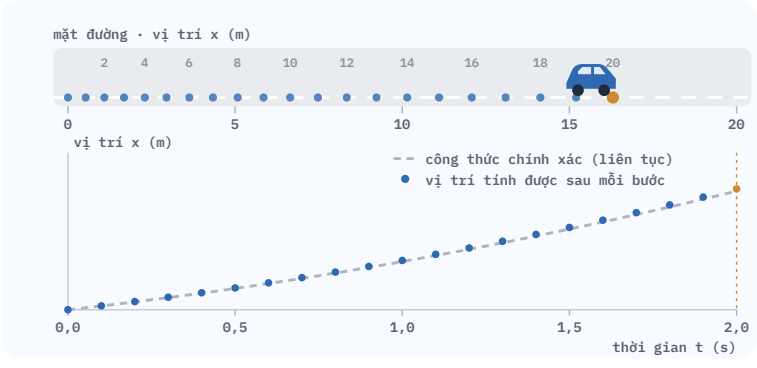
> **Trong Isaac Sim:** bộ PD được giải ngay trong bộ giải vật lý (*implicit*, card 4) nên ổn định hơn cách tính từng dòng ở trên, nhưng ý tưởng giống hệt: góc mục tiêu giữ nguyên trong tick, mô-men và góc được cập nhật ở mỗi bước 5 ms, cho cả 43 khớp cùng lúc, cùng với va chạm và ma sát của búa, thùng, ngón tay.

7 Mô phỏng: chiếc xe qua từng bước vật lý

Xe xuất phát với vận tốc v_0 và tăng tốc đều với gia tốc a trong 2 giây. Bấm **Chạy**: xe không trượt liên tục mà **nhảy tới vị trí mới mỗi khi một bước vật lý kết thúc**; mỗi chấm là một vị trí máy tính tính được, giữa hai chấm máy tính không biết xe ở đâu. Đổi độ dài bước để thấy bước dài thì chấm thưa và vị trí tính được lệch khỏi công thức chính xác.

XE CHẠY TRÊN ĐƯỜNG · VỊ TRÍ SAU MỖI BƯỚC VẬT LÝ

MINH HOẠ · TĂNG TỐC ĐỀU



BƯỚC HIỆN TẠI

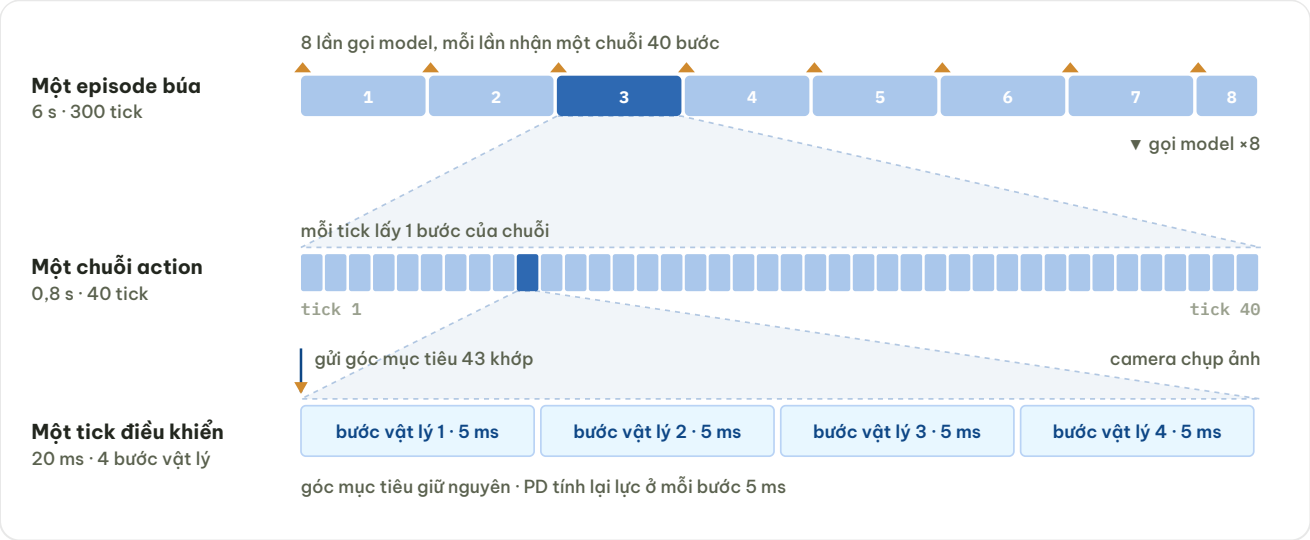
Bước	**20 / 20**
Thời điểm t	**2000 ms**
Vận tốc v	**11,00 m/s**
Vị trí x	**16,300 m**

Sau 2 s (20 bước): tính được 16,30 m, đúng là 16,00 m, sai 0,300 m (1,9 %)

Mỗi bước dùng đúng ba dòng như card 6: `a` (ở đây không đổi) → `v = v + a·Δt` → `x = x + v·Δt`. Với giá trị mặc định và Δt = 5 ms như trong dự án, sau 2 s sai số chỉ khoảng 1,5 cm; với Δt = 200 ms sai số lên tới hơn nửa mét. Khớp tay của robot được tính y hệt, chỉ thay vị trí x bằng góc θ và gia tốc a bằng gia tốc góc α = τ / I, với mô-men τ tính lại ở mỗi bước.

8 Tick điều khiển và tick vật lý

Mô phỏng chạy theo **ba nhịp lồng nhau**. Nhịp nhanh nhất là bước vật lý của Isaac Sim; cứ 4 bước vật lý là một tick điều khiển của pipeline; cứ 40 tick lại gọi model GR00T một lần để lấy chuỗi action mới.



Ba nhịp lồng nhau. Mỗi lần gọi model cho một chuỗi 0,8 s; mỗi chuỗi chạy trong 40 tick điều khiển; mỗi tick gồm 4 bước vật lý 5 ms.

Bước vật lý
5 ms
200 Hz · PhysX trong Isaac Sim

Tick điều khiển
20 ms
50 Hz · = 4 bước vật lý

Một chuỗi action
0,8 s
40 tick · 1 lần gọi model

Một episode búa
6 s
300 tick · 1.200 bước vật lý · 8 lần gọi model

NHỊP	CHU KỲ	AI CHẠY	LÀM GÌ
Bước vật lý	5 ms · 200 Hz	PhysX (Isaac Sim)	Tính lực ở từng khớp bằng PD từ góc mục tiêu (card 4), giải va chạm và ma sát giữa ngón tay, vật thể, bàn; cập nhật vị trí mọi vật
Tick điều khiển	20 ms · 50 Hz	Pipeline của dự án	Đọc quan sát, lấy một bước trong chuỗi action, giới hạn tốc độ khớp, cho AGILE tính 12 khớp chân, gửi góc mục tiêu cho 43 khớp; camera chụp một ảnh
Chuỗi action	0,8 s · 40 tick	Server GR00T	Nhìn ảnh, câu lệnh và trạng thái khớp, trả về 40 bước góc mục tiêu cho tay, bàn tay và eo

Ràng buộc trong code: bước vật lý × số bước mỗi tick phải đúng bằng một tick (0,005 s × 4 = 0,02 s). Cấu hình lệch nhau sẽ báo lỗi ngay khi nạp (`ControlCfg` trong `g1gr00t/config.py`). Kịch bản 2 (mục III.02) dùng nhịp khác cho khớp với dữ liệu G1 thật ghi ở 20 Hz: vẫn 200 Hz vật lý nhưng 10 bước mỗi tick, tức điều khiển 20 Hz; chuỗi 16 bước cũng dài 0,8 s.

9 Bên trong một tick điều khiển

Mỗi tick, pipeline làm đúng một vòng: đọc trạng thái, quyết định góc mục tiêu cho cả 43 khớp, rồi để vật lý chạy 4 bước với góc mục tiêu đó.

01 · QUAN SÁT
Ảnh và trạng thái
Camera đầu, 43 góc khớp, vận tốc, IMU

→

02 · CHUỖI ACTION
Lấy bước kế tiếp
Hết 40 bước thì gọi server GR00T

→

03 · AN TOÀN
Giới hạn tốc độ
Tay 0,2 rad/tick (card 5)

→

04 · CHÂN
AGILE giữ thăng bằng
80 số quan sát → 12 khớp chân

→

05 · VẬT LÝ
4 bước × 5 ms
Góc mục tiêu giữ nguyên

```
mỗi tick điều khiển (20 ms):
  obs   = backend.observe()                    # ảnh, 43 góc khớp, vận tốc, IMU
  step  = scheduler.step(policy.fetch, obs)    # 1/40 bước; hết chuỗi thì gọi server
  upper = rate_limiter.apply(step["upper_q"])  # tay ≤ 0,2 rad/tick (card 5)
  lower = agile.step(q, dq, imu)               # 12 khớp chân, cũng 50 Hz
  robot.set_joint_position_target(upper + lower)
  for _ in range(4):                           # 4 bước × 5 ms, giữ góc mục tiêu
      sim.step()                               # PD tính lực; va chạm, ma sát
  camera.update()                              # ảnh cho tick sau
```

GIỮ LỆNH

Góc mục tiêu giữ nguyên trong 4 bước vật lý

Lệnh gửi đi là góc mục tiêu, không phải lực. Trong 20 ms của một tick, bộ PD tính lại lực 4 lần theo góc thực đang thay đổi, nên khớp tiến dần tới đích chứ không nhảy. Robot thật cũng vậy: driver motor nhận lệnh vị trí rồi tự điều khiển ở tần số cao hơn nhiều.

HAI TẦN SỐ

Vì sao vật lý phải chạy nhanh hơn điều khiển

Va chạm và tiếp xúc giữa ngón tay với vật cần bước thời gian nhỏ thì mới ổn định; bước dài dễ làm vật xuyên vào nhau hoặc nảy lên. Ngược lại, tick điều khiển phải đúng 20 ms vì AGILE và dữ liệu demo đều ở 50 Hz: tick khác đi thì một bước trong chuỗi action không còn ứng với 20 ms như lúc model học.

10 Thời gian mô phỏng và thời gian thực

Một lần gọi model mất khoảng 1,2–1,5 s (trung vị khi đánh giá, mục I.02), lâu hơn 0,8 s của một chuỗi action. Mô phỏng và robot thật xử lý chênh lệch này rất khác nhau.

MÔ PHỎNG

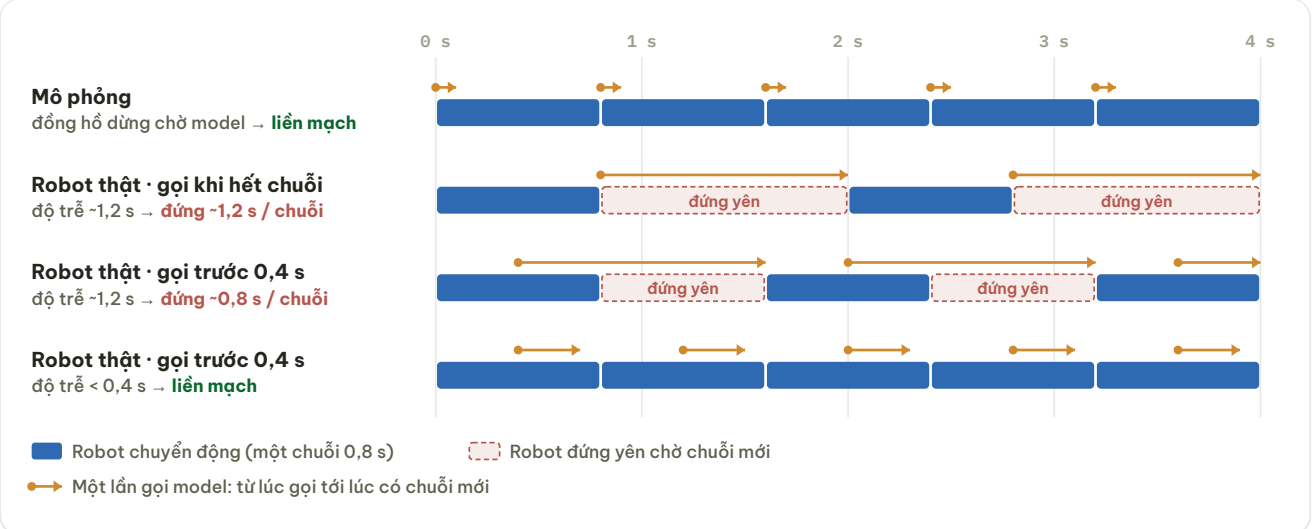
Đồng hồ mô phỏng dừng chờ model

Thời gian mô phỏng chỉ tiến khi gọi `sim.step()`. Trong lúc chờ server, không có bước vật lý nào chạy, nên độ trễ không ảnh hưởng kết quả. Đổi lại, chạy chậm hơn thời gian thực: một episode búa 6 s mô phỏng mất trung bình ~36 s thật (lần chạy mix400, 8 lần gọi model mỗi episode).

ROBOT THẬT

Thời gian không dừng

Nếu chờ hết chuỗi mới gọi, robot sẽ đứng yên trong lúc chờ. Code có sẵn `PrefetchingChunkScheduler`: gọi server khi chuỗi còn 20 tick (0,4 s), để chuỗi mới tới kịp lúc chuỗi cũ hết. Đổi lại, chuỗi mới được tính từ ảnh chụp sớm 0,4 s.



Trục thời gian 4 giây đầu. Mô phỏng không bao giờ phải đứng chờ vì đồng hồ mô phỏng dừng theo model. Trên robot thật, chỉ khi gọi trước 0,4 s *và* độ trễ dưới 0,4 s thì chuyển động mới liền mạch; bản thân model chỉ mất ~0,1 s, phần còn lại là đường truyền (mục I.02).

Ít demo thật, nhiều dữ liệu huấn luyện.

Fine-tune GR00T cần vài trăm demo cho mỗi tác vụ, nhưng demo teleop đắt và model lại rất nhạy với hình ảnh. Mục này giải thích ba kỹ thuật để có thêm dữ liệu mà không teleop thêm (dự án đã áp dụng hai kỹ thuật sau, kỹ thuật đầu mới ở mức đề xuất): **domain randomization**, **real2sim** và **sinh thêm demo**. Mỗi kỹ thuật có một khung minh hoạ tương tác để thấy nó hoạt động ra sao.

KỸ THUẬT
Domain randomization · real2sim · sinh thêm demo

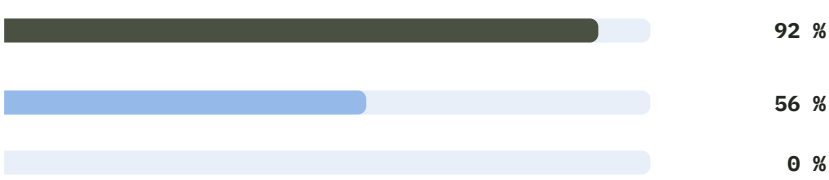
ÁP DỤNG
Apple & Plate · Multi Fruit · Hammer & Bin

TỪ KHÓA
Visual gap · digital twin · MimicGen

1 Problem 1: Visual Gap

Nếu mọi demo được ghi trong cùng một khung cảnh, model học thuộc hình ảnh đó. Bài táo cho thấy rõ điều này.

Cùng checkpoint táo → đĩa, chỉ đổi khung cảnh — tỉ lệ thành công · số liệu đo, mục III.01



Cùng checkpoint, cùng câu lệnh; mốc 92 % đo bằng client NVIDIA, hai mức sau bằng pipeline của dự án. Model chỉ thấy một khung cảnh lúc huấn luyện nên gắn với hình ảnh đó thay vì với quả táo. Domain randomization (card 2) là cách xử lý.

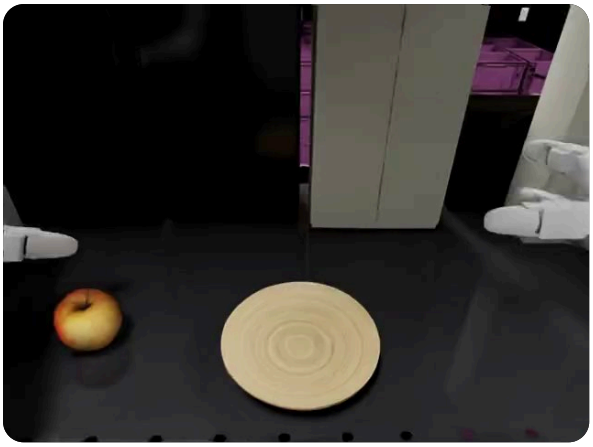
2 Solution 1: Domain Randomization

Ý tưởng một câu: đổi ngẫu nhiên mọi thứ **không liên quan** tới tác vụ (ánh sáng, màu, nền, góc camera) để model chỉ còn bám được vào thứ không đổi là **vật thể và bàn tay**. Gặp khung cảnh mới, kể cả ảnh thật, model coi đó là thêm một biến thể (Tobin và cộng sự, 2017).

Thử ngay: hai cách làm đa dạng hình ảnh

CÁCH 1 · TĂNG CƯỜNG ẢNH 2D

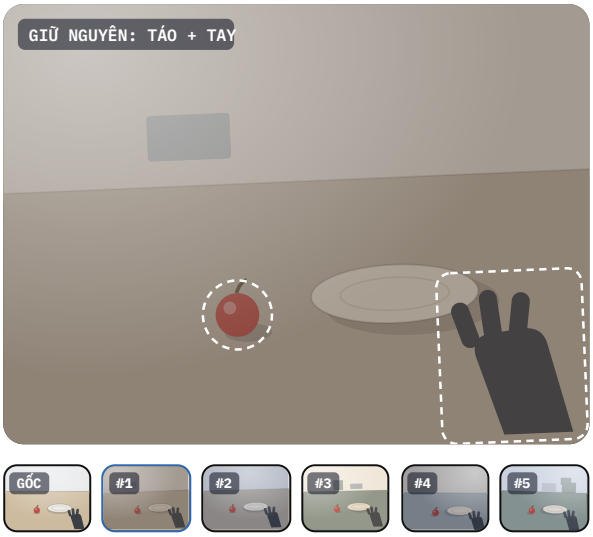
ẢNH THẬT · CAMERA ĐẦU



Color jitter: rẻ, áp lên ảnh khi huấn luyện, nhưng **không tạo được bóng đổ, phản chiếu hay góc nhìn mới**. Dự án đã dùng cách này khi fine-tune Multi Fruit.

CÁCH 2 · DOMAIN RANDOMIZATION

MINH HOẠ · KHÔNG PHẢI DỮ LIỆU ĐO



Đổi chính khung cảnh rồi **render lại**. Quả táo và bàn tay (viền nét đứt) giữ nguyên: đó là thứ model phải học.

Yếu tố có thể ngẫu nhiên hoá

Ba nhóm yếu tố, từ dễ tới khó:

NGOẠI QUAN

Thứ model nhìn thấy

Ánh sáng, vật liệu, texture, phông nền, vật trang trí ngoài vùng thao tác.

HÌNH HỌC

Bố cục và góc nhìn

Vị trí, hướng vật; vị trí, góc chúc, tiêu cự camera.

ĐỘNG LỰC HỌC

Vật lý

Khối lượng, ma sát, độ cứng khớp, độ trễ (Peng và cộng sự, 2018). Cần khi lên robot thật.

> **Đánh đổi:** ngẫu nhiên càng rộng thì model càng bền nhưng càng cần nhiều dữ liệu và bước huấn luyện. Nên ưu tiên đúng yếu tố đã gây sụt giảm (ở đây là ánh sáng và render), rồi mới nới rộng.

Áp dụng trong dự án

ÁP DỤNG	ĐÃ LÀM	CÒN THIẾU
Apple & Plate	Đo visual gap 92 → 56 → 0 %	Ngẫu nhiên hoá khung cảnh rồi phát lại 208 demo (đề xuất ở mục III.01)
Multi Fruit	Đổi quả, vị trí, hướng, đĩa, quả gây nhiễu mỗi episode; color jitter khi fine-tune	Ánh sáng, vật liệu chưa đổi

3 Problem 2: Sim Data Scarcity

Kịch bản Multi Fruit không có dataset mô phỏng phù hợp cho G1 + Dex3. Bộ gần nhất là **PhysicalAI-Robotics-GR00T-Teleop-G1** (1.095 quỹ đạo teleop), nhưng được ghi trên robot thật nên không dùng thẳng trong mô phỏng được.

THIẾU 1

Không có vị trí vật

Dữ liệu chỉ có góc khớp và video; không biết quả và đĩa nằm ở đâu để dựng khung cảnh.

THIẾU 2

Ảnh thật ≠ ảnh render

Video RealSense thật không khớp với ảnh mô phỏng mà model sẽ thấy khi chạy.

THIẾU 3

Action khác nghĩa

Action trên robot thật và trong mô phỏng cùng tên nhưng mang ý nghĩa khác nhau.

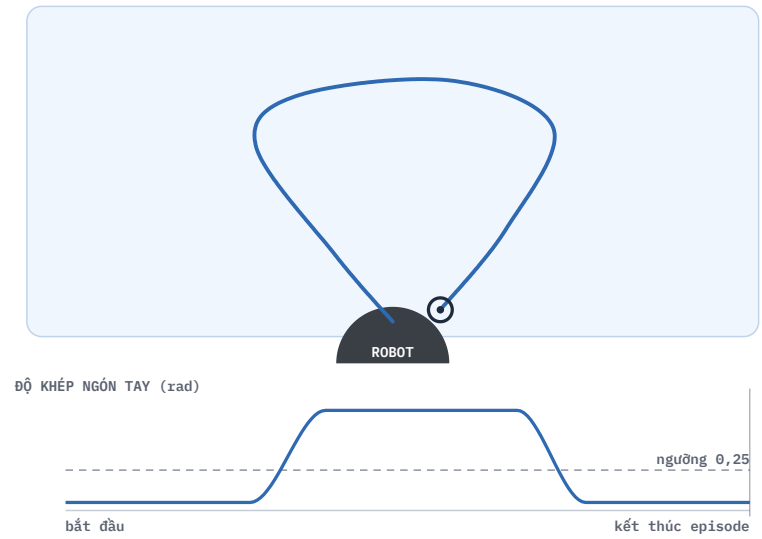
4 Solution 2: Real2sim

Ý tưởng một câu: dựng lại trong mô phỏng một **bản sao số** (digital twin) của khung cảnh thật, rồi phát lại chuyển động thật trong đó để có dữ liệu mô phỏng. Dataset không có vị trí đồ vật, nên dự án **suy ra khung cảnh từ chuyển động của bàn tay**: tay khép lại ở đâu thì quả ở đó, tay mở ra ở đâu thì đĩa ở đó.

Thử ngay: tái tạo khung cảnh từ một quỹ đạo

NHÌN TỪ TRÊN XUỐNG MẶT BÀN

MINH HOẠ · KHÔNG PHẢI DỮ LIỆU ĐO



BƯỚC 1 / 5

Động học thuận (FK)

Từ góc khớp ở mỗi khung hình, nhân lần lượt các phép biến đổi dọc cánh tay (file URDF) để ra vị trí lòng bàn tay. Ghép lại thành quỹ đạo bàn tay trên mặt bàn.

Trong dự án: Bộ FK tự viết chỉ dùng numpy; tư thế thân suy ra từ giả định hai bàn chân đặt phẳng trên sàn (mục II.03).

5 Problem 3: Teleop Bottleneck

Kịch bản Hammer & Bin hoàn toàn mới: không có dataset hay checkpoint sẵn, mọi demo phải tự thu bằng teleop kính Meta Quest trong Isaac Lab.

CHẬM

~10 demo mỗi phiên

100 demo thật đầu tiên mất 10 phiên teleop.

KHÔNG ỔN ĐỊNH

Nhiều lượt phải làm lại

Tay robot đứng im khi mất tín hiệu tay cầm, giật lag từ lượt thứ 2.

KHÔNG ĐỀU

Demo hỏng lẫn vào dữ liệu

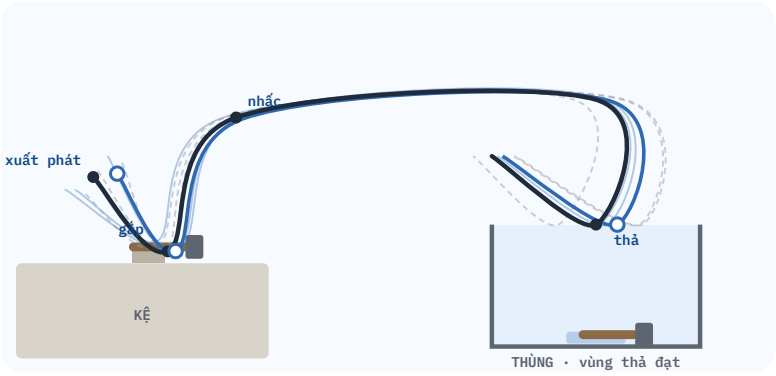
Có demo được chấm thành công khi búa còn gác lên thành thùng hoặc tay chưa buông.

6 Solution 3: Demo Generation

Ý tưởng một câu: lấy một demo teleop làm **khuôn**, dịch nhẹ các điểm then chốt (xuất phát, gắp, thả) và tốc độ để được demo mới, rồi **chạy lại trong mô phỏng**: chỉ giữ demo vẫn thành công. Đây là cách của MimicGen (Mandlekar và cộng sự, 2023), mà NVIDIA dùng ở quy mô lớn để sinh 780.000 quỹ đạo cho GR00T N1.

Thử ngay: sinh demo búa → thùng

NHÌN NGANG · KỆ → THÙNG



MINH HOẠ · ĐỘ LỆCH PHÓNG ĐẠI ĐỂ DỄ NHÌN

5	3
giữ lại	bị loại

DEMO VỪA SINH · #8

Thời gian	×0,96
Xuất phát lệch	1,2 cm
Điểm gắp lệch	4 mm
Điểm thả lệch	+1,0 cm

✓ Chạy lại trong mô phỏng: búa vào giữa thùng → GIỮ

Thực tế trong dự án: **100 / 133** lượt giữ lại, 60 demo khuôn, ~17 phút cho 100 demo.

Demo khuôn (teleop) — Demo sinh ra, đạt ---- Demo sinh ra, bị loại (búa không vào giữa thùng)

7 Kết hợp và kết luận

Ba kỹ thuật không thay thế nhau mà nối thành một chuỗi. Hướng kết hợp cho giai đoạn đưa model lên robot thật:

01 **Teleop thật**
Ít demo, tại môi trường làm việc →

02 **Real2sim**
Dựng khung cảnh, phát lại trong mô phỏng →

03 **Sinh thêm demo**
Nhân số lượng, lọc bằng mô phỏng →

04 **Domain randomization**
Đa dạng ngoại quan và vật lý →

05 **Fine-tune & sim2real**
Đánh giá trong mô phỏng rồi trên robot thật

KẾT LUẬN

Dự án đã kiểm chứng hai trong ba kỹ thuật: **real2sim** biến dữ liệu robot thật thành 790 episode mô phỏng (66 %), **sinh thêm demo** nhân đôi dữ liệu teleop trong ~17 phút (66 % với mix200). **Domain randomization** mới dừng ở đo visual gap và đề xuất phương pháp; đây là bước cần làm tiếp trước khi chuyển sang robot thật.

III.01 Kịch bản Apple & Plate

Robot G1 gắp quả táo trên kệ, đặt lên đĩa.

Kịch bản mẫu của NVIDIA Isaac Lab Arena, chạy lại bằng **pipeline điều khiển do dự án tự xây dựng**. Dự án kiểm chứng pipeline theo tầng (open-loop với dataset, phát lại demo) rồi đánh giá checkpoint của NVIDIA trong ba khung cảnh. Kết quả: **92 %** trong môi trường gốc, **56 %** khi chỉ lệch ánh sáng, **0 %** khi cả khung cảnh khác — model rất nhạy với hình ảnh, pipeline hoạt động đúng.

DATASET

208 episode

Nguồn	**NVIDIA · teleop XR**
Frame	**35.066**
Tần số	**50 Hz**

FINETUNE

Không huấn luyện lại

Dùng	**Checkpoint NVIDIA**
Đã tune cho	**Táo → đĩa**
Câu lệnh	**Theo dataset**

MODEL

GR00T N1.7 3B

Checkpoint	**GN1x-Tuned-Arena**
Embodiment	**G1 + Dex3**
Chuỗi action	**40 bước · 50 Hz**

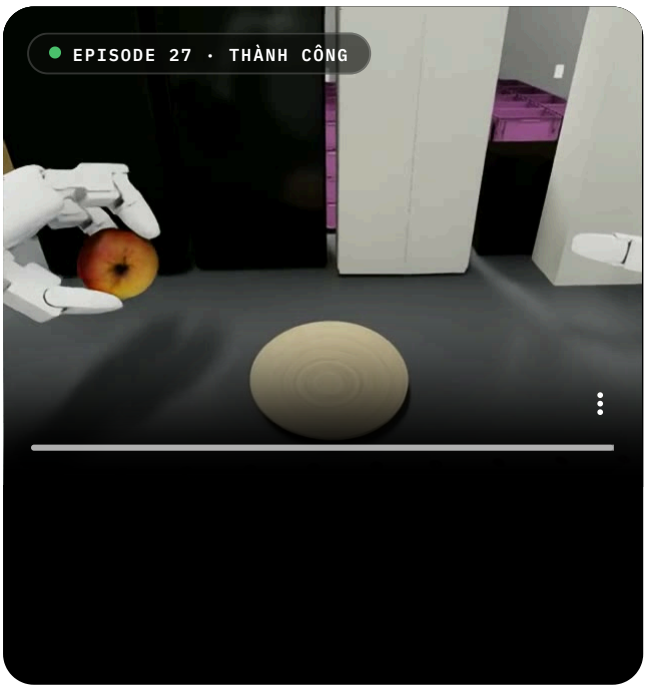
RESULT

92 % thành công

92 % GỐC | 56 % ÁNH SÁ… | 0 % KHUNG …

Cùng model; chỉ đổi môi trường chạy

1 Video Showcase



Robot nhặt táo đặt lên đĩa, nhìn từ camera đầu

CÂU LỆNH GỬI MODEL

"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."

2,9 s
Thời gian tới thành công

1,0 cm
Tâm táo cách tâm đĩa

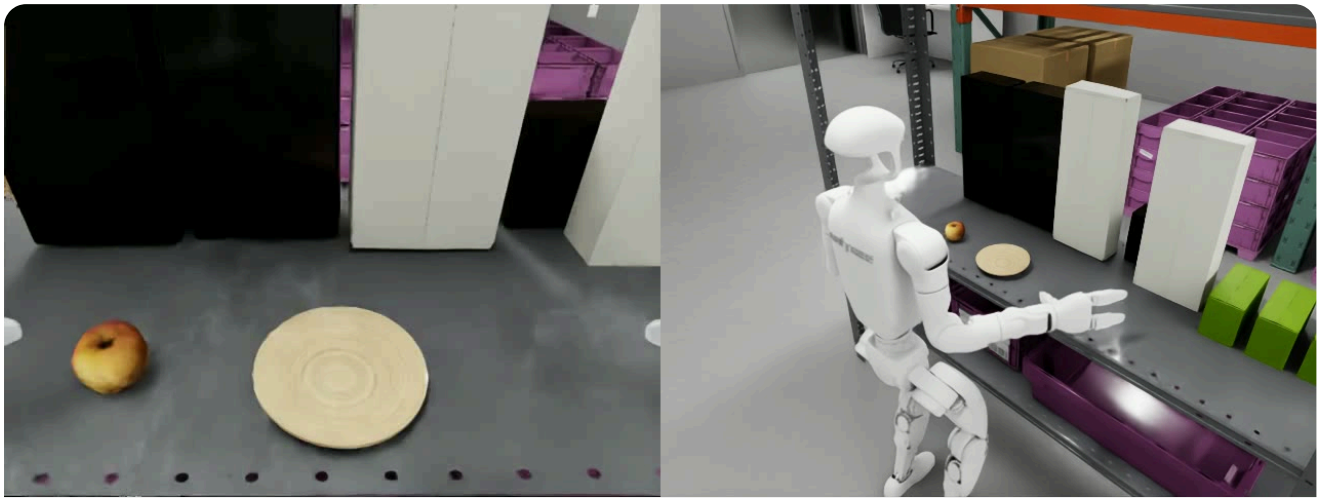
4
Lần gọi model, mỗi lần 40 bước

1,1 s
Độ trễ trung bình mỗi lần gọi

i Trích từ đợt 50 episode trên khung cảnh Galileo dựng lại (28/50 thành công), checkpoint NVIDIA.

2 Bối cảnh kịch bản

Robot G1 đứng trước một kệ trong phòng lab Galileo của Arena; quả táo và chiếc đĩa đặt cạnh nhau trên mặt kệ. Ảnh dưới ghép **khung đầu tiên** của episode trong Video Showcase (ảnh model nhận vào ở bước 0) với góc nhìn xiên toàn cảnh.



Khung 0 · trái: camera đầu robot, ảnh model nhận vào · phải: góc nhìn xiên từ camera ngoài (dựng lại cảnh Galileo trong Isaac Sim, cùng góc camera với kịch bản Hammer & Bin), chỉ để minh hoạ.

KHUNG CẢNH & VẬT THỂ	CHI TIẾT
Nền	Phòng lab Galileo (`galileo_locomanip.usd`); ẩn 3 hộp thiết bị che tầm tay (jetson_orin_06, jetson_orin_03, hesai_box_06)
Mặt kệ	Tấm đỡ ẩn 0,8 × 1,5 × 0,04 m; mặt trên cao 0,765 m
Quả táo	Đặt tại (0,58; 0,27) m
Đĩa	Đặt tại (0,58; 0,06) m — cách táo ~21 cm về bên phải robot
Ma sát	Tăng ma sát ngón tay Dex3 để giữ táo chắc

ROBOT, CAMERA & ĐIỀU KHIỂN	CHI TIẾT
Robot	Unitree G1 + bàn tay Dex3, đứng tại (0,25; 0,08) m, quay mặt theo trục +x; chân giữ thăng bằng bằng bộ điều khiển AGILE, hông cao ~0,73–0,75 m (lệnh 0,72 m)
Camera	Gắn trên `head_link` , 640×480, tiêu cự 15 mm, chúc xuống ~35°
Tần số	Vật lý 200 Hz, điều khiển 50 Hz (4 bước vật lý / lệnh)
Episode	Tối đa 300 bước = 6 s; mỗi lần gọi model trả về chuỗi 40 bước
Câu lệnh	"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
Thành công	Tâm táo cách tâm đĩa < 8 cm, cao hơn đĩa 0–4 cm và vận tốc < 0,1 m/s

3 Mục tiêu kịch bản

Kịch bản mẫu của NVIDIA được dùng làm **chuẩn đối chiếu**: model và dữ liệu đã có sẵn, nên mọi sai khác đều đến từ pipeline hoặc môi trường của dự án.

MỤC TIÊU 1

Hiểu đường dữ liệu

Nắm đầy đủ luồng từ camera và góc khớp tới model, và từ model tới động cơ của G1.

MỤC TIÊU 2

Tự viết lại pipeline

Xây package độc lập, không phụ thuộc Arena khi chạy — bước bắt buộc trước khi lên robot thật, vì Arena chỉ chạy trong mô phỏng.

MỤC TIÊU 3

Kiểm chứng và đo

Chứng minh pipeline đúng theo tầng, rồi đo tỉ lệ thành công của checkpoint NVIDIA và tìm nguyên nhân khi thất bại.

4 Dữ liệu kịch bản

DATASET ARENA-G1-STATIC-PICKNPLACE	GIÁ TRỊ
Nguồn	Teleop kính XR trong Isaac Lab, 50 Hz — dữ liệu đã dùng để fine-tune checkpoint NVIDIA
Số episode	208 file (card ghi 200; metadata đánh chỉ số tới 251)
Tổng frame	35.066 (11,7 phút); mỗi episode 2,3–5,3 s
Câu lệnh	"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
Quan sát	Chỉ tay trái nắm táo; eo, bàn tay phải gần như bất động; robot đứng yên, thân cao ~0,72 m

5 Vấn đề gặp phải

Khi tạo lại pipeline của Arena bằng package tự viết, mọi khâu điều khiển đều chạy đúng, nhưng tỉ lệ thành công phụ thuộc mạnh vào **hình ảnh camera** mà model nhận được.

> **Visual gap:** cùng checkpoint, cùng câu lệnh, chỉ đổi khung cảnh mô phỏng mà tỉ lệ thành công đi từ **92 %** (client NVIDIA, Arena gốc) xuống **56 %** rồi **0 %** (pipeline của dự án).

VẤN ĐỀ 1

Khung cảnh khác dataset thì model không làm được

Khung cảnh tự dựng (bàn hộp, sàn lưới, đèn dome) cho **0/50** thành công, chỉ 11/50 nhấc được táo. Model vươn tay giống demo nhưng không bao giờ ra lệnh nắm.

VẤN ĐỀ 2

Dựng lại đúng khung cảnh vẫn chưa đủ

Khung cảnh Galileo dựng lại khớp bố trí và góc camera với dataset nhưng chỉ đạt **56 %**, so với 92 % trong Arena gốc. Khác biệt còn lại chủ yếu ở ánh sáng và cách render.

VẤN ĐỀ 3

Model chỉ học từ một khung cảnh

Checkpoint được fine-tune trên dữ liệu ghi trong đúng một khung cảnh, một điều kiện ánh sáng, nên gắn chặt với hình ảnh đó thay vì với vật thể cần thao tác.

VẤN ĐỀ 4

Khó chuyển sang môi trường thật

Robot thật sẽ gặp ánh sáng, nền, vật liệu khác hẳn mô phỏng; với mức nhạy hình ảnh hiện tại, checkpoint không thể dùng trực tiếp.

6 Phương pháp đề xuất

Đề xuất dùng **domain randomization**: khi tạo dữ liệu huấn luyện, thay đổi ngẫu nhiên những yếu tố hình ảnh không liên quan tới tác vụ, để model buộc phải dựa vào vật thể và tư thế tay thay vì nền và ánh sáng.

Domain randomization cũng là kỹ thuật phổ biến cho **sim2real**; ở đây áp dụng trước cho khoảng cách giữa các khung cảnh mô phỏng, là bước tiền đề trước khi chuyển sang robot thật.

> **Trạng thái:** chưa thực hiện. Mục này chỉ nêu phương pháp và cách áp dụng vào pipeline hiện có.

Các yếu tố ngẫu nhiên hoá

Ngoại quan — thứ model nhìn thấy nhưng không ảnh hưởng tới cách làm tác vụ:

ÁNH SÁNG

Cường độ, màu, hướng

Thay đổi cường độ và nhiệt độ màu của đèn dome/rect, hướng chiếu, bật tắt đèn bake sẵn — đúng yếu tố làm 92 % rơi xuống 56 %.

VẬT LIỆU

Màu và texture

Đổi màu, độ bóng, texture của mặt kệ/bàn, đĩa và nền phòng; giữ nguyên hình dạng vật thể.

KHUNG CẢNH

Nền và vật xung quanh

Đổi phông nền, thêm/bớt hộp và vật trang trí ngoài vùng thao tác.

Hình học — góc nhìn và vị trí vật, để model không học thuộc một bố cục cố định:

CAMERA

Tư thế và thông số

Dao động nhỏ vị trí, góc chúc và tiêu cự camera đầu; thêm nhiễu, độ phơi sáng và color jitter khi huấn luyện.

VẬT THỂ

Vị trí và hướng

Xáo vị trí, hướng của táo và đĩa trong vùng với tới của tay.

Cách áp dụng

BƯỚC 1
Khung cảnh ngẫu nhiên
Sự kiện reset của Isaac Lab / Replicator đổi các yếu tố trên mỗi episode

→

BƯỚC 2
Phát lại demo
Phát lại action của 208 demo trong từng khung cảnh, ghi ảnh mới

→

BƯỚC 3
Lọc
Giữ episode vẫn đạt điều kiện thành công

→

BƯỚC 4
Fine-tune lại
Từ checkpoint NVIDIA trên dữ liệu đã đa dạng hoá

→

BƯỚC 5
Đánh giá chéo
Galileo, khung cảnh tự dựng và khung cảnh chưa từng thấy

Kỳ vọng: thu hẹp khoảng cách 92 % → 56 % → 0 % giữa các khung cảnh. Thước đo là tỉ lệ thành công trên khung cảnh tự dựng (hiện 0 %) và Galileo dựng lại (hiện 56 %), mỗi cấu hình ≥ 50 episode.

7 Chu trình thực hiện

Tái hiện lại luồng chạy của Isaac Lab Arena bằng package tự viết `standalone_g1_gr00t`: tách từng khâu của Arena ra, viết lại và kiểm chứng riêng trong một notebook, rồi ghép lại thành vòng kín không còn phụ thuộc Arena.

BƯỚC 1 · ARENA
Chạy Arena gốc
Checkpoint NVIDIA trong môi trường gốc — mốc đối chiếu 92 %

→

BƯỚC 2 · NB01–02
Giao tiếp model
Server ZMQ, dịch quan sát và action

→

BƯỚC 3 · NB03–04
Kiểm chứng từng khối
Open-loop với dataset, chân AGILE

→

BƯỚC 4 · NB05–06
Backend Isaac Sim
Robot, camera, táo/đĩa, phát lại demo

→

BƯỚC 5 · NB07–08
Vòng kín & đóng gói
Khung cảnh Galileo, runner, lớp an toàn

BƯỚC	VIỆC LÀM	NOTEBOOK / CÔNG CỤ	KẾT QUẢ
1	Chạy pipeline Arena với client và checkpoint của NVIDIA để có mốc so sánh	Isaac Lab Arena	92 % trong môi trường gốc
2	Giao thức ZMQ + msgpack, hợp đồng modality; bảng 4 thứ tự khớp, `build_observation`, `decode_action`, `ChunkScheduler`	nb01_server_wire, nb02_joints_obs_action	Gửi/nhận đúng chuỗi 40 bước
3	Ảnh và trạng thái thật từ dataset → server → so với action ghi sẵn; tái tạo vector quan sát 80 chiều của policy chân ONNX	nb03_open_loop_dataset, nb04_agile_onnx	Sai số nhỏ hơn biên độ 10–70×; AGILE trùng từng bit
4	Dựng tay backend: SimulationContext → robot → camera đầu → táo/đĩa → ma sát ngón tay → observe/apply; ghép đủ đường dữ liệu	nb05_backend_anatomy, nb06_full_pipeline	Đứng vững 10 s; phát lại demo thành công
5	Dựng lại khung cảnh Galileo của Arena, chạy 50 episode GR00T; đóng gói runner, policy, `sanitize_chunk`, `RateLimiter`	nb07_full_pipeline_loop, nb08_runner_policy_safety	56 % (lệch ánh sáng), 0 % (khung cảnh tự dựng)

8 Kết quả đạt được

Số liệu đo được ở từng bước của chu trình, từ kiểm chứng từng khối tới chạy vòng kín. Cùng một checkpoint NVIDIA và cùng câu lệnh cho mọi lần chạy.

Arena gốc

92 %

client NVIDIA · môi trường gốc

Galileo dựng lại

56 %

28/50 · Wilson 42–69 %

Khung cảnh tự dựng

0 %

0/50 · nhấc được 22 %

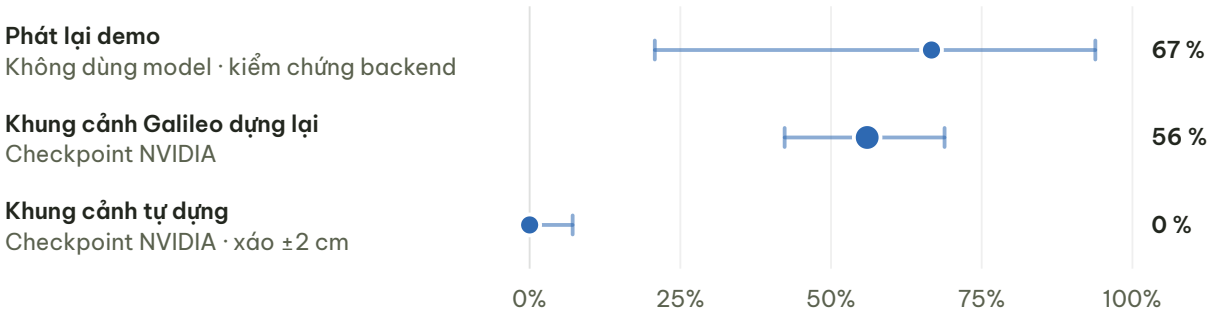
Sai số open-loop

≤ 0,032 rad

MAE 0,008–0,032 · nhỏ hơn biên độ 10–70×

Tỉ lệ thành công và khoảng tin cậy Wilson 95 %

Cùng checkpoint, chỉ đổi khung cảnh. Phát lại demo không dùng model, chỉ để kiểm chứng backend.



BƯỚC	SỐ LIỆU ĐO ĐƯỢC
Giao tiếp model	Chuỗi 40 bước mỗi lần gọi; độ trễ ~1,2 s/lần (trung vị 50 episode; mô phỏng chờ model nên không ảnh hưởng kết quả)
Open-loop với dataset	MAE tay trái 0,032 · tay phải 0,019 · bàn tay trái 0,008 rad (3 episode, 11 chunk)
Câu lệnh	Câu trong dataset 0,025 rad; câu mặc định Arena 0,047 rad; câu vô nghĩa 0,039 rad
Chân AGILE	Trùng từng bit với bản tham chiếu; đứng vững 500 tick, thân 0,73–0,75 m, nghiêng tối đa ~10°
Phát lại demo	2/3 lần thành công; táo cách tâm đĩa 3 cm; sai số bám khớp 0,04 rad
Vòng kín · Galileo	**28/50 thành công; nhấc được 72 %; khoảng cách táo–đĩa trung vị 3,8 cm; thời gian tới thành công trung vị 2,7 s**
Vòng kín · tự dựng	0/50 thành công; nhấc được 22 %; khoảng cách táo–đĩa trung vị 20,6 cm

9 Phân tích kết quả

So sánh các lần chạy theo từng pha của tác vụ để xác định thất bại đến từ đâu.

PHA (50 EPISODE)	TỰ DỰNG	GALILEO
Nhấc được táo	11 (22 %)	36 (72 %)
Đặt trúng đĩa / số đã nhấc	0 / 11	28 / 36 (78 %)
Thành công	**0 %**	**56 %**
Ngã	3	2

NHẬN ĐỊNH 1

Thị giác quyết định kết quả

Cùng checkpoint, câu lệnh: 92 % (client NVIDIA, Arena gốc) → 56 % → 0 % (pipeline của dự án) chỉ do đổi khung cảnh. Hai khoảng tin cậy 0–7 % và 42–69 % không chồng nhau.

NHẬN ĐỊNH 2

Thất bại nằm ở pha nhìn và tiếp cận

Ở khung cảnh tự dựng model hầu như không nhấc được táo (22 %). Ở Galileo, khi đã nhấc thì 78 % đặt trúng đĩa — kỹ năng thao tác vẫn còn.

NHẬN ĐỊNH 3

Pipeline tự viết hoạt động đúng

28 lần thành công với khoảng cách cuối 1–6 cm, cùng các kiểm chứng open-loop và phát lại, loại trừ lỗi dịch dữ liệu.

NHẬN ĐỊNH 4

Hệ quả cho triển khai

Model fine-tune trên một khung cảnh không tổng quát hoá; cần thu dữ liệu trong môi trường làm việc hoặc tăng đa dạng hình ảnh khi huấn luyện.

Chi tiết 22 lần thất bại (khung cảnh Galileo)

12 **Không nhấc được táo** –

9 lần táo gần như không đổi chỗ; 2 lần táo bị đẩy trượt 4–9 cm; 1 lần táo bị hất rơi khỏi kệ.

Nguyên nhân khả dĩ: nắm trượt hoặc tiếp cận sai. Vị trí ban đầu giống nhau ⇒ khác biệt đến từ tính ngẫu nhiên khi model lấy mẫu.

8 **Nhấc được nhưng đặt trượt** –

6 lần thả cách đĩa 9–20 cm (gần nhất 9,1 cm, vừa vượt ngưỡng 8 cm); 2 lần làm rơi táo khỏi kệ.

2 **Ngã ngay đầu episode** –

Ngã ở tick 7 và 30, trước khi chạm táo. Chunk tay đầu tiên chồng lên giai đoạn AGILE còn đang ổn định sau reset. Đề xuất cho robot "khởi động" vài chục tick trước khi giao quyền cho model.

Các yếu tố đã loại trừ

ĐỘ TRỄ

Mô phỏng chờ server; độ trễ của episode thành công và thất bại như nhau.

LỚP AN TOÀN

Chỉ cắt khớp tay ở biên chunk, chưa lần nào cắt bàn tay hay eo.

LỖI PIPELINE

28 lần thành công với khoảng cách cuối 1–6 cm ⇒ chuỗi dữ liệu hoạt động đúng.

10 Kết luận kịch bản

KẾT LUẬN

Model fine-tune trên một khung cảnh **rất nhạy với hình ảnh**: 92 % ở môi trường gốc lúc ghi dữ liệu, 56 % khi chỉ lệch ánh sáng, 0 % khi khác hẳn. Triển khai thực tế phải thu dữ liệu ngay trong môi trường làm việc hoặc tăng đa dạng hình ảnh khi huấn luyện.

- Đã **làm chủ và tái hiện độc lập** pipeline; mã nguồn sẵn sàng thay backend robot thật.
- Phải dùng đúng câu lệnh lúc ghi dữ liệu; mỗi cấu hình cần ≥ 50 episode để kết luận.
- Chưa thực hiện: chạy pipeline của dự án bên trong Docker Arena gốc để đối chứng trực tiếp với mốc 92 %; khung cảnh Galileo với xáo ±2 cm; giai đoạn khởi động sau reset.

III.02 Kịch bản Multi Fruit

Robot G1 chọn đúng loại quả theo câu lệnh, đặt lên đĩa.

Trên bàn có 4 loại quả (táo, lê, khế, nho); câu lệnh quyết định quả cần gắp. Từ 1.095 quỹ đạo teleop trên robot G1 thật, dự án **tái tạo khung cảnh mô phỏng chỉ từ dữ liệu khớp** (real2sim), sinh 790 episode huấn luyện và fine-tune GR00T. Kết quả: checkpoint táo dùng lại đạt **0 %**; model fine-tune đạt **66 %** trên khung cảnh ngẫu nhiên có quả gây nhiễu và **không lần nào gắp nhầm**.

DATASET

790 episode

Nguồn	**Real2sim · G1 thật**
Frame	**86.822**
Tập test	**89 episode · 20 Hz**

FINETUNE

20.000 bước

Thời gian	**2 giờ 34 phút**
Batch · GPU	**32 · 1× H100**
Loss	**1,50 → 0,019**

MODEL

GR00T N1.7 3B

Checkpoint	`g1_fruits_n17`
Câu lệnh	**4 loại quả**
Chuỗi action	**16 bước · 20 Hz**

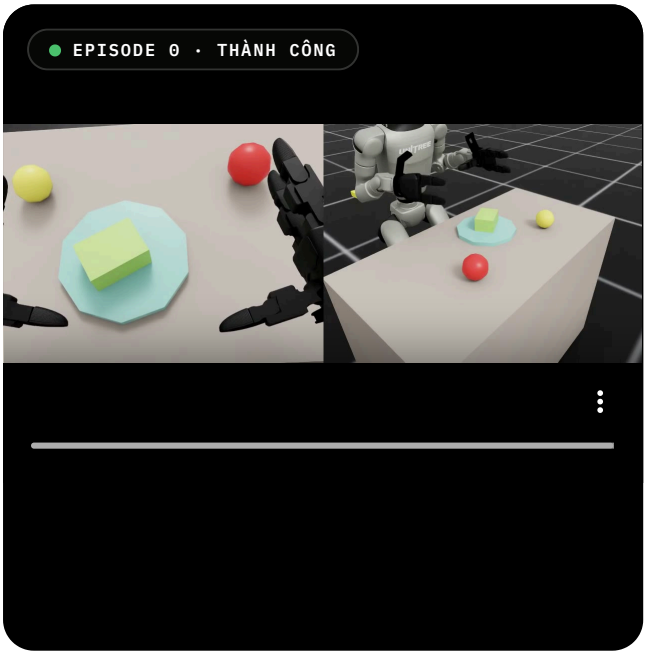
RESULT

66 % thành công

0 %	66 %
CHƯA TUNE	FINE-TUNE

Không lần nào gắp nhầm quả

1 Video Showcase



Robot chọn đúng chùm nho giữa các quả khác và đặt lên đĩa

CÂU LỆNH GỬI MODEL

"Pick up the green grapes and place it on the plate"

1,4 cm
Tâm quả cách tâm đĩa

16,5 cm
Độ nâng cao nhất

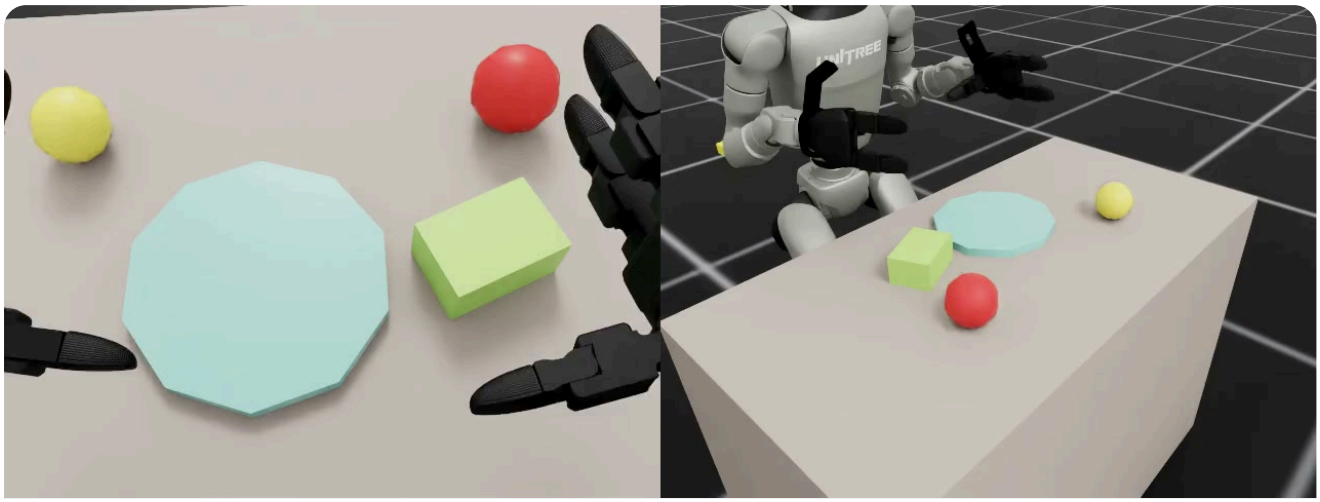
2
Quả gây nhiễu, không bị xô lệch

10 s
Độ dài episode (200 bước × 50 ms)

Trích từ đợt 50 episode real2sim ngẫu nhiên (33/50 thành công). Nho là khối xanh lá.

2 Bối cảnh kịch bản

Robot G1 đứng cố định trước một bàn; trên bàn có một đĩa và ba quả, trong đó một quả là mục tiêu và hai quả còn lại gây nhiễu. Ảnh dưới là **khung đầu tiên** của episode trong Video Showcase (episode 0: nho xanh).



Khung 0 · trái: camera đầu robot, ảnh model nhận vào · phải: camera ngoài, chỉ dùng để quay video. Mục tiêu là nho (khối xanh lá), gây nhiễu là táo và lê.

KHUNG CẢNH & VẬT THỂ	CHI TIẾT
Bàn	Bàn đơn giản trên nền lưới; ánh sáng dome 700 + sun 600
Đĩa	Đĩa kinematic bán kính 0,115 m; episode 0 đặt tại (0,40; 0,08) m
Quả	4 loại dạng khối đơn giản (táo, lê, khế, nho = khối xanh lá); ma sát 1,5
Mục tiêu	Episode 0: nho tại (0,41; −0,12) m
Gây nhiễu	2 quả đặt ngoài vùng quét của tay (episode 0: táo, lê)
Ngẫu nhiên hoá	Mỗi episode đổi quả mục tiêu, vị trí và hướng quả, vị trí đĩa, quả gây nhiễu; seed riêng

ROBOT, CAMERA & ĐIỀU KHIỂN	CHI TIẾT
Robot	Unitree G1 29 bậc tự do + bàn tay Dex3, hông cố định tại (0; 0; ~0,74) m
Camera	RealSense D435 gắn đầu, 640×480 (fx 606) + 1 camera ngoài để quay
Tần số	Vật lý 200 Hz, 10 bước con → điều khiển 20 Hz
Episode	Tối đa 200 bước = 10 s; nhận chuỗi 16 bước, thực thi 8 rồi hỏi lại
Câu lệnh	Theo quả mục tiêu, vd. "Pick up the green grapes and place it on the plate"
Thành công	Quả được nhấc > 3 cm, nằm trên đĩa (trong bán kính + 1 cm, điểm thấp nhất cao hơn mặt bàn), quả gây nhiễu xê dịch ≤ 1 cm

3 Mục tiêu kịch bản

Chuyển dữ liệu teleop của robot G1 thật vào mô phỏng (**real2sim**) để huấn luyện và đánh giá một model chọn đúng loại quả theo câu lệnh.

MỤC TIÊU 1

Đánh giá vòng kín

Có môi trường mô phỏng cho tác vụ vốn chỉ có dữ liệu robot thật.

MỤC TIÊU 2

Nhân dữ liệu

Thay vật nhiễu, vị trí, ánh sáng mà không cần teleop thêm.

MỤC TIÊU 3

Đo lặp lại được

So sánh checkpoint và cấu hình huấn luyện trên cùng một thước đo.

4 Dữ liệu kịch bản

Dataset **PhysicalAI-Robotics-GR00T-Teleop-G1** (NVIDIA GEAR, CC-BY-4.0): teleop robot G1 thật, camera RealSense 640×480, 20 Hz. Robot chọn đúng quả theo câu lệnh và đặt lên đĩa. Dữ liệu **chỉ có góc khớp và video**, không có vị trí vật thể.

LOẠI QUẢ	EPISODE	FRAME
Táo	311	29.864
Nho	233	26.970
Lê	264	32.319
Khế	287	34.640
Tổng (card dataset ghi 1.000)	**1.095**	**123.793**

5 Vấn đề gặp phải

Mục tiêu là một model gắp đúng quả theo câu lệnh. Trước khi tự tạo dữ liệu, dự án thử fine-tune trên các dataset có sẵn.

> **Không có dataset mô phỏng phù hợp.** Bộ duy nhất đúng robot G1 và đúng dạng bài toán gắp-đặt lại được ghi trên **robot thật**, nên không dùng thẳng trong Isaac Sim được.

VẤN ĐỀ 1

Checkpoint có sẵn không dùng lại được

Checkpoint táo của kịch bản 1 với câu lệnh 4 loại quả đạt **0 %**: model chỉ học một tác vụ trong một khung cảnh.

VẤN ĐỀ 2

Thiếu dataset mô phỏng

Rà soát các dataset công khai để fine-tune thử: không có bộ mô phỏng nào cho G1 + Dex3 gắp-đặt nhiều loại quả theo câu lệnh.

VẤN ĐỀ 3

Bộ phù hợp duy nhất là dữ liệu thật

PhysicalAI-Robotics-GR00T-Teleop-G1: cùng robot G1, cùng gắp-đặt quả lên đĩa, nhưng ghi trên robot thật.

VẤN ĐỀ 4

Dữ liệu thật thiếu thông tin cho mô phỏng

Chỉ có góc khớp và video, **không có vị trí vật thể**; ảnh thật không khớp với ảnh render; action trên robot thật và trong mô phỏng mang ý nghĩa khác nhau.

6 Phương pháp đề xuất

Giải quyết bằng **real2sim**: dùng quỹ đạo khớp của dữ liệu thật để dựng lại khung cảnh mô phỏng tương ứng, phát lại trong Isaac Lab và ghi thành dataset mô phỏng — giữ chuyển động thật của người teleop, nhưng ảnh và vật lý nhất quán với môi trường đánh giá.

BƯỚC 1
Động học thuận (FK)
URDF G1 29-DoF + Dex3 → tư thế thân, quỹ đạo bàn tay, thời điểm nắm/thả

→

BƯỚC 2
Suy ra vị trí vật
Quả = lòng bàn tay lúc nắm; đĩa = vị trí lúc thả

→

BƯỚC 3
Vật nhiễu an toàn
Đặt ngoài vùng tay quét qua, đường mang quả và đĩa

→

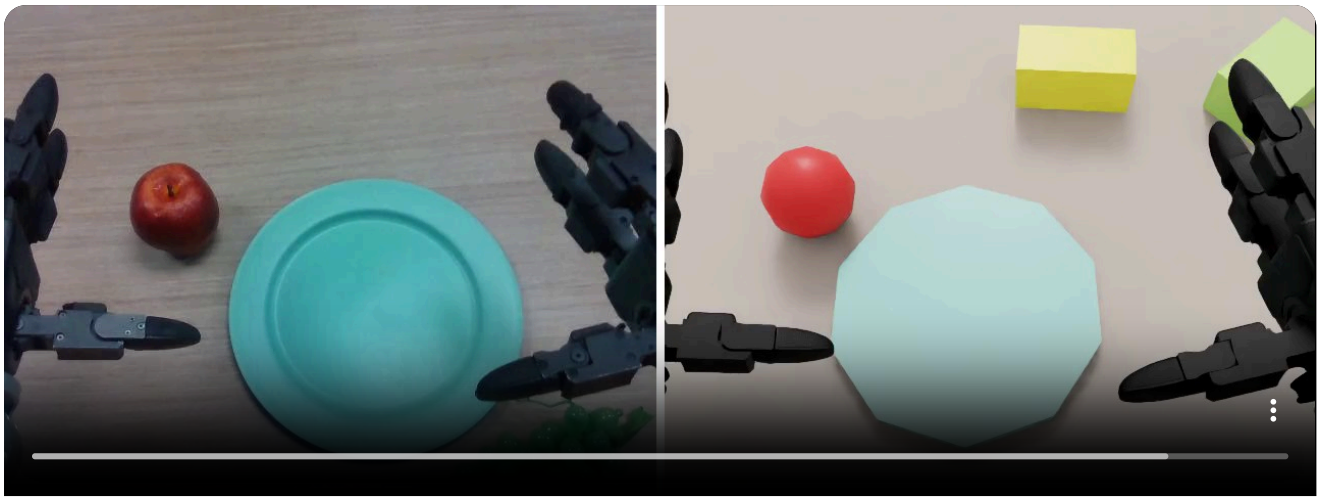
BƯỚC 4
Phát lại & lọc
Isaac Lab; giữ episode thành công, không chạm vật nhiễu

→

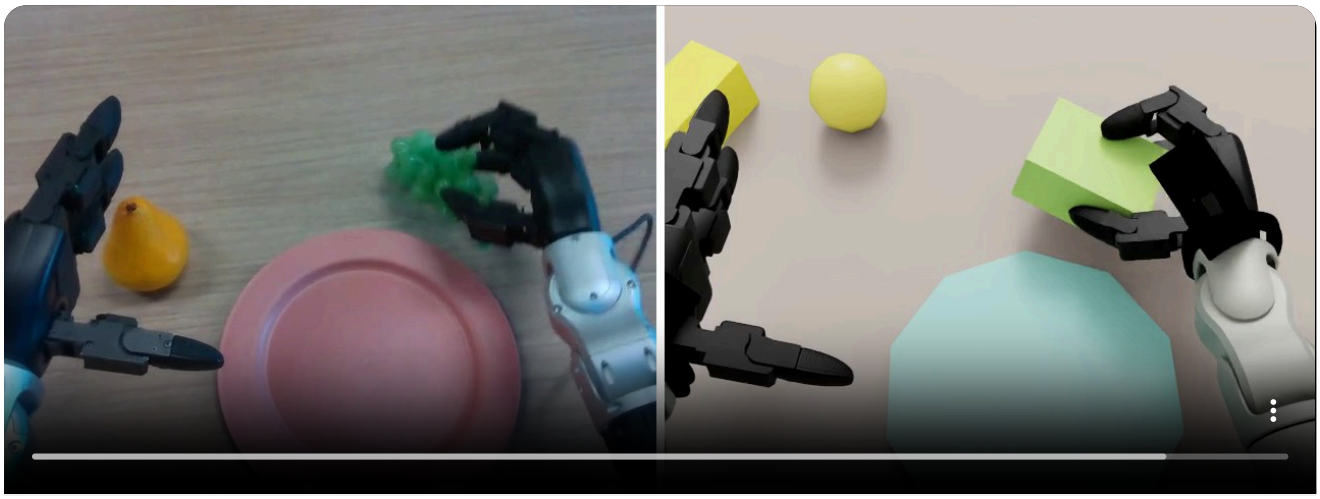
BƯỚC 5
Dataset LeRobot
Đa nhiệm 4 quả; chia train/test theo episode gốc

- **Tái tạo không dùng thị giác:** giả định hai bàn chân phẳng trên sàn để suy ra góc nghiêng thân; phát hiện nắm/thả từ độ khép ngón tay; bộ FK viết riêng chỉ dùng numpy.
- **Mô phỏng:** G1 + Dex3 pelvis cố định, camera D435 theo thông số RealSense, vật lý 200 Hz / điều khiển 20 Hz, PD cứng cho tay, ma sát quả 1,5, đĩa cố định.
- **Chấm điểm:** quả nhấc > 3 cm; cuối cùng nằm trên đĩa; không vật nhiễu nào dịch > 1 cm.
- **Chống rò rỉ dữ liệu:** khung cảnh test không bao giờ có trong train; giữ lại 100 khung cảnh đánh giá kể cả khung cảnh phát lại thất bại để không thiên về khung cảnh dễ.

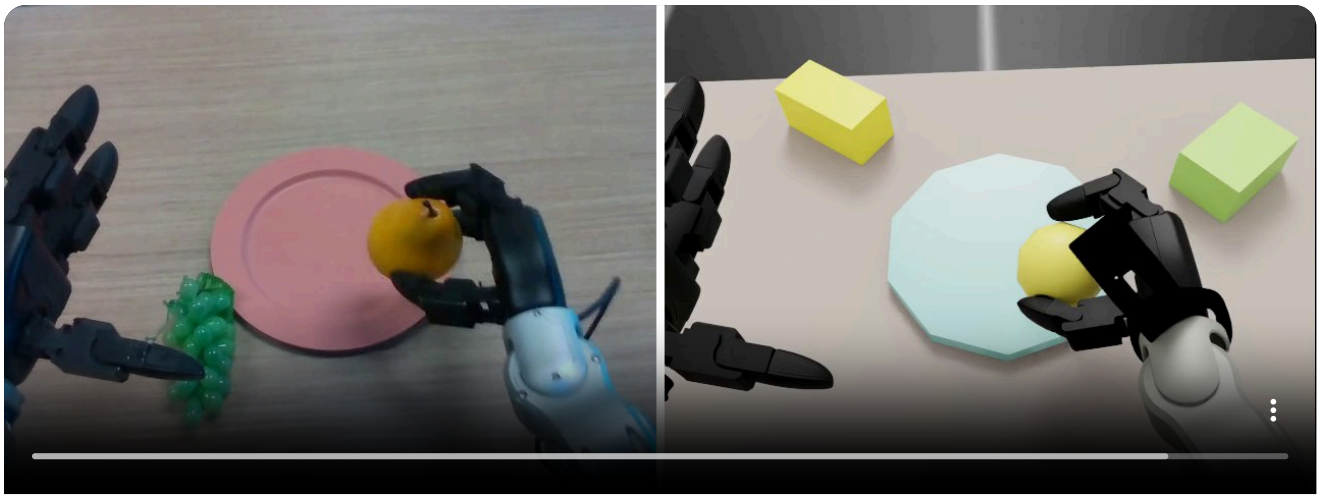
Ba cặp video thật ↔ mô phỏng. Camera đầu robot, 20 Hz: trái là video teleop gốc, phải là episode real2sim được phát lại từ chính quỹ đạo khớp đó. Hai episode dài bằng nhau nên chạy khớp từng khung; ảnh chờ của mỗi video là khung tiêu biểu của một pha (bắt đầu, nắm, đặt).



Cặp 1 · Táo · episode thật 0 ↔ sim 0 · 111 khung (5,6 s). Ngay khung đầu, vị trí táo và đĩa trong mô phỏng đã gần trùng ảnh thật dù không dùng thị giác: táo suy từ lòng bàn tay lúc nắm, đĩa từ vị trí lúc thả. Chùm nho ở góc ảnh thật không có trong dữ liệu khớp nên không được tái tạo; mô phỏng đặt 2 vật nhiễu mới (khế, nho) ở xa vùng tay quét. Trái: thật · phải: mô phỏng.



Cặp 2 · Nho · episode thật 1 ↔ sim 0 · 102 khung (5,1 s). Tư thế cánh tay và bàn tay Dex3 lúc vươn, khép ngón, nhấc đều giống video thật vì cùng góc khớp. Chùm nho mềm được thay bằng khối xanh lá cứng; quả lê thật bên trái không được tái tạo, mô phỏng đặt lê và khế làm vật nhiễu ở vị trí khác. Trái: thật · phải: mô phỏng.



Cặp 3 · Lê · episode thật 0 ↔ sim 0 · 131 khung (6,6 s). Quả được mang đến vùng đĩa cùng thời điểm với video thật. Đĩa thật màu hồng, đĩa mô phỏng luôn là đa giác xanh ngọc; mặt bàn gỗ thành mặt phẳng trơn; góc nhìn mô phỏng thấy cả mép bàn và nền phía sau. Trái: thật · phải: mô phỏng.

GIỐNG

Chuyển động và bố cục tác vụ

Quỹ đạo cánh tay, độ khép ngón, thời điểm nắm/thả giữ nguyên từ teleop; vị trí quả mục tiêu và đĩa khớp với ảnh thật; tay trái nghỉ cùng tư thế; câu lệnh giữ nguyên văn.

KHÁC

Hình ảnh và vật nhiễu

Quả là khối hình học đơn giản; đĩa, mặt bàn, ánh sáng là của mô phỏng; vật nhiễu trong ảnh thật bị bỏ và thay bằng 2 vật nhiễu ngẫu nhiên đặt an toàn; góc camera lệch nhẹ. Vì vậy model chỉ học trên ảnh render, không dùng ảnh thật.

TẬP	EPISODE	FRAME	GHI CHÚ
Train	**790**	**86.822**	**4 loại quả, 20 Hz**
Test	89	—	Khung cảnh không có trong train
Khung cảnh đánh giá	100	—	30 táo, 24 lê, 26 khế, 20 nho

7 Chu trình thực hiện

Dataset gốc là teleop trên robot G1 thật nên không chạy thẳng trong mô phỏng được: phải **chuyển real2sim** để có khung cảnh và dữ liệu mô phỏng tương ứng, **fine-tune** GR00T trên dữ liệu đó, rồi mới chạy vòng kín trong Isaac Sim.

BƯỚC 1
Dữ liệu thật
1.095 quỹ đạo teleop G1, chỉ góc khớp + video

→

BƯỚC 2
Chuyển real2sim
FK suy vị trí quả, đĩa; phát lại & lọc trong Isaac Lab

→

BƯỚC 3
Dataset mô phỏng
LeRobot đa nhiệm · train 790 / test 89

→

BƯỚC 4
Fine-tune
GR00T N1.7 · 20.000 bước · 1× H100

→

BƯỚC 5

Chạy & đánh giá

Khung cảnh ngẫu nhiên, 50 episode

BƯỚC	VIỆC LÀM	NOTEBOOK / CÔNG CỤ	KẾT QUẢ
1	Tải dataset teleop 4 loại quả (táo, lê, khế, nho); thử checkpoint táo của kịch bản 1	PhysicalAI-Robotics-GR00T-Teleop-G1	Checkpoint táo dùng lại: 0 %
2	Động học thuận URDF → quỹ đạo bàn tay, thời điểm nắm/thả → vị trí quả và đĩa; đặt vật nhiễu ngoài vùng tay quét; phát lại action trong Isaac Lab, giữ episode thành công	`g1_real2sim` (urdf_fk, scene, distractors, isaac_scene)	Khung cảnh mô phỏng cho từng quỹ đạo thật
3	Gộp 4 loại quả thành một dataset đa nhiệm; chia train/test theo episode gốc; giữ 100 khung cảnh đánh giá	make_groot_dataset, lerobot_io	790 / 86.822 frame train · 89 test
4	Fine-tune GR00T N1.7-3B, `NEW_EMBODIMENT`, 31 chiều state/action, horizon 16 bước @ 20 Hz, batch 32	launch_finetune.py trên server H100	`g1_fruits_n17` · 2 giờ 34 phút · loss 1,50 → 0,019
5	Dựng khung cảnh giống dataset, chọn câu lệnh, model tự gắp; chạy N episode với seed riêng	nb09_multi_object_task_pipeline, nb10_eval_multi_object	66 %, không lần nào gắp nhầm quả

8 Fine-tune

Theo hướng dẫn chính thức của NVIDIA (fine-tuning on sim data): GR00T N1.7-3B, `NEW_EMBODIMENT`, camera `rs_view`, 5 nhóm state/action (31 chiều), horizon 16 bước @ 20 Hz (0,8 s); batch 32 trên H100; đóng băng LLM, tune thị giác + projector + diffusion; color jitter (sáng 0,3 · tương phản 0,4 · bão hoà 0,5 · sắc 0,08).

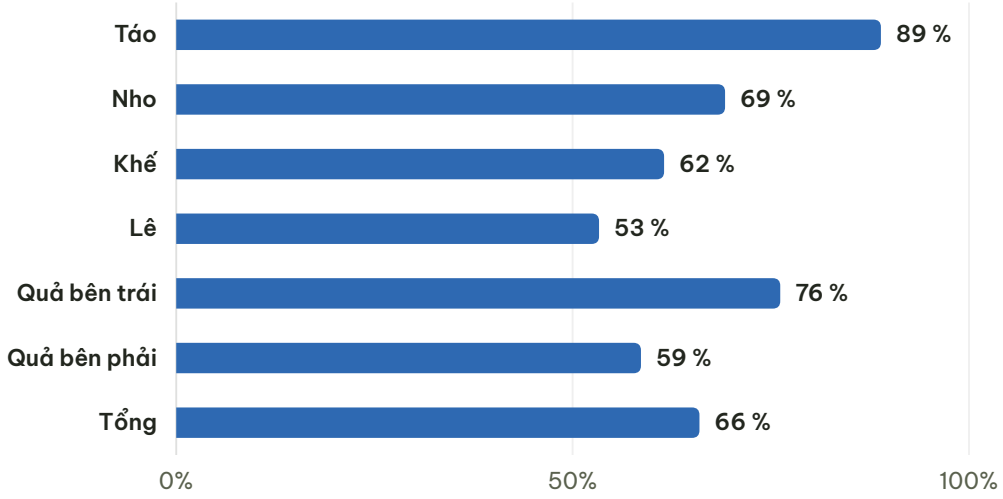
9 Kết quả đạt được

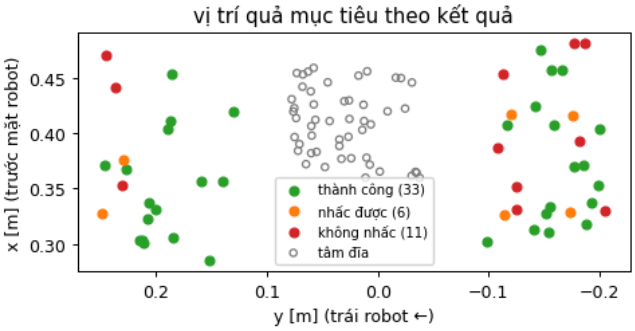
Mỗi episode ngẫu nhiên hoá quả mục tiêu, vị trí/hướng quả, vị trí đĩa và quả gây nhiễu; seed riêng cho từng episode. Câu lệnh nguyên văn dataset (vd. "Pick up the green grapes and place it on the plate"). Nhận 16 bước, thực thi 8 rồi hỏi lại; tối đa 10 s. 50 episode, 0 lỗi kỹ thuật.

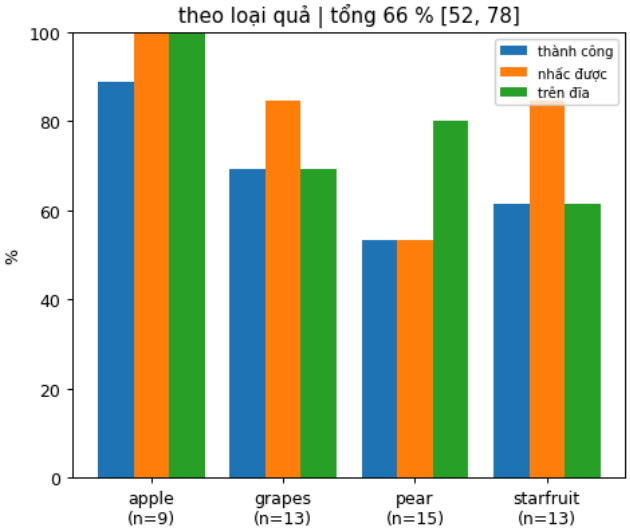
MODEL	THÀNH CÔNG	WILSON 95 %	NHẮC ĐƯỢC	TRÊN ĐĨA	GẮP NHẦM
Fine-tune real2sim	**33/50 (66 %)**	**52–78 %**	**78 %**	**76 %**	**0 %**
Checkpoint táo của Arena (đối chứng)	0/50 (0 %)	0–7 %	0 %	0 %	0 %

Tỉ lệ thành công theo nhóm · model real2sim

Di chuột lên thanh để xem số lần thử. n nhỏ (9–29) ⇒ khoảng tin cậy rộng.







Vị trí quả mục tiêu theo kết quả (xanh = thành công, cam = nhấc được nhưng không thành công, đỏ = không nhấc được, vòng xám = tâm đĩa) và tỉ lệ theo loại quả.

NHẬN XÉT

Model hiểu câu lệnh

Không lần nào gắp nhầm dù luôn có quả khác trên bàn.

NHẬN XÉT

Thất bại chủ yếu ở pha nhấc

11/50 không nhấc được; lê có tỉ lệ nhấc thấp nhất (53 %).

LƯU Ý

Đối chứng là model tác vụ khác

Không phải N1.7 gốc chưa fine-tune; 0 % chỉ cho thấy không dùng lại được model tác vụ khác.

Bài học kỹ thuật: lần chạy thử đầu tiên dùng cấu hình không render camera mỗi bước điều khiển (đặc thù Isaac Lab 3.0) — model nhìn một ảnh đứng yên. Đã phát hiện và sửa; kết quả lần thử đó không được dùng.

10 Phân tích kết quả

Theo dõi 50 episode của model fine-tune qua từng pha để thấy hao hụt nằm ở đâu.

PHA (50 EPISODE)	SỐ EPISODE	TỈ LỆ
Nhấc được quả mục tiêu	39	78 %
Nhấc được và quả nằm trên đĩa	34	68 %
Thành công (không xê dịch vật nhiễu)	**33**	**66 %**

LOẠI THẤT BẠI (17 LẦN)	SỐ LẦN	BIỂU HIỆN
Không nhấc được	11	Nắm trượt hoặc không khép đủ tay; lê có tỉ lệ nhấc thấp nhất (53 %)
Nhấc nhưng không lên đĩa	5	Thả lệch đĩa; khế lên đĩa thấp nhất (62 %) dù nhấc được 85 %
Lên đĩa nhưng xê dịch vật nhiễu	1	Tay chạm quả gây nhiễu > 1 cm

NHẬN ĐỊNH 1

Model hiểu câu lệnh

0 lần gắp nhầm trong 50 episode luôn có quả khác trên bàn: phần ngôn ngữ được học tốt từ dữ liệu real2sim.

NHẬN ĐỊNH 2

Nút thắt là pha nắm

11/17 thất bại là không nhấc được quả. Cải thiện tiếp nên tập trung vào dữ liệu nắm (hình dạng, ma sát từng loại quả).

NHẬN ĐỊNH 3

Khác biệt theo loại quả và phía bàn

Táo 89 %, nho 69 %, khế 62 %, lê 53 %; bên trái 76 %, bên phải 59 %. n nhỏ, khoảng tin cậy chồng nhau — chưa đủ kết luận.

NHẬN ĐỊNH 4

Phạm vi kết luận

Huấn luyện và đánh giá cùng trong mô phỏng: kết quả dùng để so sánh tương đối, không dự đoán hiệu năng trên robot thật.

11 Kết luận kịch bản

KẾT LUẬN

Từ dữ liệu teleop của robot G1 thật chỉ có góc khớp và video, dự án đã **dựng được dataset mô phỏng bằng real2sim** (790 episode) và fine-tune GR00T: từ **0 %** với checkpoint táo dùng lại lên **66 %** trên khung cảnh ngẫu nhiên có quả gây nhiễu, **không lần nào gắp nhầm**. Model đã hiểu câu lệnh; phần còn hao hụt chủ yếu nằm ở pha nắm.

- Xây dựng được quy trình **real2sim chỉ từ dữ liệu khớp**: động học thuận suy vị trí quả và đĩa, đặt vật nhiễu an toàn, phát lại và lọc tự động trong Isaac Lab.
- Fine-tune GR00T N1.7 trong 2 giờ 34 phút trên một H100; 11/17 thất bại là không nhấc được quả, lê có tỉ lệ nhấc thấp nhất.
- Kết quả chỉ dùng để **so sánh tương đối** trong mô phỏng: mô phỏng dễ hơn thực tế (PD cứng, ma sát cao, đĩa cố định), không dự đoán hiệu năng trên robot thật.
- Hướng tiếp theo: tăng dữ liệu rẻ bằng cách đổi bố cục vật nhiễu, ánh sáng, chất liệu rồi phát lại; bổ sung dữ liệu nắm theo hình dạng và ma sát từng loại quả.

III.03 Kịch bản Hammer & Bin

Robot G1 gắp búa trên kệ, thả vào thùng.

Tác vụ hoàn toàn mới do dự án tự xây dựng: tay trái nắm cán búa, nhấc qua thành thùng, thả vào giữa thùng và rút tay. Khó hơn quả táo vì búa **dài, mỏng, sát mặt kệ** và dễ gác lên mép thùng. Dự án dựng môi trường, thu **100 demo teleop** bằng kính Meta Quest, sinh thêm demo và fine-tune GR00T (~1 giờ trên 1 H100) — trọn chu trình trong khoảng 6 ngày. Kết quả: chưa tune **0 %** → 200 episode **66 %** → 400 episode **63 %** → nới ngưỡng đứng yên **73 %**.

DATASET

200 · 400 episode

Nguồn	**Teleop + sinh thêm**
Frame	**28.022 · 54.541**
Tần số	**50 Hz**

FINETUNE

10k · 20k bước

Thời gian	**61 phút · 2 giờ 15**
Batch · GPU	**32 · 1× H100**
Loss cuối	**0,020 · 0,019**

MODEL

GR00T N1.7 3B

mix200	`hammer_mix200_n17`
mix400	`hammer_mix400_n17`
Chuỗi action	**40 bước · 50 Hz**

RESULT

73 % thành công

0 %	66 %	63 %	73 %
GỐC	200 …	400 …	NGƯỠ…

Nới ngưỡng đứng yên 0,01 → 0,02

1 Video Showcase



Robot nắm cán búa, nhấc qua thành thùng, thả vào giữa thùng và buông tay

CÂU LỆNH GỬI MODEL
"put the hammer in the bin"

3,0 s
Thời gian tới thành công

12,8 cm
Độ nâng búa cao nhất

0,7 cm
Tâm búa cách tâm thùng

0 N
Lực tay lên búa: đã buông

Trích từ đợt 100 episode ở cấu hình tốt nhất (73/100).

2 Bối cảnh kịch bản

Robot G1 đứng trước kệ trong phòng lab Galileo; chiếc búa nằm bên trái, đầu búa kê lên một khối nhỏ, thùng nhựa đỏ ở bên phải. Ảnh dưới là **khung đầu tiên** của episode trong Video Showcase.



Khung 0 · trái: camera đầu robot, ảnh model nhận vào · phải: camera ngoài, chỉ dùng để quay video.

KHUNG CẢNH & VẬT THỂ	CHI TIẾT
Nền	Phòng lab Galileo; ẩn thêm các hộp shield_controller_box_02, jetson_orin_05, jetson_orin_07 để tay không va chạm (khe hở ≥ 2 cm)
Búa	`hammer_2_handal_robolab` 33 × 13 × 3 cm tại (0,63; 0,25) m, yaw 0; đầu búa tựa lên khối kê 5 cm (nghiêng ~10°) để dễ nắm cán
Thùng	`red_container` tỉ lệ 0,65 → 40 × 27 × 5,9 cm tại (0,68; 0,00) m
Khoảng cách	Búa cách mép thùng 5,1 cm

ROBOT, CAMERA & ĐIỀU KHIỂN	CHI TIẾT
Robot	Unitree G1 + Dex3 tại (0,25; 0,08) m; tay trái nắm cán, nhấc qua thành thùng cao 5,9 cm, dịch sang phải ~25 cm rồi thả
Camera	Camera đầu robot (ảnh model nhận vào) + 1 camera ngoài để quay
Tần số	Điều khiển 50 Hz; mỗi lần gọi model trả về chuỗi 40 bước
Episode	Tối đa 300 bước = 6 s
Câu lệnh	"put the hammer in the bin"
Thành công	Tâm búa cách tâm thùng ≤ 10 cm và thấp hơn thành; búa đứng yên (< 0,01 m/s, < 0,1 rad/s — sau nới lên 0,02 m/s); tay đã buông (22 khâu tay ≤ 0,01 N); giữ liên tục 10 bước

3 Mục tiêu kịch bản

Kịch bản kiểm chứng năng lực **dạy robot một tác vụ mới từ đầu** bằng công cụ và dữ liệu do dự án tự xây dựng.

MỤC TIÊU 1

Tác vụ hoàn toàn mới

Tự đặt ra một tác vụ chưa có dữ liệu hay model sẵn: gắp búa trên kệ, thả vào thùng.

MỤC TIÊU 2

Trọn chu trình

Đi hết các khâu: dựng môi trường, teleop thu demo, tăng cường dữ liệu, fine-tune, đánh giá.

MỤC TIÊU 3

Đo chi phí và hiệu quả

Biết cần bao nhiêu dữ liệu, thời gian huấn luyện, và mỗi cải tiến đóng góp bao nhiêu vào tỉ lệ thành công.

4 Dữ liệu kịch bản

Kịch bản mới nên không có dataset sẵn: dự án tự thu demo bằng **teleop kính Meta Quest** trong Isaac Lab, 50 Hz, rồi bổ sung demo sinh thêm (card 6). Mọi demo đều được chạy lại với điều kiện thành công chặt trước khi đưa vào huấn luyện.

DATASET HAMMER_TO_BIN	GIÁ TRỊ
Nguồn	Teleop kính Meta Quest trong Isaac Lab, 50 Hz; 100 demo thật đầu tiên thu trong 10 phiên
Demo thật	~13.700 bước; búa nhấc trung bình 10,6 cm; tâm búa cuối cách tâm thùng 3,7 cm (tối đa 8,4 cm)
Bộ mix200	200 episode (100 thật + 100 tăng cường), 28.022 frame (~9,3 phút), LeRobot v2.1, 131 MB
Bộ mix400	400 episode (mix200 + 200 demo teleop mới), 54.541 frame (~18 phút)
Câu lệnh	"put the hammer in the bin"
Kiểm tra	Toàn bộ demo thành công; với mix200: búa nhấc tối thiểu 6,4 cm, khoảng cách cuối trung bình 3,8 cm

5 Vấn đề gặp phải

Tác vụ búa → thùng hoàn toàn mới: không có dữ liệu, không có model, không có khung cảnh sẵn. Khó khăn

lớn nhất nằm ở việc có đủ dữ liệu tốt để fine-tune.

> **Không có gì để bắt đầu.** Mọi dữ liệu phải tự thu bằng teleop, và đây là khâu tốn công nhất: 100 demo thật mất 10 phiên.

VẤN ĐỀ 1

Không có dữ liệu sẵn

Không có dataset hay checkpoint nào cho tác vụ gắp búa đặt vào thùng trên G1; model gốc chưa fine-tune không làm được tác vụ.

VẤN ĐỀ 2

Teleop tốn công và chậm

Người điều khiển đeo kính Meta Quest thao tác từng demo; mỗi phiên chỉ được khoảng 10 demo, cần nhiều phiên mới đủ dữ liệu.

VẤN ĐỀ 3

Teleop không ổn định

Tay robot đứng im khi mất tín hiệu tay cầm, giật lag từ lượt thứ 2; nhiều lượt phải bỏ và làm lại.

VẤN ĐỀ 4

Demo có chất lượng không đều

Có demo được chấm thành công khi búa còn gác lên thành thùng hoặc tay chưa buông; demo hỏng lẫn vào dữ liệu làm model học sai.

6 Phương pháp đề xuất

Giải quyết bằng **sinh thêm dữ liệu**: từ các demo teleop thật, tự động tạo demo mới theo kiểu MimicGen, chạy lại trong mô phỏng và chỉ giữ demo đạt điều kiện thành công chặt. Cách này nhân dữ liệu mà không tốn thêm giờ teleop, và mọi demo đưa vào huấn luyện đều đã được kiểm tra.

CÁCH SINH

Biến đổi demo khuôn

Lấy demo thật làm khuôn (60 khuôn khác nhau); co giãn thời gian ×0,85–1,2; dịch điểm xuất phát (σ 1 cm), gắp (σ 4 mm), thả (±1–1,2 cm), nối bằng quỹ đạo min-jerk; thêm rung tay 0,2–0,6 mm.

LỌC

100/133 lượt giữ lại

Mỗi demo sinh ra được chạy lại trong mô phỏng với điều kiện thành công chặt; 100 trên 133 lượt đạt và được đưa vào bộ mix200.

CHỈ SỐ	TĂNG CƯỜNG	THẬT
Độ dài episode (bước)	145	137
Độ nâng búa	10,5 cm	10,6 cm
Khoảng cách tới tâm thùng, trung bình	4,0 cm	3,7 cm
Khoảng cách tới tâm thùng, tối đa	8,0 cm	8,4 cm



Một demo tăng cường: khung đầu, giữa, cuối. Tư thế ở các điểm then chốt lệch dưới 1 cm so với demo thật; dùng 60 demo khuôn khác nhau.

7 Chu trình thực hiện

Kịch bản mới hoàn toàn: không có dữ liệu hay model sẵn. Dự án dựng môi trường, **tự tạo dữ liệu bằng teleop**, **chạy tăng cường** để nhân dữ liệu, **fine-tune** GR00T, rồi mới chạy và đánh giá vòng kín.

BƯỚC 1 · 07/10
Thiết kế môi trường
Vật thể, bố trí, khối kê, điều kiện thành công →

BƯỚC 2 · 08–10/10
Teleop thu demo
Meta Quest · 100 demo thật →

BƯỚC 3 · 08–10/10
Tăng cường dữ liệu
Sinh kiểu MimicGen · 100/133 giữ lại →

BƯỚC 4 · 12/10
Fine-tune
mix200 10k bước · mix400 20k bước →

BƯỚC 5 · 12–19/10
Chạy & đánh giá
Arena và Isaac Sim local · 50–100 episode

BƯỚC	VIỆC LÀM	NOTEBOOK / CÔNG CỤ	KẾT QUẢ
1	Chọn búa, thùng; đặt búa tựa khối kê 5 cm; ẩn hộp nền vướng tay; viết điều kiện thành công chặt	Isaac Lab Arena, hammer_to_bin/scripts	Bố trí cuối, khe hở ≥ 2 cm
2	Thu demo bằng kính Meta Quest với preset và công cụ chẩn đoán teleop; chạy lại từng demo với điều kiện chặt	teleop_kit, record_demos.sh	100 demo thật (10 phiên), sau thêm 200 demo

BƯỚC	VIỆC LÀM	NOTEBOOK / CÔNG CỤ	KẾT QUẢ
3	Biến đổi demo khuôn (thời gian, điểm gắp/thả, rung tay), chạy lại và lọc	gen_scripted_demos.py	100 demo tăng cường → bộ mix200
4	Fine-tune GR00T N1.7-3B; LLM đóng băng, tune thị giác + projector + diffusion	Server H100	mix200: 61 phút · mix400: 2 giờ 15 phút
5	Chạy vòng kín N episode, phân loại thất bại, quay video	Arena eval, nb11_hammer_to_bin, nb12_eval_hammer_to_bin	0 % gốc → 66 % → 63 % → 73 %

8 Fine-tune

Thời gian huấn luyện

61 phút

1× H100 · 0,36 s/bước · ~53 GB VRAM

Số bước · batch

10k · 32

~11 epoch · checkpoint mỗi 2k bước

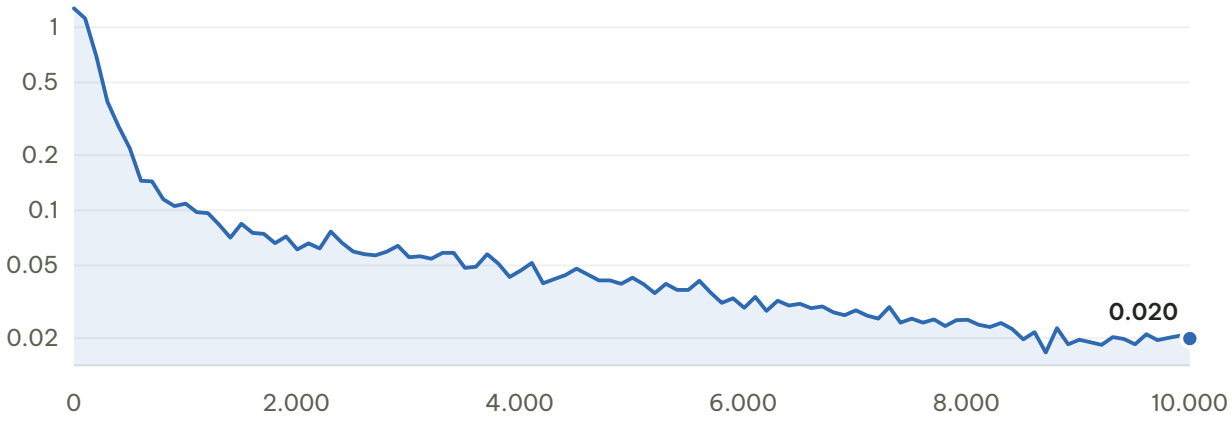
Loss cuối

0,020

từ 1,27 ở bước 10

Loss huấn luyện · thang log

GR00T N1.7-3B trên mix200. LLM đóng băng; tune thị giác + projector + diffusion. Di chuột để xem từng bước.



9 Kết quả đạt được

Số liệu đo được qua chu trình teleop → tăng cường → fine-tune → đánh giá, trong Isaac Sim local (điều kiện

`inside`: tâm búa trong lòng thùng, đứng yên, đã buông).

Tốt nhất · local

73 %

mix400 · ngưỡng 0,02 · 73/100

Local · mix200

66 %

66/100 · Wilson 56–75 %

Dữ liệu

400 demo

54.541 frame · 300 teleop + 100 sinh

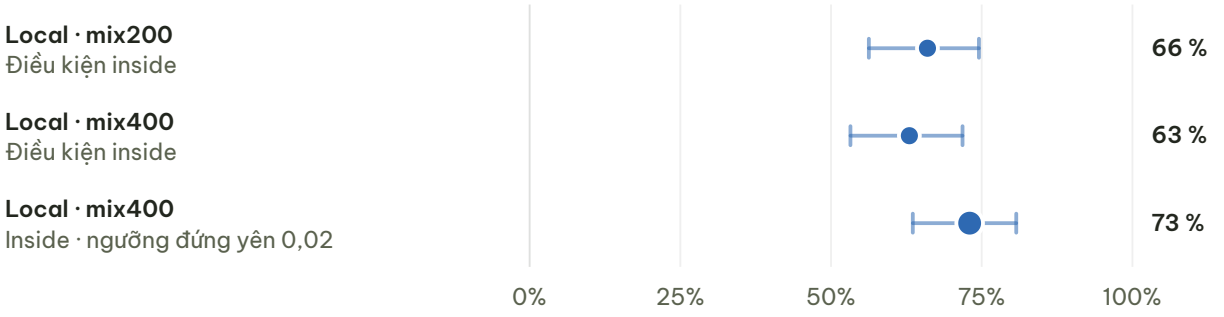
Loss fine-tune

0,019

mix400 · 20k bước · 2 giờ 15

Tỉ lệ thành công và khoảng tin cậy Wilson 95 %

Isaac Sim local · điều kiện inside · 100 episode mỗi lần chạy.

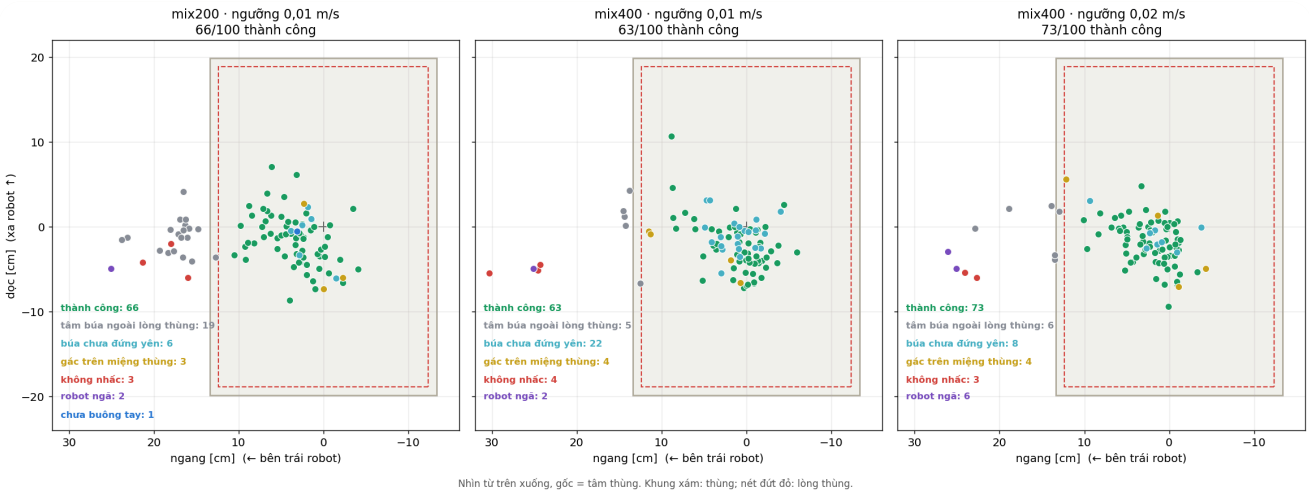


BƯỚC	SỐ LIỆU ĐO ĐƯỢC
Teleop	100 demo thật (10 phiên, ~13.700 bước); sau thêm 200 demo
Tăng cường	133 lượt sinh → 100 demo đạt điều kiện chặt
Dataset	mix200: 200 episode, 28.022 frame · mix400: 400 episode, 54.541 frame
Fine-tune	mix200: 10k bước, 61 phút, loss 0,020 · mix400: 20k bước, 2 giờ 15 phút, loss 0,019
Local · mix200	66/100; nhấc ≥ 3 cm 95/100; khoảng cách cuối trung vị 6,1 cm
Local · mix400	63/100; nhấc ≥ 3 cm 94/100; khoảng cách cuối trung vị 4,4 cm
Local · mix400, ngưỡng 0,02	**73/100; nhấc ≥ 3 cm 91/100; thành công trung bình 2,5 s**

10 Phân tích kết quả

Phân loại kết quả của ba lần chạy 100 episode trên Isaac Sim local để thấy thêm dữ liệu và nới ngưỡng thay đổi điều gì.

KẾT QUẢ (100 EPISODE)	MIX200	MIX400	MIX400 · ĐẾM THỦ CÔNG	MIX400 · NGƯỠNG 0,02
Thành công	**66**	**63**	**87**	**73**
Tâm búa ngoài lòng thùng	19	5	5	6
Búa chưa đứng yên khi hết giờ	6	22	0	8
Búa gác trên miệng thùng	3	4	2	4
Không nhấc được (< 1 cm)	3	4	4	3
Robot ngã	2	2	2	6
Chưa buông tay	1	0	0	0



Vị trí tâm búa lúc kết thúc episode, nhìn từ trên xuống, gốc là tâm thùng · trái: mix200 · giữa: mix400 · phải: mix400 với ngưỡng đứng yên 0,02 m/s. Ở mix200, nhiều lần búa được thả lệch về phía tay trái, ngay ngoài thành thùng (cụm xám); ở mix400, các điểm dồn quanh tâm thùng hơn. Với mix400 ngưỡng 0,01, phần lớn thất bại là búa đã nằm giữa thùng nhưng chưa đứng yên khi hết giờ (22 điểm xanh lam); ở lần chạy với ngưỡng 0,02, nhóm này giảm còn 8. Điểm đỏ (không nhấc được) và tím (robot ngã) là các lần búa chưa được đưa tới thùng.

NHẬN ĐỊNH 1

Pha nắm và nhấc đã ổn

91–95/100 lần nhấc búa ≥ 3 cm ở cả ba lần chạy; chỉ 3–4 lần không nhấc được. Hao hụt chủ yếu ở pha thả.

NHẬN ĐỊNH 2

Thêm dữ liệu giúp đặt chính xác hơn

mix400 giảm số lần búa rơi ngoài lòng thùng từ 19 xuống 5, khoảng cách cuối trung vị từ 6,1 xuống 4,4 cm.

NHẬN ĐỊNH 3

Ngưỡng đứng yên là nút thắt còn lại

Ảnh giữa cho thấy với mix400 (ngưỡng 0,01 m/s), cả 22 lần thất bại "chưa đứng yên" đều nằm trong lòng thùng và robot đã buông tay. Các điểm này còn gần tâm thùng hơn các lần thành công (trung vị 2,9 cm so với 4,3 cm). Búa chỉ còn chuyển động chậm tới hết 6 s (vận tốc cuối 0,005–0,023 m/s, quay 0,04–0,32 rad/s), nên chưa lần nào đạt điều kiện đứng yên. Đây là thất bại do cách chấm điểm hơn là do đặt sai chỗ. Ở lần chạy với ngưỡng 0,02 m/s (ảnh phải), nhóm này còn 8 và tỉ lệ lên 73 %.

NHẬN ĐỊNH 4

Vì sao búa nằm yên vẫn bị chấm là "chưa đứng yên"

Điều kiện đứng yên (giữ nguyên từ Arena) chỉ xét **vận tốc tức thời** của búa tại từng bước mô phỏng: phải đồng thời dưới 0,01 m/s và 0,1 rad/s. Cả 22 búa này đều nằm trong lòng thùng và tay đã buông; video các episode đã xem không thấy búa trôi. Nếu búa thật sự di chuyển với vận tốc đo được (0,5–2,3 cm/s) suốt khoảng 3,5 s từ lúc tới tâm thùng đến hết giờ, nó phải trôi xa nhiều centimet; thực tế khoảng cách tới tâm thùng lúc cuối chỉ lệch trung vị 1,2 cm so với lúc gần tâm nhất. Nghĩa là búa chủ yếu **rung tại chỗ**: mô phỏng vật lý không đưa vận tốc về đúng 0 khi búa nằm trên đáy lưới của thùng (12/22 búa nằm phẳng trên đáy, so với 8/63 lần thành công), và thùng cũng là vật thể động đặt trên kệ. Dao động nhỏ này vượt ngưỡng 0,01 m/s, nên ngưỡng tức thời quá chặt so với độ nhiễu của mô phỏng; cách chấm hợp lý hơn là xét búa có dịch chuyển trong một khoảng thời gian (ví dụ 0,5 s) hay không.

NHẬN ĐỊNH 5

Robot ngã nhiều hơn không phải do model hay ngưỡng

Lần chạy 73 % có 6 lần robot ngã, so với 2 ở hai lần chạy kia. Cả 6 lần đều xảy ra ngay đầu episode, trong 0,02–0,78 s (tối đa 39/300 bước), khi tay chưa chạm tới búa: búa vẫn nằm nguyên trên kệ, cách tâm thùng 25,6 cm. Lúc đó model gần như chưa kịp điều khiển, còn ngưỡng đứng yên chỉ dùng để chấm điểm cuối episode nên không ảnh hưởng tới việc ngã. Dấu hiệu này khớp với lỗi reset đã ghi nhận trong nb12: khi episode trước kết thúc với tay và thân ở xa tư thế mặc định, reset chưa xoá hết trạng thái cũ (hông còn gập, khớp còn vận tốc), và robot bước hoặc ngã ngay ở tick đầu. Ngay cả khi reset hai lần, phát lại 20 demo vẫn có 2/20 lần khởi đầu hỏng. Chênh lệch 6 so với 2 chưa có ý nghĩa thống kê (Fisher $p = 0{,}28$); tính chung ba lần chạy là 10/300 episode (≈ 3 %). Nếu bỏ các lần ngã này, lần chạy 73 % đạt 73/94 ≈ 78 %. Cần sửa khâu reset (ví dụ kiểm tra tư thế và vận tốc khớp trước khi bắt đầu episode) để loại nhiễu này khỏi đánh giá.

> **Lưu ý:** lần chạy với ngưỡng 0,02 là một đợt 100 episode mới, không phải chấm lại đợt cũ, nên còn chịu biến động ngẫu nhiên.

11 Kết luận kịch bản

KẾT LUẬN

Dự án đã **tự dạy G1 một tác vụ hoàn toàn mới**: từ 0 % khi chưa tune lên **66 %** chỉ với 200 episode (100 teleop + 100 sinh thêm) và ~1 giờ fine-tune trên một H100, trọn chu trình trong khoảng 6 ngày. Pha nắm và nhấc đã ổn; phần còn hao hụt nằm ở pha thả búa vào thùng và ở cách chấm điểm.

- Đi hết chu trình bằng công cụ tự xây dựng: dựng môi trường, teleop thu demo, **sinh thêm dữ liệu**, fine-tune, đánh giá.
- Sinh thêm demo giúp nhân dữ liệu mà không tốn thêm giờ teleop (133 lượt sinh → 100 demo đạt điều kiện chặt).
- Gấp đôi dữ liệu (mix400) giúp đặt búa chính xác hơn (ngoài lòng thùng 19 → 5 lần) nhưng tỉ lệ chung chưa tăng (63 %); với ngưỡng đứng yên 0,02 đạt **73 %**.
- Chưa xử lý: đổi cách chấm "đứng yên" sang xét dịch chuyển của búa trong một khoảng thời gian thay vì vận tốc tức thời; sửa khâu reset gây ngã đầu episode (≈ 3 % số episode).

IV.01 Sản phẩm đầu ra

Khách hàng nhận được gì, nằm ở đâu, dùng thế nào.

Toàn bộ sản phẩm của giai đoạn này nằm trong **một thư mục gồm 6 phần**, đánh số theo thứ tự nên đọc: báo cáo → mã nguồn → dữ liệu → model → kết quả → asset mô phỏng. Dữ liệu gốc và checkpoint được **cung cấp riêng**.

THƯ MỤC	GÓI CHÍNH	CUNG CẤP RIÊNG
shimizu-robot/	**6 phần**	**Dữ liệu gốc + checkpoint**

1 Cấu trúc gói bàn giao

Mỗi ô là một thư mục con. Bấm vào ô để xem danh sách file bên trong (card 3).

01 report/

Báo cáo

Báo cáo tương tác này (mở bằng trình duyệt, không cần cài đặt) và bản markdown để chỉnh sửa.

Xem nội dung →

02 codes/

Mã nguồn

Pipeline điều khiển độc lập, 12 notebook nghiên cứu, công cụ real2sim, bản vá Arena và bộ script teleop.

Xem nội dung →

03 datasets/

Dữ liệu huấn luyện

Dataset búa → thùng (200 và 400 episode) và dataset real2sim 4 loại quả (790 train + 89 test), định dạng LeRobot.

Xem nội dung →

04 models/

Model đã huấn luyện

3 checkpoint fine-tune (búa → thùng mix200, mix400; real2sim 4 loại quả), đủ để chạy GR00T server ngay.

Xem nội dung →

05 evals/

Kết quả đánh giá

Log từng episode, bảng tổng kết, đồ thị và video thành công / thất bại của 3 kịch bản.

Xem nội dung →

06 assets/

Asset mô phỏng

File 3D dùng khi dựng cảnh: bản sao offline từ NVIDIA (nền Galileo, búa, thùng, robot G1) cùng quả táo và cái đĩa.

Xem nội dung →

2 Đối tượng sử dụng theo vai trò

Không phải ai cũng cần cả gói. Bảng dưới gợi ý phần nên đọc trước theo vai trò.

PHẦN	QUẢN LÝ DỰ ÁN	KỸ SƯ PHẦN MỀM	KỸ SƯ AI / ROBOT	NGƯỜI THU DỮ LIỆU (TELEOP)
01 Báo cáo	✓	·	·	·
02 Mã nguồn		✓	✓	·

PHẦN	QUẢN LÝ DỰ ÁN	KỸ SƯ PHẦN MỀM	KỸ SƯ AI / ROBOT	NGƯỜI THU DỮ LIỆU (TELEOP)
03 Dữ liệu			✓	·
04 Model		·	✓	
05 Kết quả	✓		✓	
06 Asset		·	✓	

■ Cần dùng chính ☐ Tham khảo khi cần

3 Danh mục nội dung chi tiết

Bấm vào từng thư mục để xem các file chính bên trong.

01 report/ — Báo cáo

`Mở: index.html` `Không cần cài đặt`

- `index.html` — **báo cáo tương tác**: mục lục, video, bảng so sánh, xuất PDF
- `assets/`, `images/` — giao diện, video minh hoạ, hình ảnh
- `src/`, `build.py`, `README.md` — nội dung từng mục và cách chỉnh sửa báo cáo
- `markdown/` — bản báo cáo dạng văn bản (10 file) để trích dẫn, chỉnh sửa

02 codes/ — Mã nguồn

`Python` `84 unit test` `Không kèm môi trường đã cài`

- `standalone_g1_gr00t/` — pipeline điều khiển G1 bằng GR00T, không phụ thuộc Arena
  - `g1gr00t/` — package: kết nối server, ánh xạ khớp, quan sát, action, lớp an toàn, AGILE, backend Isaac Lab, runner
  - `notebooks/` — 12 notebook nghiên cứu (nb01 → nb12)
  - `configs/` — cấu hình kịch bản táo, búa và bộ giữ thăng bằng
  - `scripts/`, `tools/` — chạy vòng kín, đánh giá open-loop, tải tài nguyên, chạy notebook tự động
  - `tests/` — unit test và test trong mô phỏng
  - `README.md`, `Makefile` — cài đặt và các lệnh chạy
- `g1_real2sim/` — tái tạo khung cảnh từ dữ liệu G1 thật: FK từ URDF, vật nhiễu, khung cảnh Isaac Lab, đọc/ghi LeRobot, cấu hình GR00T
- `arena_bua_thung/` — mở rộng Isaac Lab Arena cho kịch bản búa
  - `patches/` — 4 bản vá môi trường: bố trí búa–thùng, khối kê, điều kiện thành công chặt
  - `teleop/` — script mở teleop, ghi, phát lại, gộp demo, chẩn đoán
  - `scripts/` — sinh demo tăng cường, kiểm tra dataset, kiểm chứng khung cảnh

03 datasets/ — Dữ liệu huấn luyện

`Định dạng LeRobot v2.1` `Video H.264 640×480`

THƯ MỤC	NỘI DUNG
`hammer_to_bin/static_hammer_mix200/`	200 episode (100 teleop thật + 100 sinh thêm), 28.022 frame @ 50 Hz
`hammer_to_bin/static_hammer_mix400/`	400 episode (mix200 + 200 demo teleop mới), 54.541 frame @ 50 Hz
`real2sim_multi_fruit/g1-fruits-sim-train/`	Tập huấn luyện mô phỏng từ dữ liệu G1 thật: 790 episode, 86.822 frame @ 20 Hz (táo 246, lê 211, khế 174, nho 159)
`real2sim_multi_fruit/g1-fruits-sim-test/`	Tập kiểm tra: 89 episode, 9.906 frame, khung cảnh không có trong tập huấn luyện; `test_scenes.json` là danh sách khung cảnh đánh giá

Mỗi dataset gồm `meta/` (mô tả, câu lệnh, thống kê), `data/` (góc khớp, action theo từng episode) và `videos/` (camera đầu robot).

04 models/ — Model đã huấn luyện

Mỗi thư mục là một checkpoint dùng được ngay với GR00T server: trọng số (3 file `safetensors`), cấu hình, processor, thống kê chuẩn hoá, kèm log và script huấn luyện; không kèm trạng thái optimizer.

CHECKPOINT	HUẤN LUYỆN TRÊN
Búa → thùng, 200 episode `hammer_to_bin_mix200/` · bước 10.000	mix200 · 10.000 bước · 1× H100 · 61 phút
Búa → thùng, 400 episode `hammer_to_bin_mix400/` · bước 20.000	mix400 · 20.000 bước · 1× H100 · 2 giờ 15 phút
Real2sim 4 loại quả `real2sim_multi_fruit/` · bước 20.000	790 episode mô phỏng · 20.000 bước · batch 32
Táo → đĩa (của NVIDIA) không kèm trong gói	Không huấn luyện lại — tải từ Hugging Face vào `models/isaaclab_arena/static_apple_tutorial/gn1x_tuned_static_apple/` (đường dẫn mặc định trong cấu hình)

05 evals/ — Kết quả đánh giá

- `apple_plate/` — táo → đĩa, checkpoint NVIDIA, mỗi đợt 50 episode kèm 50 video camera đầu
  - `self_built_scene_50ep/` — khung cảnh tự dựng: 0/50
  - `galileo_scene_50ep/` — khung cảnh Galileo dựng lại: 28/50
- `multi_fruit/` — 4 loại quả, mỗi đợt 50 episode; video ghi cho 10 episode đầu
  - `finetune_real2sim_50ep/` — model fine-tune real2sim: 33/50
  - `baseline_apple_ckpt_50ep/` — đối chứng, dùng lại checkpoint táo: 0/50
- `hammer_to_bin/` — búa → thùng, mỗi đợt 100 episode kèm 100 video
  - `arena_mix200_100ep/` — mix200 trong Arena (camera ngoài + camera đầu): 56/100
  - `local_mix200_100ep/` — mix200, Isaac Sim local: 66/100
  - `local_mix400_100ep/` — mix400, Isaac Sim local: 63/100
  - `local_mix400_at_rest_0.02_100ep/` — mix400, nới ngưỡng đứng yên lên 0,02 m/s: 73/100

`README.md` ở thư mục gốc tóm tắt kết quả và nguồn gốc từng đợt. File chung ở mỗi đợt: `episodes.jsonl` (một dòng mỗi lần thử), `summary.json` / `summary.md` (tỉ lệ thành công, khoảng tin cậy, phân loại thất bại), video `…_success.mp4` / `…_fail.mp4`.

06 assets/ — Asset mô phỏng

`Đặt tại: ~/workspace/shimizu-robot/assets` `Mã nguồn đọc theo đường dẫn này`

- `nucleus_cache/` — bản sao offline các file NVIDIA mà Isaac Lab tải về khi dựng cảnh: nền Galileo, búa `hammer_2`, thùng đỏ, robot G1 và mô hình điều khiển thân dưới (WBC). Có sẵn thì chạy không cần mạng; thiếu thì Isaac Lab tự tải lại.
- `objects/objaverse/apple_01.usd`, `objects/hot3d/clay_plates.usd` — quả táo và cái đĩa của kịch bản táo → đĩa trong pipeline độc lập.

Kịch bản real2sim 4 loại quả không dùng thư mục này (URDF robot nằm trong `codes/g1_real2sim/assets/`).

4 Dữ liệu gốc bàn giao riêng

Dữ liệu HDF5 gốc của bài búa, chỉ cần khi muốn **chuyển đổi lại hoặc sinh thêm dữ liệu huấn luyện**; không cần để chạy model, đọc báo cáo hay xem kết quả.

THÀNH PHẦN	CẦN KHI
Dữ liệu teleop gốc bài búa (HDF5)	Chuyển đổi lại, sinh thêm demo
Bộ huấn luyện mix200 dạng HDF5	Chuyển LeRobot với cấu hình khác
Bộ huấn luyện mix400 dạng HDF5	như trên

Không kèm: môi trường Python đã cài (tạo lại bằng lệnh trong mã nguồn) và dữ liệu, checkpoint gốc của NVIDIA (tải từ Hugging Face).

5 Giấy phép và điều khoản sử dụng

THÀNH PHẦN	ĐIỀU KHOẢN
Model gốc GR00T N1.7-3B và các checkpoint fine-tune từ nó	Theo NVIDIA Open Model Agreement
Checkpoint táo → đĩa của NVIDIA (Arena)	Model card ghi "ready for non-commercial use only"
Dataset NVIDIA (táo → đĩa, dữ liệu G1 thật)	CC-BY-4.0: dùng được, cần ghi nguồn
Dữ liệu teleop bài búa, dataset sinh thêm, dataset real2sim	Do dự án tạo ra
Video minh hoạ mô hình VLA (mục II.01)	Của bên thứ ba
Asset 3D trong `assets/` (file NVIDIA Isaac / Arena, táo Objaverse, đĩa HOT3D)	Của NVIDIA và bên thứ ba; kiểm tra điều khoản trước khi phân phối lại

IV.02 Cài đặt môi trường

Chuẩn bị hai máy để chạy lại các kịch bản.

Các mục IV.02–IV.04 hướng dẫn tự chạy lại các kịch bản để kiểm chứng số liệu trong báo cáo. Cần hai máy: **máy trạm** chạy mô phỏng và notebook (client), **máy chủ GPU** của khách hàng chạy GR00T (server). Mục này cài đặt cả hai phía; khởi động server và chạy đánh giá được trình bày ở IV.03 và IV.04.

CLIENT

Isaac Sim 6.0.1 · Isaac Lab 3.0.0-beta2 · standalone_g1_gr00t

SERVER

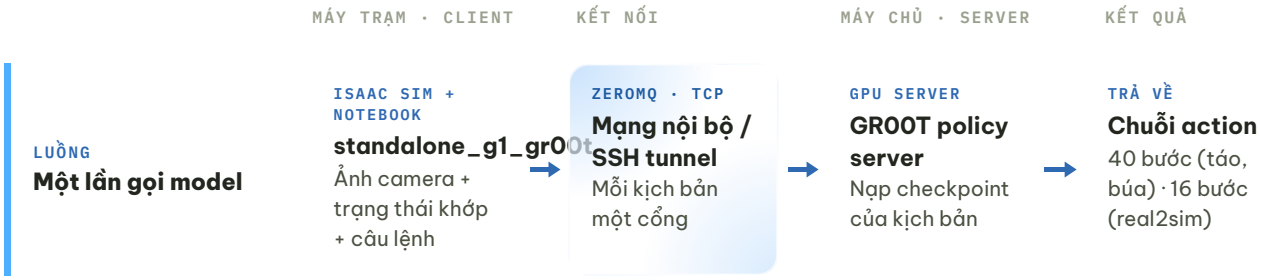
Máy chủ GPU của khách hàng · Isaac-GR00T 4b1dca9

KẾT NỐI

ZeroMQ qua mạng nội bộ hoặc SSH tunnel

1 Kiến trúc kết nối

Notebook trên máy trạm gửi ảnh camera, trạng thái khớp và câu lệnh tới GR00T server; server trả về một chuỗi action. Hai máy có thể nối thẳng qua mạng nội bộ, hoặc qua SSH tunnel nếu server chỉ nghe ở `127.0.0.1`.



2 Yêu cầu phần cứng và phần mềm

MÁY TRẠM · CLIENT

Mô phỏng và notebook

- Ubuntu 24.04, GPU NVIDIA có RTX (dự án dùng RTX 5090), driver 580
- Isaac Sim **6.0.1** cài cục bộ tại `~/isaac-sim-6` (Python 3.12)
- Isaac Lab **release/3.0.0-beta2** tại `~/IsaacLab`, cài vào cùng Python của Isaac Sim
- Isaac Lab Arena **release/0.2.1** tại `~/IsaacLab-Arena` (nguồn file YAML khớp và modality config)
- `uv`, `make`, `git`; truy cập Internet cho lần cài đầu

MÁY CHỦ · SERVER

GPU server của khách hàng

- Linux, GPU NVIDIA hỗ trợ CUDA (dự án dùng 1× H100 80 GB)
- Isaac-GR00T commit **4b1dca9**, môi trường `uv` (Python 3.10), `ffmpeg`
- Quyền truy cập Hugging Face tới model gốc `nvidia/GR00T-N1.7-3B` (đã chấp nhận giấy phép NVIDIA)
- Checkpoint trong `models/` và modality config của từng kịch bản (card 5)
- Mở được cổng cho máy trạm, hoặc cho phép SSH

Giữ đúng cặp phiên bản: toàn bộ kết quả trong báo cáo đo trên Isaac Sim 6.0.1 + Isaac Lab 3.0.0-beta2 và Isaac-GR00T 4b1dca9. Isaac Sim 6.1 cần driver ≥ 595 và chưa được kiểm thử với mã nguồn này.

3 Đặt gói sản phẩm trên máy trạm

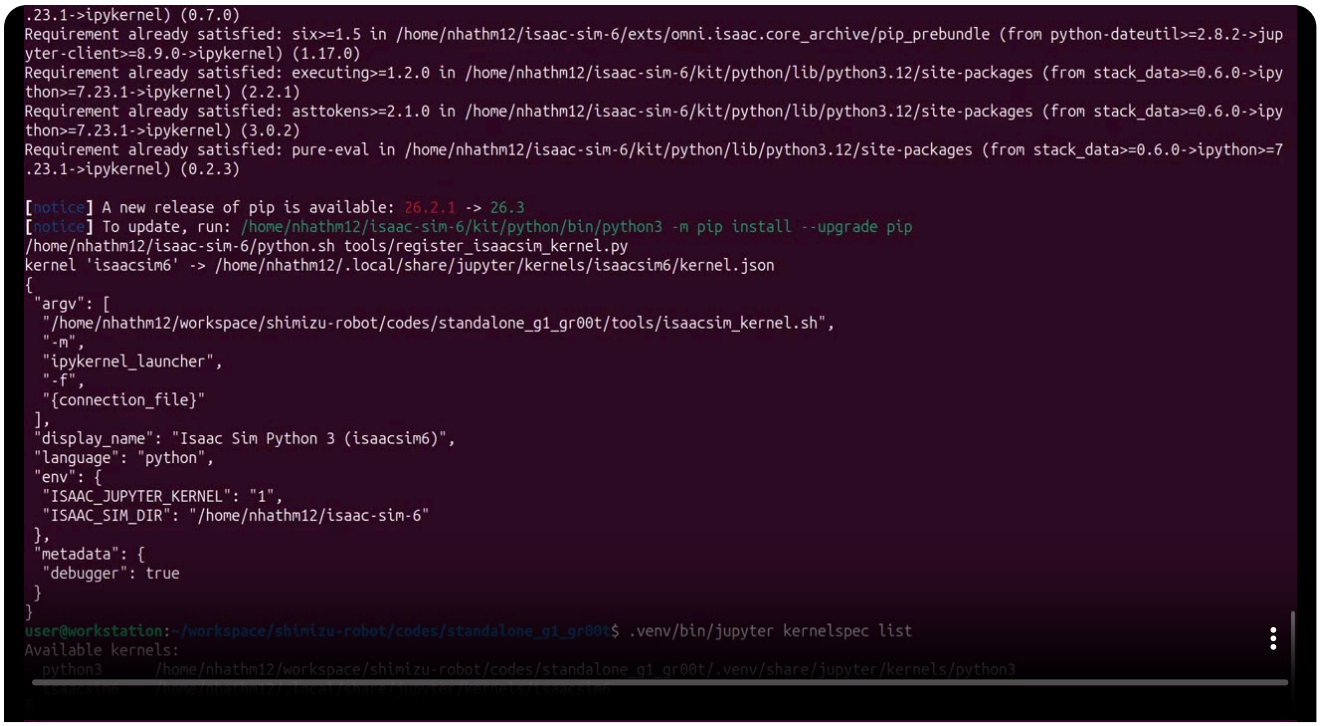
Mã nguồn đọc `assets/`, `datasets/` và `models/` theo đường dẫn `~/workspace/shimizu-robot/`, nên giải nén gói đúng vị trí này. Isaac Sim, Isaac Lab và Arena đặt trong thư mục home như card 2.

```
~/workspace/shimizu-robot/      # gói sản phẩm (IV.01)
├── codes/standalone_g1_gr00t/  # client: package g1gr00t, notebook, Makefile
├── assets/                     # nucleus_cache + táo, đĩa
├── datasets/                   # dataset LeRobot
└── models/                     # checkpoint (chép lên server, card 6)
~/isaac-sim-6/   ~/IsaacLab/   ~/IsaacLab-Arena/
```

Cài ở vị trí khác: Makefile cho phép ghi đè `ISAAC_PY` (Python của Isaac Sim), `ARENA_DIR` và `ISAAC_GR00T_DIR`, ví dụ `make sim-test ISAAC_PY=/opt/isaac-sim/python.sh`. Đường dẫn dataset và model nằm trong `configs/*.yaml`.

4 Cài client trên máy trạm

Package `standalone_g1_gr00t` dùng hai môi trường: **.venv** (Python 3.10, cho kiểm thử và notebook không mô phỏng) và **Python của Isaac Sim** (cho notebook có mô phỏng).



Video mẫu: chạy các lệnh kiểm tra client trên máy trạm đã cài Isaac Sim (đoạn `make sim-test-scene` được tua nhanh). Tên máy hiển thị là `user@workstation` và `user@gpu-server`.

```
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t

# môi trường .venv
make venv            # uv venv .venv --python 3.10, cài package + dev + notebook
make sync            # chép 4 file YAML khớp/WBC từ ~/IsaacLab-Arena, ghi hash nguồn
make assets          # tải policy chân AGILE (ONNX) vào assets/wbc/
make test            # unit test không cần mô phỏng

# môi trường Isaac Sim
export PYTHONNOUSERSITE=1   # không để gói trong ~/.local chen vào Python của Isaac Sim
make sim-install     # cài package vào ~/isaac-sim-6/python.sh
make sim-test        # unit test bằng Python của Isaac Sim
make sim-test-scene  # test có scene: robot đứng vững, bàn / táo / đĩa
make sim-kernel      # đăng ký kernel Jupyter "isaacsim6" cho notebook mô phỏng
```

DỮ LIỆU TRÊN MÁY TRẠM	ĐƯỜNG DẪN	DÙNG CHO
Policy chân AGILE	`codes/standalone_g1_gr00t/assets/wbc/` (tải bằng `make assets`)	Mọi notebook mô phỏng
Asset cảnh Galileo, búa, thùng, robot G1	`assets/nucleus_cache/` (có sẵn trong gói)	Kịch bản táo (cảnh Galileo) và búa; chạy không cần mạng

DỮ LIỆU TRÊN MÁY TRẠM	ĐƯỜNG DẪN	DÙNG CHO
Táo, đĩa	`assets/objects/`	Kịch bản táo (cảnh tự dựng)
Dataset búa → thùng mix200	`datasets/hammer_to_bin/static_hammer_mix200/`	nb12: phát lại demo, khớp màu ảnh
Dataset táo → đĩa của NVIDIA	`datasets/isaaclab_arena/static_apple_tutorial/` (tải từ Hugging Face)	Phát lại demo, open-loop kịch bản táo

> **Kiểm tra xong khi:** `make test`, `make sim-test` và `make sim-test-scene` đều không có test lỗi, và Jupyter thấy kernel **Isaac Sim Python 3 (isaacsim6)**.

5 Cài server trên máy chủ GPU

Cài Isaac-GR00T đúng commit đã dùng khi huấn luyện. Bảng dưới cho biết mỗi kịch bản cần checkpoint và modality config nào; cách đưa chúng lên server ở card 6.



Video mẫu: clone Isaac-GR00T đúng commit, tạo môi trường `uv` và kiểm tra GPU trên máy chủ (các đoạn tải được tua nhanh). Tên máy hiển thị là `user@workstation` và `user@gpu-server`.

```
# cần sẵn: git, uv, ffmpeg (sudo apt-get install -y ffmpeg)
git clone https://github.com/NVIDIA/Isaac-GR00T.git ~/Isaac-GR00T
cd ~/Isaac-GR00T
git checkout 4b1dca9d88d2a0b9ea5a65aa61c82ff89f5c4f0e
git submodule update --init --recursive
uv sync --python 3.10      # tạo môi trường, cài GR00T và thư viện GPU (vài phút)
```

```
uv run python -c "import gr00t, torch; print('GR00T ok | CUDA', torch.cuda.is_available())"
# nếu model gốc chưa có trong cache Hugging Face, đăng nhập bằng tài khoản
# đã chấp nhận giấy phép nvidia/GR00T-N1.7-3B:
hf auth login
```

KỊCH BẢN · CỔNG CLIENT	CHECKPOINT TRONG `MODELS/`	MODALITY CONFIG
Táo → đĩa 5556 · `g1_static_apple.yaml`	Checkpoint NVIDIA tải từ Hugging Face	Bản của Arena

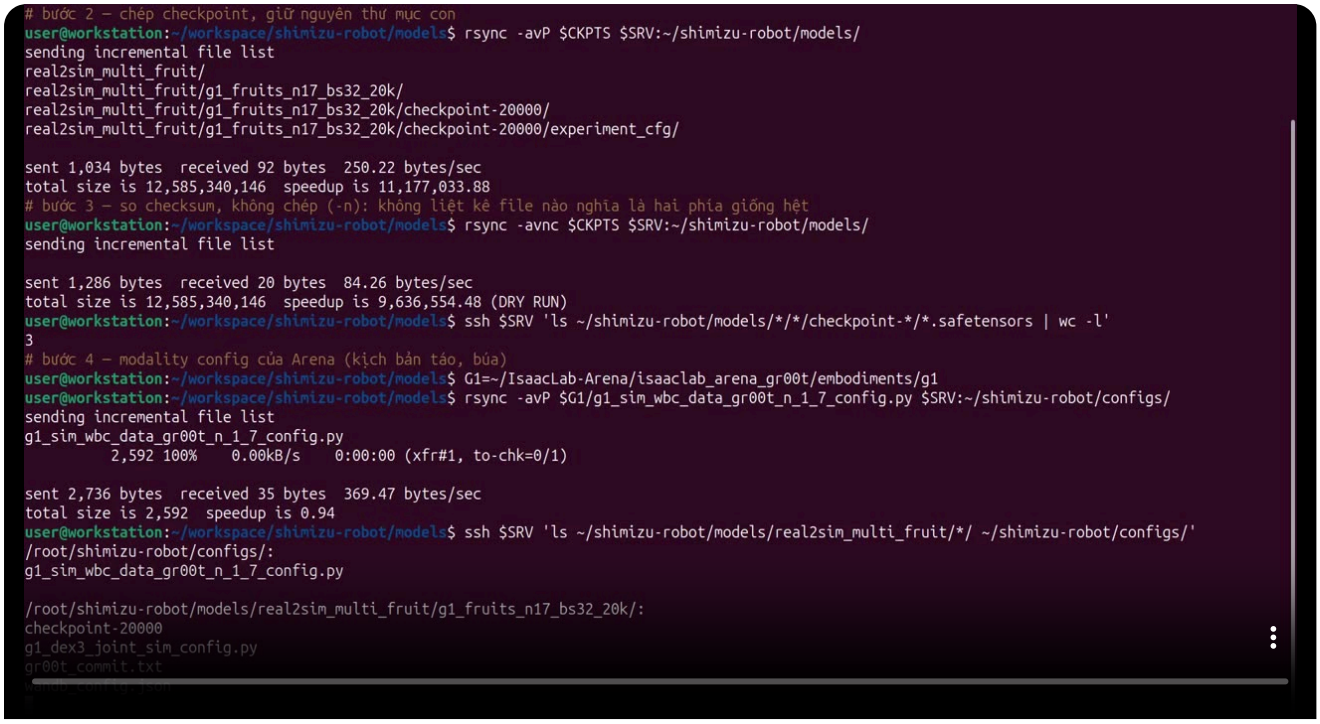
Búa → thùng
5557 · `NB12_PORT`
	`hammer_to_bin_mix200/`
	`hammer_mix200_n17/checkpoint-10000`
	`hammer_to_bin_mix400/`
	`hammer_mix400_n17/checkpoint-20000`
		Bản của Arena

Real2sim 4 loại quả 5556 · `NB10_PORT`	`real2sim_multi_fruit/` `g1_fruits_n17_bs32_20k/checkpoint-20000`	Bản real2sim

Modality config phải khớp lúc huấn luyện: dùng sai file thì server vẫn chạy nhưng model nhận sai thứ tự khớp. Bản của Arena là `isaaclab_arena_gr00t/embodiments/g1/g1_sim_wbc_data_gr00t_n_1_7_config.py` trong Arena **release/0.2.1** (chép riêng file đó lên server hoặc clone Arena). Bản real2sim là `g1_dex3_joint_sim_config.py`, nằm trong thư mục `real2sim_multi_fruit/g1_fruits_n17_bs32_20k/` (cạnh `checkpoint-20000`) và trong `codes/g1_real2sim/groot/`. So `md5sum` hai phía trước khi chạy.

6 Đưa checkpoint bàn giao lên server

Ba checkpoint fine-tune nằm trong thư mục `models/` của gói bàn giao, mỗi thư mục dùng được ngay với GR00T server: trọng số (3 file `safetensors`), cấu hình, processor và thống kê chuẩn hoá. Chỉ cần chép những checkpoint của kịch bản muốn kiểm chứng.



Video mẫu với checkpoint real2sim. Server trong video đã có sẵn bản sao giống hệt nên rsync chỉ đối chiếu, không chép lại 12 GB; lần đầu chép sẽ lâu hơn tuỳ tốc độ mạng. Tên máy hiển thị là `user@workstation` và `user@gpu-server`.

BƯỚC 1
Tạo thư mục trên server
Ví dụ `~/shimizu-robot/models`

→

BƯỚC 2
Chép checkpoint
`rsync` từ máy trạm, chạy tiếp được nếu bị ngắt

→

BƯỚC 3
Kiểm tra toàn vẹn
So checksum hai phía, đủ 3 file trọng số

→

BƯỚC 4
Chép modality config
Bản của Arena cho kịch bản táo và búa

```
# chạy trên máy trạm
SRV=user@gpu-server
CKPTS="hammer_to_bin_mix200 hammer_to_bin_mix400 real2sim_multi_fruit"
cd ~/workspace/shimizu-robot/models

# bước 1 — tạo thư mục trên server
ssh $SRV 'mkdir -p ~/shimizu-robot/models ~/shimizu-robot/configs'

# bước 2 — chép checkpoint cần dùng, giữ nguyên thư mục con
rsync -avP $CKPTS $SRV:~/shimizu-robot/models/

# bước 3 — so checksum, không chép (-n):
# không liệt kê file nào nghĩa là hai phía giống hệt
rsync -avnc $CKPTS $SRV:~/shimizu-robot/models/
# đếm file trọng số: kỳ vọng 9 (3 checkpoint × 3 file)
ssh $SRV 'ls ~/shimizu-robot/models/*/*/checkpoint-*/*.safetensors | wc -l'

# bước 4 — modality config của Arena (kịch bản táo, búa)
```

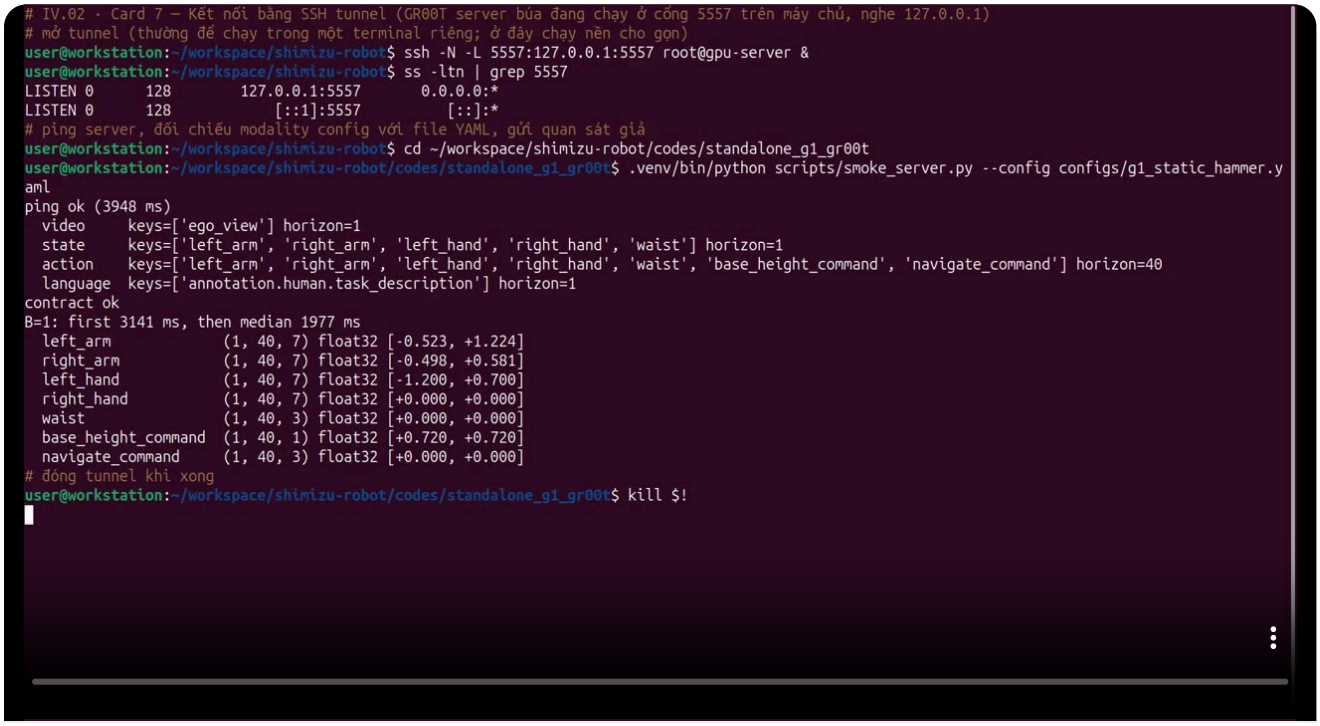
```
G1=~/IsaacLab-Arena/isaaclab_arena_gr00t/embodiments/g1
rsync -avP $G1/g1_sim_wbc_data_gr00t_n_1_7_config.py \
      $SRV:~/shimizu-robot/configs/
```

THƯ MỤC TRONG `MODELS/`	ĐƯỜNG DẪN `--MODEL-PATH` TRÊN SERVER
`hammer_to_bin_mix200/`	`~/shimizu-robot/models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000`
`hammer_to_bin_mix400/`	`~/shimizu-robot/models/hammer_to_bin_mix400/hammer_mix400_n17/checkpoint-20000`
`real2sim_multi_fruit/`	`~/shimizu-robot/models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/checkpoint-20000`

Không qua mạng được? Chép thư mục `models/` bằng ổ cứng ngoài rồi đặt trên server theo cùng cấu trúc; vẫn chạy bước 3 bằng `sha256sum` trên các file `safetensors` ở hai phía. Checkpoint táo → đĩa của NVIDIA không có trong gói: tải thẳng từ Hugging Face về server.

7 Thiết lập kết nối

Chọn một trong hai cách. Cổng phía server phải trùng cổng client mong đợi (card 5); có thể đổi cổng client bằng biến môi trường `NB10_PORT`, `NB12_PORT` hoặc trường `server.port` trong file YAML.



Video mẫu: mở SSH tunnel tới server búa (cổng 5557), kiểm tra cổng và ping server bằng `smoke_server.py`. Tên máy hiển thị là `user@workstation` và `user@gpu-server`.

CÁCH 1

Mạng nội bộ

- Server nghe `--host 0.0.0.0`
- Mở cổng của kịch bản trên firewall cho riêng máy trạm
- Client đặt host là IP server, ví dụ `NB12_HOST=10.0.0.5`

CÁCH 2

SSH tunnel

- Server nghe `--host 127.0.0.1`, không mở cổng ra ngoài
- Máy trạm mở tunnel và để chạy suốt lúc đánh giá
- Client giữ host mặc định `127.0.0.1`

```
# cách 2, trên máy trạm (ví dụ kịch bản búa, cổng 5557)
ssh -N -L 5557:127.0.0.1:5557 user@gpu-server
```

IV.03 Chạy server GR00T

Mỗi kịch bản một checkpoint, một cổng.

Server GR00T chạy trên máy chủ GPU của khách hàng, nạp checkpoint của kịch bản cần kiểm chứng và nghe ở một cổng. Máy trạm kết nối tới đúng cổng đó (qua mạng nội bộ hoặc SSH tunnel, mục IV.02 card 7) rồi mới chạy notebook ở mục IV.04.

LỆNH
gr00t/eval/run_gr00t_server.py

CỔNG
5556 (táo, real2sim) · 5557 (búa)

KIỂM TRA
Cổng đang nghe · ping từ máy trạm

1 Cấu hình server theo kịch bản

Đường dẫn dưới đây theo cách đặt ở mục IV.02 card 6 (gốc `~/shimizu-robot/` trên server). Cổng là giá trị client mặc định mong đợi.

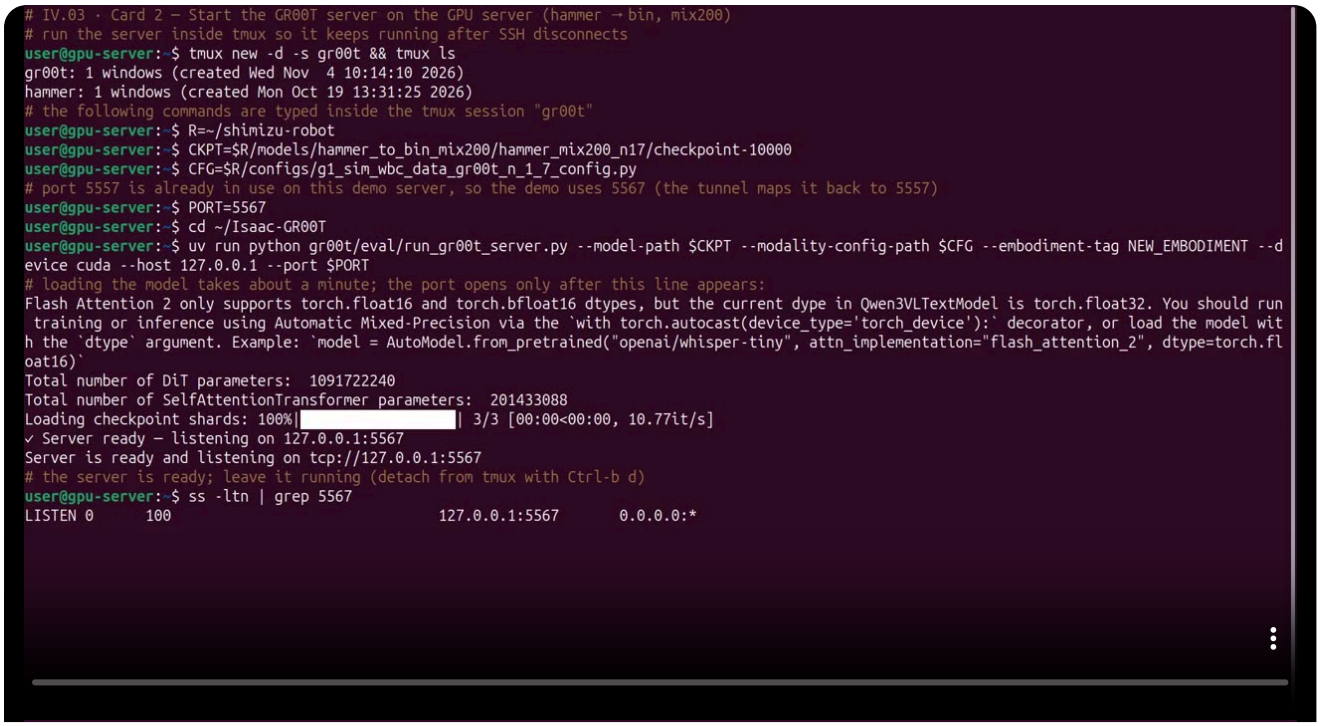
KỊCH BẢN · CỔNG	CHECKPOINT (`--MODEL-PATH`)	MODALITY CONFIG
Táo → đĩa 5556	`models/gn1x_tuned_static_apple` (tải từ Hugging Face)	`configs/g1_sim_wbc_data_gr00t_n_1_7_config.py`
Búa → thùng, mix200 5557	`models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000`	như trên
Búa → thùng, mix400 5557	`models/hammer_to_bin_mix400/hammer_mix400_n17/checkpoint-20000`	như trên
Real2sim 4 loại quả 5556	`models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/checkpoint-20000`	`models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/g1_dex3_joint_sim_config.py`

> Táo và real2sim dùng chung cổng 5556 nên mỗi lúc chỉ chạy một trong hai. Búa dùng cổng 5557, chạy song song được nếu GPU đủ bộ nhớ.

```
# tải checkpoint táo → đĩa của NVIDIA về server (một lần)
hf download nvidia/GN1x-Tuned-Arena-G1-Static-PickNPlace \
   --local-dir ~/shimizu-robot/models/gn1x_tuned_static_apple
```

2 Khởi động server

Chạy trong `tmux` để server không tắt khi ngắt SSH. Ví dụ dưới là kịch bản búa mix200; với kịch bản khác chỉ đổi ba biến đầu theo card 1.



Video mẫu: chạy server búa mix200 trong tmux trên máy chủ, chờ dòng `✓ Server ready` (đoạn nạp model được tua nhanh). Máy chủ trong video đã dùng cổng 5557 cho việc khác nên server mẫu nghe cổng 5567; tunnel ở card 3 chuyển về cổng 5557 trên máy trạm.

```
tmux new -s gr00t                  # thoát tạm: Ctrl-b d · vào lại: tmux a -t gr00t
R=~/shimizu-robot
CKPT=$R/models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000
CFG=$R/configs/g1_sim_wbc_data_gr00t_n_1_7_config.py
PORT=5557

cd ~/Isaac-GR00T
uv run python gr00t/eval/run_gr00t_server.py \
  --model-path $CKPT --modality-config-path $CFG \
  --embodiment-tag NEW_EMBODIMENT --device cuda \
  --host 127.0.0.1 --port $PORT    # 0.0.0.0 nếu máy trạm nối thẳng qua mạng nội bộ
```

NẠP MODEL
Khoảng 1 phút
Chưa mở cổng; client gọi lúc này sẽ hết giờ

→

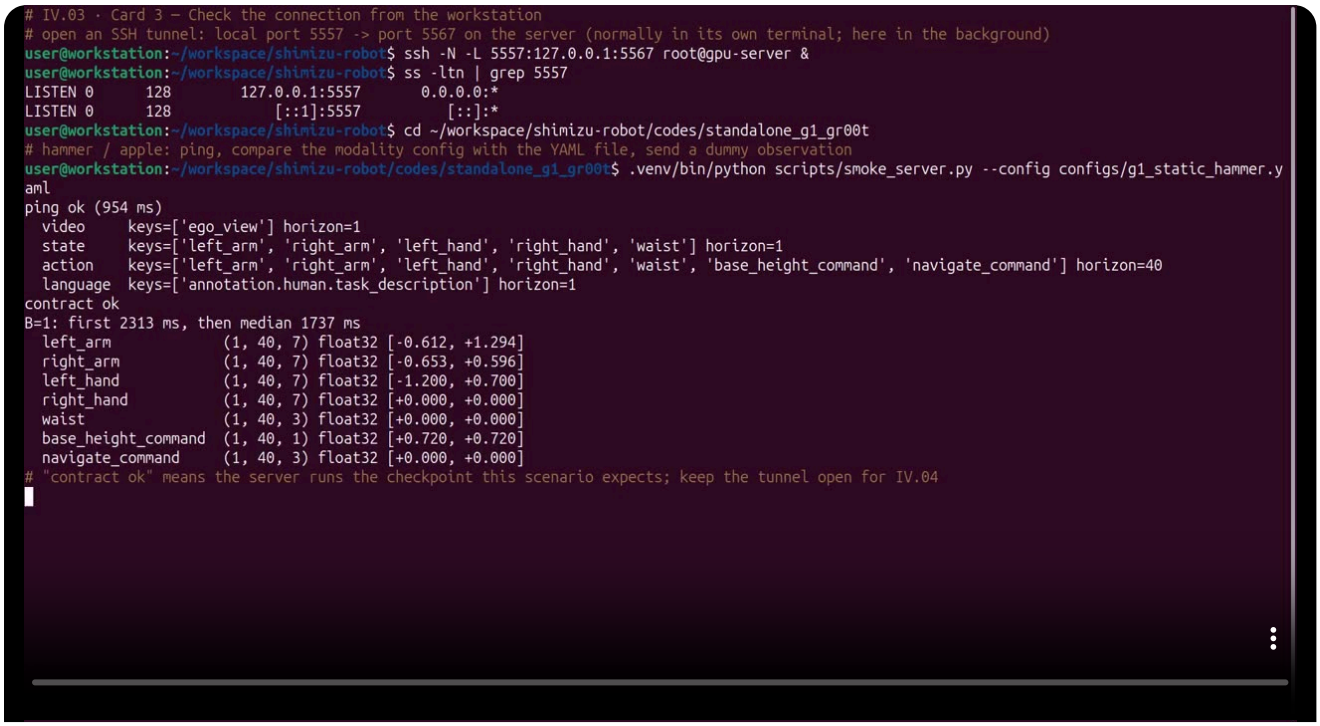
SẴN SÀNG
In dòng báo sẵn sàng
`✓ Server ready — listening on …`

→

ĐỔI KỊCH BẢN
Ctrl-C rồi chạy lại
Với checkpoint, config, cổng mới

3 Kiểm tra kết nối từ máy trạm

Nếu dùng SSH tunnel, mở tunnel trong một terminal riêng và để chạy suốt lúc đánh giá. Sau đó ping server bằng Python của môi trường `.venv`.



Video mẫu: mở SSH tunnel, kiểm tra cổng 5557 trên máy trạm và chạy `smoke_server.py` với cấu hình búa (kết quả `contract ok`).

```
# (nếu dùng SSH tunnel) terminal riêng, để chạy suốt
ssh -N -L 5557:127.0.0.1:5557 user@gpu-server

# ping server và hỏi modality config đang nạp
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t
# táo, búa: ping, đối chiếu modality config với file YAML, gửi quan sát giả
.venv/bin/python scripts/smoke_server.py --config configs/g1_static_hammer.yaml
#   (táo: configs/g1_static_apple.yaml)
# real2sim: ping và in modality config server đang nạp
.venv/bin/python -c "from g1gr00t.client import Gr00tZmqClient, modality_summary as m
c = Gr00tZmqClient('127.0.0.1', 5556); print(c.ping(), m(c.get_modality_config()))"
```

TRIỆU CHỨNG	NGUYÊN NHÂN	CÁCH XỬ LÝ
Hết giờ ngay lần gọi đầu	Tunnel chưa mở, sai cổng, hoặc server còn đang nạp model	`ss -ltn | grep 555` ở cả hai phía; chờ dòng báo sẵn sàng
Mất kết nối giữa chừng	Tunnel hoặc mạng bị ngắt	Mở lại tunnel rồi chạy lại notebook; nb10 và nb12 chạy tiếp từ episode còn dở
Notebook báo sai video key hoặc số khớp	Server đang nạp checkpoint của kịch bản khác	Khởi động lại server với đúng checkpoint và modality config ở card 1
Mỗi lần gọi > 3 s	GPU bận hoặc mạng chậm	Không ảnh hưởng kết quả (mô phỏng dừng chờ model), chỉ chạy lâu hơn

IV.04 Chạy kiểm chứng

Chạy lại notebook và so với số liệu báo cáo.

Mọi kịch bản được đánh giá bằng notebook trong `codes/standalone_g1_gr00t`, chạy trực tiếp trên Isaac Sim cục bộ, **không cần Docker Arena**. Mỗi notebook có lệnh `make` để chạy không giao diện (headless) và ghi kết quả vào `data/`.

THƯ MỤC
codes/standalone_g1_gr00t

CHẠY
make sim-nbXX (headless) · Jupyter (tương tác)

ĐẦU RA
data/: log từng episode, tổng kết, video

1 Quy trình chung

01 · SERVER
Nạp đúng checkpoint
Mục IV.03, card 1–2

→

02 · KẾT NỐI
Mở tunnel nếu cần
Để chạy suốt lúc đánh giá

→

03 · KIỂM TRA
Ping server
Mục IV.03, card 3

→

04 · CHẠY
Lệnh make của kịch bản
Card 3 bên dưới

→

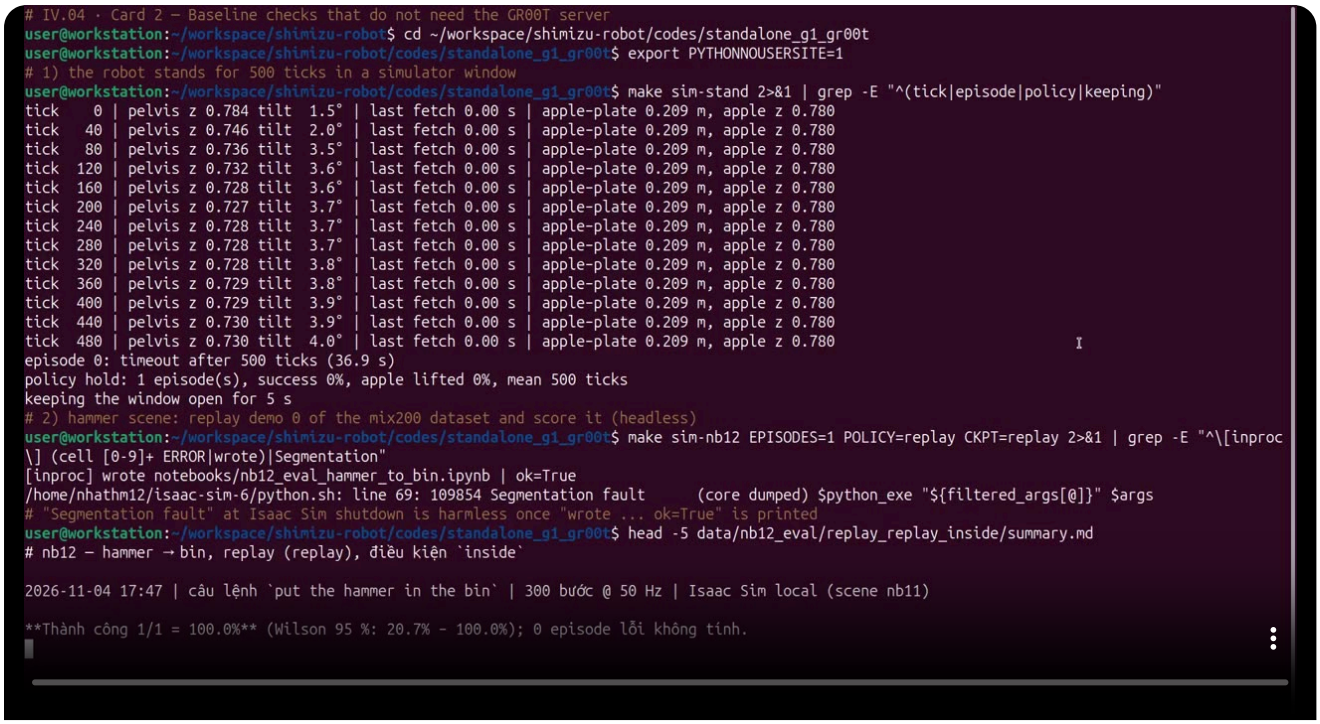
05 · SO SÁNH
Đọc tổng kết
So với khoảng tin cậy (card 4)

Kết quả có sẵn trong gói sẽ bị ghi đè: lệnh `make sim-nbXX` ghi output mới vào chính file `.ipynb`; nb07 còn ghi nối vào `data/runs_galileo.jsonl` và ghi đè video `data/videos/gr00t_galileo/epN.mp4`. Sao lưu `notebooks/` và `data/` trước khi chạy, và đổi tên `runs_galileo.jsonl` để tổng kết chỉ gồm lần chạy mới. Bản gốc của các kết quả cũng có trong `evals/`.

Lỗi lúc tắt Isaac Sim: cuối mỗi lần chạy có thể thấy `Segmentation fault` và `make: *** Error 1`. Nếu trước đó đã có dòng `[inproc] wrote notebooks/… | ok=True` thì notebook đã chạy xong và kết quả đã được ghi đủ.

2 Kiểm tra nền không cần server

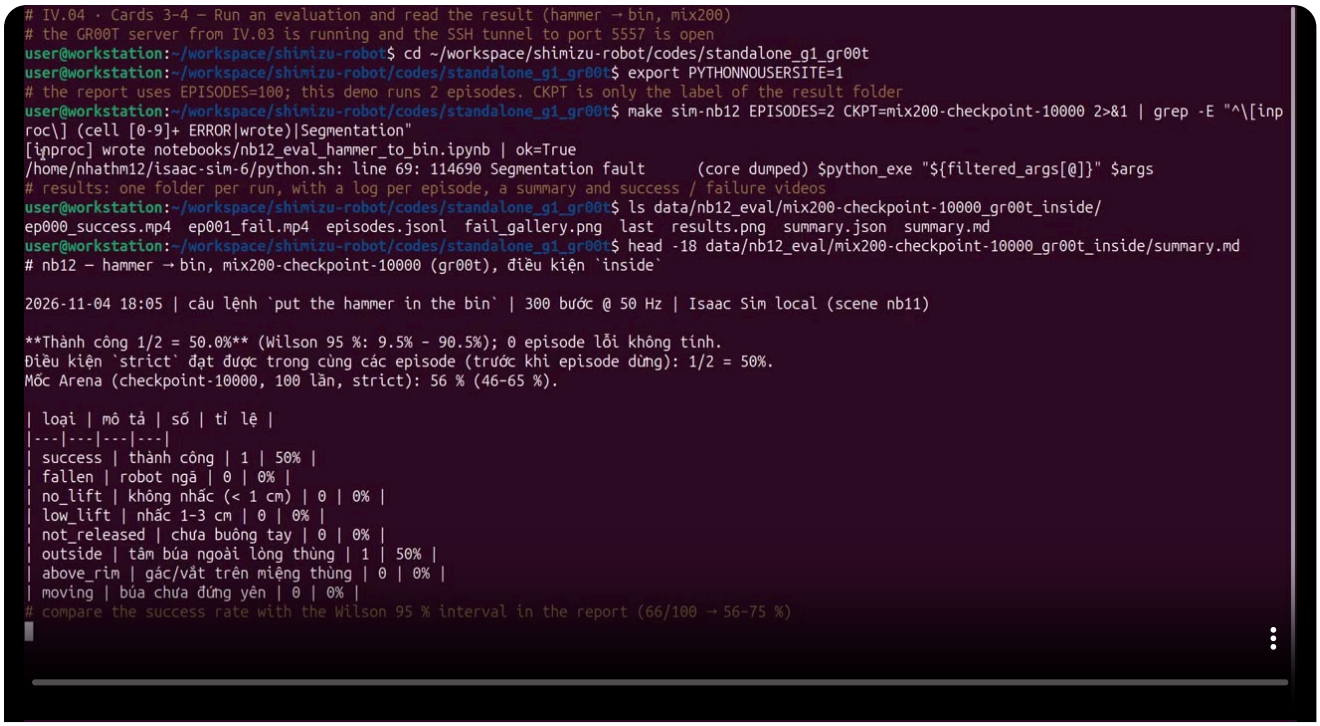
Xác nhận mô phỏng, robot, cảnh và cách chấm điểm chạy đúng trước khi tốn thời gian chạy model.



Video mẫu: `make sim-stand` mở cửa sổ mô phỏng, robot đứng 500 tick; sau đó phát lại demo 0 của dataset búa (headless) và đọc tổng kết 1/1. Các đoạn chạy mô phỏng được tua nhanh.

LỆNH	LÀM GÌ	ĐẠT KHI
`make sim-stand`	Robot đứng 500 tick, có cửa sổ	Robot đứng vững, không ngã
`make sim-replay`	Phát lại demo episode 0 của dataset táo, có cửa sổ	Robot nhặt táo đặt lên đĩa
`make sim-nb12 EPISODES=1 POLICY=replay CKPT=replay`	Cảnh búa → thùng, phát lại demo 0 của dataset mix200	Tổng kết báo 1/1 thành công

3 Lệnh kiểm chứng từng kịch bản



Video mẫu: chạy nb12 (búa → thùng, mix200) với server ở mục IV.03, rồi xem thư mục kết quả và `summary.md` như card 4. Video chỉ chạy 2 episode cho ngắn (kết quả 1/2); số liệu trong báo cáo dùng 100 episode. Đoạn mô phỏng được tua nhanh.

```
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t

# Táo → đĩa, cảnh Galileo (server táo, cổng 5556)
make sim-nb07 EPISODES=50

# Real2sim 4 loại quả (server real2sim, cổng 5556)
make sim-nb10 EPISODES=50

# Búa → thùng (server búa, cổng 5557); CKPT chỉ là nhãn tên thư mục kết quả
make sim-nb12 EPISODES=100 CKPT=mix200-checkpoint-10000
make sim-nb12 EPISODES=100 CKPT=mix400-checkpoint-20000
```

KỊCH BẢN	NOTEBOOK · THỜI GIAN	KẾT QUẢ TRONG BÁO CÁO
Táo → đĩa, Galileo	`nb07_full_pipeline_loop` ~45 s / episode	**56 %** · 28/50 · Wilson 42–69 %
Real2sim 4 loại quả	`nb10_eval_multi_object` ~1–2 phút / episode	**66 %** · 33/50 · Wilson 52–78 %
Búa → thùng, mix200	`nb12_eval_hammer_to_bin` ~15–30 s / episode	**66 %** · 66/100 · Wilson 56–75 %
Búa → thùng, mix400	như trên	**73 %** · 73/100 · Wilson 64–81 % 63 % · 63/100 với ngưỡng đứng yên của Arena

Ngưỡng đứng yên của búa: nb12 hiện coi búa đã nằm yên khi vận tốc < 0,02 m/s và vận tốc góc < 0,12 rad/s (biến `LIN_VEL_TH`, `ANG_VEL_TH` ở §3). Kết quả 73 % đo với ngưỡng này. Muốn tái lập 63 % (ngưỡng gốc của Arena, 0,01 m/s và 0,1 rad/s), sửa hai biến đó trong notebook rồi chạy lại với nhãn `CKPT` khác.

Ngoài phạm vi các lệnh trên: 92 % của kịch bản táo và 56/100 của búa trong Arena đo bằng client Arena gốc trong Docker, không dùng notebook standalone.

4 Đọc và so sánh kết quả

NOTEBOOK	THƯ MỤC KẾT QUẢ	TỈ LỆ THÀNH CÔNG NẰM Ở
nb07	`data/runs_galileo.jsonl`, video trong `data/videos/gr00t_galileo/`	Bảng tổng kết cuối notebook
nb10	`data/nb10_eval/` (đổi bằng `NB10_OUT`)	`seed0_….summary.json`
nb12	`data/nb12_eval/<CKPT>_gr00t_inside/`	`summary.json`, `summary.md`

KHỚP VỚI BÁO CÁO

Nằm trong khoảng tin cậy

Mỗi episode được ngẫu nhiên hoá; với 50 episode, chênh khoảng ±13 điểm phần trăm vẫn bình thường, với 100 episode khoảng ±9 điểm.

LỖI KỸ THUẬT

Không tính vào tỉ lệ

Episode lỗi (server hết giờ, mất kết nối) được ghi lý do và loại khỏi tỉ lệ thành công; notebook dừng nếu server không trả lời lại.

CHẠY TIẾP

Không phải chạy lại từ đầu

nb10 và nb12 bỏ qua các episode đã có trong thư mục kết quả; bật lại server rồi chạy lại đúng lệnh cũ.

5 Chạy tương tác trong Jupyter

Muốn xem từng bước hoặc cửa sổ mô phỏng, mở notebook trong Jupyter thay vì chạy headless.

```
make sim-kernel && make nb     # Jupyter Lab, chọn kernel "Isaac Sim Python 3 (isaacsim6)"
```

GẶP	XỬ LÝ
`No module named 'isaaclab'`	Đang ở kernel `.venv`: đổi sang kernel `isaacsim6`
`print` không hiện trong cell	Isaac Lab gán lại `sys.stdout`; xem output ở terminal chạy Jupyter
Treo khi đóng mô phỏng	Không gọi `app.close()`; dùng Kernel ▸ Restart

IV.05 Đánh giá video bằng VSS

Để mô hình thị giác–ngôn ngữ xem video và tự chấm kết quả.

NVIDIA VSS (Video Search & Summarization) là bộ công cụ phân tích video bằng mô hình thị giác–ngôn ngữ (VLM). Mục này hướng dẫn cài VSS 3.2.1 trên máy chủ GPU, rồi dùng **cầu nối Isaac Sim → VSS** cho VSS xem video episode búa → thùng và kết luận thành công hay thất bại, so với kết luận của Arena. Mỗi bước có lệnh, kết quả mong đợi và video quay lại thao tác thật.

PHIÊN BẢN
VSS 3.2.1 · profile LVS · 1× H100

3 CÁCH CHẤM
Tóm tắt video · Hỏi đáp · Cảnh báo thời gian thực

KẾT LUẬN
Cảnh báo thời gian thực, `top_k: 1` : 94,2 % (cần xác nhận)

1 VSS làm gì trong dự án

Arena chấm episode bằng trạng thái mô phỏng (vị trí búa, thùng). Với robot thật thì không có trạng thái đó, chỉ có video. VSS được thử để trả lời câu hỏi: **chỉ nhìn video, VLM có kết luận đúng như Arena không?**

01 · EPISODE
Video + kết luận Arena
Kịch bản búa → thùng (mục III.03)

→

02 · XUẤT VIDEO
15 fps, 480p
Giữ khung cuối 1,5 s để VLM thấy trạng thái cuối

→

03 · VSS
VLM xem video
Cosmos3 nano reasoner chạy trên GPU; LLM Nemotron gọi qua build.nvidia.com

→

04 · KẾT LUẬN
Thành công / thất bại
Từ sự kiện, câu trả lời YES/NO hoặc cảnh báo

→

05 · SO SÁNH
Với kết luận Arena
Card 5

Container
26
17 chạy thường trực · 9 chỉ chạy lúc khởi tạo

GPU cho VLM
~44 GB
model Cosmos3 16,3 GB · vLLM giữ tối đa 0,5 bộ nhớ GPU

Đĩa
~80 GB
ảnh Docker + model; lớn nhất 42,9 GB

Cài đặt
~12 phút
cài và nạp model lần đầu

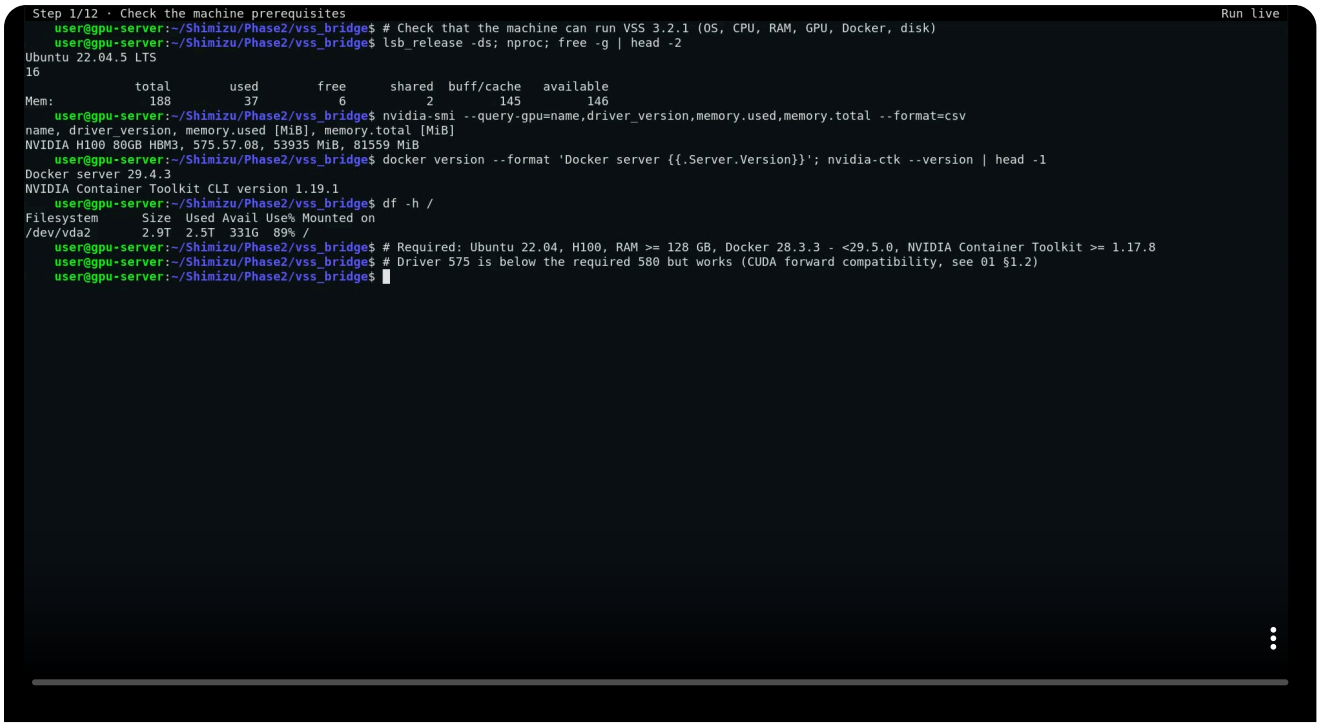
Tài liệu và video đi kèm: thư mục `vss_guide/` gồm hướng dẫn từng bước `07_step_by_step_guide.md` và 12 video, mỗi video một bước, mở đầu bằng thẻ tiêu đề 5 giây. Video có hai chế độ: **Chạy thật** (lệnh chạy ngay lúc quay) và **Chỉ kiểm tra** (bước đã làm lúc cài, chạy lại sẽ gây hại, nên video chỉ kiểm tra kết quả). Trong video ở báo cáo, tên máy chủ và địa chỉ IP đã được thay bằng `user@gpu-server` và `<server-ip>`.

2 Yêu cầu máy chủ

So máy chủ với yêu cầu của VSS 3.2.1 trước khi cài. Cột giữa là máy dùng khi quay video.

YÊU CẦU	MÁY LÚC QUAY	GHI CHÚ
Ubuntu 22.04 hoặc 24.04	Ubuntu 22.04.5 LTS	Đạt
CPU ≥ 18 nhân	16 nhân	Thiếu một chút, vẫn chạy được
RAM ≥ 128 GB	188 GB	Đạt
GPU H100	H100 80 GB HBM3	Dùng chung GPU: VSS ~44 GB, phần còn lại cho GR00T và việc khác
Driver 580.65.06	575.57.08	Không nâng cấp; chạy được nhờ CUDA forward compatibility
Docker 28.3.3 đến dưới 29.5.0	29.4.3	Đạt
NVIDIA Container Toolkit ≥ 1.17.8	1.19.1	Đạt
Đĩa trống	~331 GB	Bản cài chiếm ~80 GB; Elasticsearch cần hơn 15 GB trống

```
lsb_release -ds; nproc; free -g | head -2
nvidia-smi --query-gpu=name,driver_version,memory.used,memory.total --format=csv
docker version --format 'Docker server {{.Server.Version}}'; nvidia-ctk --version | head -1
df -h /
```



Bước 1 · Chạy thật: kiểm tra hệ điều hành, CPU, RAM, GPU, driver, Docker, Container Toolkit và đĩa trống.

3 Cài đặt VSS 3.2.1

Năm bước, theo đúng thứ tự. Bước 3 và 6 cần quyền root và làm gián đoạn máy chủ dùng chung, nên quản trị viên làm vào lúc đã báo trước.

02 · KHOÁ
NGC Personal Key
Đăng nhập nvcr.io →

03 · DOCKER
Hạ về 29.4.3
Quản trị viên · khoá phiên bản →

04 · MÃ NGUỒN
Clone v3.2.1
Áp 4 bản vá →

05 · CHẠY THỬ
Dry-run
Chỉ trong bản clone tạm →

06 · CÀI
Tải ảnh, khởi động
26 container, model 16,3 GB

```
# bước 2: khoá để riêng một file, quyền 600; không dán vào chat, không commit
mkdir -p ~/.config/vss && umask 077
cat > ~/.config/vss/keys.env <<'EOF'
export NGC_CLI_API_KEY='nvapi-...'
export NVIDIA_API_KEY='nvapi-...'
EOF
bash -c 'source ~/.config/vss/keys.env; echo "$NGC_CLI_API_KEY" | docker login nvcr.io -u "\$oauthtoken'

# bước 3 (quản trị viên): Docker trong khoảng NVIDIA đã kiểm định
VER=5:29.4.3-1~ubuntu.22.04~jammy
sudo apt-get install -y --allow-downgrades docker-ce=$VER docker-ce-cli=$VER
sudo apt-mark hold docker-ce docker-ce-cli

# bước 4: đúng bản 3.2.1, rồi áp các bản vá
git clone --branch v3.2.1 https://github.com/NVIDIA-AI-Blueprints/video-search-and-summarization.git vss
```

```
cd vss && git apply /path/to/local-patches.diff

# bước 6: cài (LLM gọi từ xa, VLM chạy trên GPU của máy chủ)
set -a && source ~/.config/vss/keys.env && set +a
export LLM_ENDPOINT_URL=https://integrate.api.nvidia.com RTVI_VLLM_GPU_MEMORY_UTILIZATION=0.5
bash deploy/docker/scripts/dev-profile.sh up -p lvs -H H100 -i <server-ip> \
  --use-remote-llm --llm nvidia/nemotron-3.5-lightning-30b-a3b
```

#	FILE	BẢN VÁ (LOCAL-PATCHES.DIFF)
1	`dev-profile.sh` , `state_down`	Chỉ xoá volume mồ côi có tiền tố `mdx_` , không bao giờ xoá volume của người dùng khác
2	`dev-profile.sh` , `state_up`	Chỉ đổi cấu hình kernel (tắt IPv6 toàn máy) khi đặt `VSS_APPLY_KERNEL_SETTINGS=true`
3	`dev-profile.sh` , `state_up`	Giữ giá trị `RTVI_VLLM_GPU_MEMORY_UTILIZATION` đặt sẵn (0,5 thay cho 0,7) để chừa GPU cho GR00T
4	`vss-agent/configs/config.yml`	Agent gọi LLM qua client `openai` , sửa lỗi 400 `verify_ssl`

> `dev-profile.sh up --dry-run` **không vô hại:** nó vẫn ghi đè `generated.env` (file chứa khoá và cấu hình LLM) và chạy `chmod -R 777` trong `data-dir` . Chỉ dry-run trong một bản clone tạm (video bước 5). Ngoài ra `dev-profile.sh up` luôn chạy `down -v` trước, tức là **xoá volume model 17,8 GB**; trên máy đã cài, không chạy lại lệnh này.

4 Cấu hình sau cài đặt và kiểm tra sức khoẻ

Hai lỗi chỉ lộ ra ở yêu cầu đầu tiên nên phải sửa ngay sau khi cài: ngưỡng đĩa của Elasticsearch (mặc định cần ~150 GB trống) và cách agent gọi LLM (bản vá 4, sau đó `docker restart vss-agent`).

```
# ngưỡng đĩa Elasticsearch: chạy lại nhiều lần vẫn an toàn
curl -s -X PUT 'http://127.0.0.1:9200/_cluster/settings' -H 'Content-Type: application/json' -d '{"pers
  "cluster.routing.allocation.disk.watermark.low":"15gb",
  "cluster.routing.allocation.disk.watermark.high":"10gb",
  "cluster.routing.allocation.disk.watermark.flood_stage":"5gb",
  "cluster.routing.allocation.disk.watermark.flood_stage.frozen":"5gb"}}'

# generated.env chứa khoá: chỉ grep đúng tên biến cần xem, không bao giờ cat
E=../vss/deploy/docker/developer-profiles/dev-profile-lvs/generated.env
grep -E '^(LLM_MODE|VLM_MODE|LLM_NAME|VLM_NAME|RTVI_VLLM_GPU_MEMORY_UTILIZATION)=' $E
```

KIỂM TRA	KẾT QUẢ ĐẠT
Container của project `mdx`	17 `Up` (`healthy` nếu có healthcheck), 9 `Exited (0)`
Dịch vụ LVS, cổng 38111	`200` ; model `nim_nvidia_cosmos3-nano-reasoner_bf16-final`

KIỂM TRA	KẾT QUẢ ĐẠT
VLM, cổng 8018, câu "Say OK"	Trả lời `OK`, không phải `EngineDeadError`
Agent, cổng 8000, câu "hi"	Câu chào của agent, sau khoảng 17 s
Elasticsearch, cổng 9200	`green`; ngưỡng đĩa 15 / 10 / 5 GB
Kafka, topic `mdx-vlm-incidents`	8 partition, đọc được số cảnh báo
GPU	Tiến trình `VLLM::EngineCore` ~43,7 GB
Giao diện web, cổng 7777	`200`
`tests/smoke_test.sh` của cầu nối	`SMOKE TEST PASSED` (dùng mock, không gọi VSS)

> **Lỗi hay gặp:** VLM chết trong khi healthcheck vẫn báo `healthy` → `docker restart vss-rtvi-vlm`. Kafka tắt → `docker start kafka`. Elasticsearch không `green` → thường do đầy đĩa.

5 Ba cách chấm và kết quả trên 99 episode

Cầu nối có ba chương trình, ứng với ba cơ chế của VSS. Mỗi chương trình đọc một file cấu hình YAML, ghi kết quả từng episode (`epNNN.json`) và bảng tổng hợp `report.md`.

CÁCH CHẤM	CÁCH HOẠT ĐỘNG	THỜI GIAN	GIẢI MÃ
Tóm tắt video (LVS) `vss_bridge.run_pipeline`	Gửi video tới `/v1/summarize` (cổng 38111); VLM viết tóm tắt và dòng thời gian sự kiện, cầu nối đọc kết luận từ sự kiện theo quy tắc chặt	~22 s / episode	Mặc định, có lấy mẫu ngẫu nhiên
Hỏi đáp (Q&A) `vss_bridge.qa_agent`	Agent (cổng 8000) hỏi VLM: "khi video kết thúc, búa có nằm hẳn trong thùng không? YES/NO"	~82 s / episode	Mặc định, có lấy mẫu ngẫu nhiên
Cảnh báo thời gian thực (RT alert) `vss_bridge.rt_alert`	Mỗi episode nằm trong một khung cố định 8 s; VLM (cổng 8018) trả lời "có thất bại không? Có / Không"; Có = cảnh báo = thất bại	~0,7 s sau mỗi đoạn 8 s của luồng trực tiếp	Mặc định; bản sửa: `top_k: 1`

Đánh giá dùng **99 episode búa → thùng** có kết luận của Arena: 13 episode để chỉnh câu hỏi và cách đọc kết quả, **86 episode giữ riêng** chỉ dùng để chấm. Mốc so sánh là luôn trả lời "thành công": đúng 63/86 (73,3 %), vì phần lớn episode thành công. Một cách chấm chỉ có ích khi vượt được mốc này.

Vì sao cần khung cố định 8 s: episode thành công dừng ngay khi đạt điều kiện nên clip ngắn hơn. Với tóm tắt video và hỏi đáp, chỉ riêng độ dài clip đã đủ đoán đúng cả 86 episode. Cảnh báo thời gian thực đưa mọi episode về cùng độ dài nên không bị rò rỉ này; đây là một lý do khiến kết quả của nó là phép thử công bằng nhất.

CÁCH CHẤM · 86 EPISODE GIỮ RIÊNG	KHỚP VỚI ARENA	NHẬN ĐÚNG THÀNH CÔNG	NHẬN ĐÚNG THẤT BẠI	HƠN MỐC?
Tóm tắt video	52/86 (60,5 %)	56 %	74 %	Không
Hỏi đáp	51/86 (59,3 %)	51 %	83 %	Không
Cảnh báo thời gian thực, 5 lần chạy cùng cấu hình	74–80 %	83–89 %	35–65 %	Không, các lần chạy khác nhau
Cảnh báo thời gian thực, `top_k: 1`	**81/86 (94,2 %)**	**100 %**	**78 % (18/23)**	**Có (p < 0,001), đo sau khi đã biết kết quả**
Mốc: luôn trả lời "thành công"	63/86 (73,3 %)	100 %	0 %	–

Nguyên nhân kết quả dao động (đã xác nhận): cùng cấu hình, cùng khung hình, các lần chạy cảnh báo thời gian thực bất đồng trên 28/86 episode; một clip gửi 8 lần nhận 5 "Có" và 3 "Không". Dịch vụ VLM thời gian thực chỉ chuyển `temperature` cho model khi giá trị khác 0, nên `temperature 0` bị bỏ và model lấy mẫu ở mặc định 1,0. Thêm `top_k: 1` buộc giải mã tất định: hai lần chạy khớp nhau trên cả 99 episode, độ khớp với Arena lên 94,2 %, nhận đúng mọi episode thành công và 18/23 episode thất bại.

94,2 % là hướng rất triển vọng, chưa phải kết quả chính thức. Câu hỏi giữ nguyên, nhưng lần chạy này được làm sau khi đã biết kết quả của tập giữ riêng. Trên 13 episode dùng để chỉnh, nó chỉ đúng 8/13 và bỏ sót cả 2 lần robot ngã, trong khi tập giữ riêng không có lần ngã nào. Cần xác nhận bằng một lần chạy trên episode mới. Tóm tắt video và hỏi đáp cũng chịu cùng lỗi giải mã và chưa được chạy lại với `top_k: 1`, nên so sánh giữa ba cách chấm chưa công bằng.

- **Hỏi đáp** gọi 31/63 episode thành công là thất bại, với câu trả lời kiểu "búa nằm một phần trong thùng", dù khung hình cuối và số đo của Arena ở hai nhóm gần như giống nhau. Nguyên nhân khả dĩ: thùng có đáy nông và chữ "hẳn" (*fully*) trong câu hỏi.
- **Agent của VSS viết lại câu hỏi** trước khi gửi cho VLM trong 46/99 lần, nên câu hỏi thực sự được hỏi không cố định.

6 Chạy lại ba cách chấm trên 2 episode (video bước 9–11)

Hướng dẫn quay lại ba lệnh trên hai episode: ep003 (Arena: thành công) và ep025 (Arena: thất bại, búa ngoài thùng). Mỗi episode được chạy nhiều lần với cùng video (giống nhau từng bit) và cùng cấu hình: một lần chạy lưu từ trước, một lần quay video tiếng Việt, một lần quay video tiếng Anh; hỏi đáp có thêm một lần trong video demo giao diện.

```
cd <thư mục vss_bridge>
python3 -m vss_bridge.run_pipeline --config configs/guide_en_lvs.yaml \
  --inputs ../data/W8/vss_inputs/ep003.mp4 ../data/W8/vss_inputs/ep025.mp4      # tóm tắt video
python3 -m vss_bridge.qa_agent --config configs/guide_en_qa.yaml --episodes ep003,ep025   # hỏi đáp
python3 -m vss_bridge.rt_alert --config configs/guide_en_rt.yaml --episodes ep003,ep025   # cảnh báo, to
cat out/guide_en/{lvs,qa,rt}/report.md
```

EPISODE · ARENA	TÓM TẮT VIDEO	HỎI ĐÁP	CẢNH BÁO THỜI GIAN THỰC
ep003 thành công	thành công, thành công, **thất bại** 2/3 đúng	NO, YES, NO, YES 2/4 đúng	Không, Không, Không **3/3 đúng**
ep025 thất bại (búa ngoài thùng)	rỗng → thất bại; "búa nằm hẳn trong thùng" → **thành công**; "búa không được nhấc lên" → thất bại 2/3 đúng, không lần nào mô tả đúng	NO trong mọi lần đúng mọi lần	Có, Có, Có **3/3 đúng**

Lần quay khớp với đánh giá đầy đủ ở card 5: tóm tắt video và hỏi đáp đổi đáp án giữa các lần, còn cảnh báo thời gian thực với `top_k: 1` cho cùng đáp án mọi lần. Thời gian trong video (tóm tắt 19–21 s, hỏi đáp 52–54 s, cảnh báo 0,3–0,45 s mỗi episode) là của lần quay, không phải của đánh giá đầy đủ.



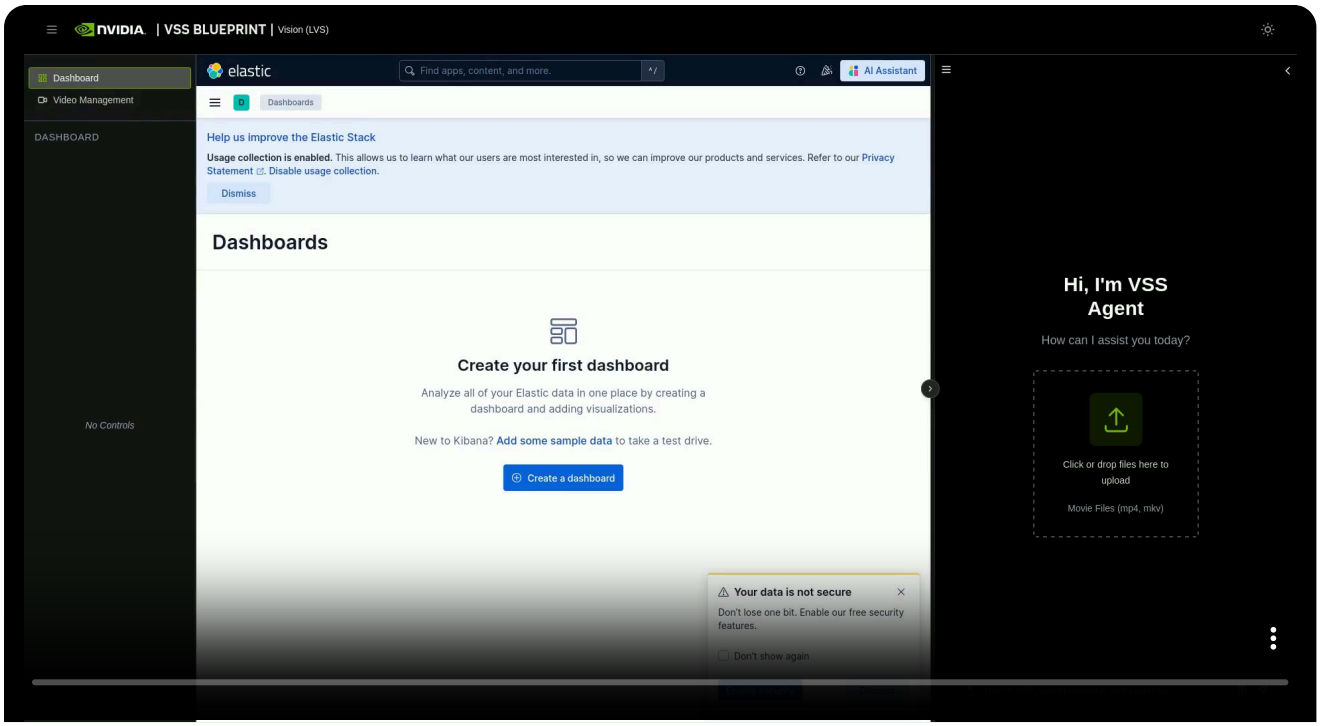
Bước 9 · Chạy thật: tóm tắt video ep003 và ep025, đọc `report.md` và câu lệnh VLM thực nhận. Đoạn chờ 43 s được tua nhanh ×6.

7 Giao diện web

Giao diện ở cổng 7777 dùng để xem video đã tải lên và hỏi agent bằng tay. Giao diện chỉ nhận truy cập qua địa chỉ nội bộ của máy chủ hoặc `localhost`, nên từ máy trạm mở qua SSH tunnel:

```
ssh -N -L 7777:127.0.0.1:7777 user@gpu-server     # rồi mở http://localhost:7777
```

THỜI ĐIỂM TRONG VIDEO	MÀN HÌNH
0:05	**Dashboard**: Kibana nhúng trong trang, chưa tạo dashboard nào
0:11	**Video Management**: 99 video đã tải lên; tải video, thêm luồng RTSP, tìm kiếm, nút "+ Chat" trên từng video
0:17–0:21	Tìm `ep003` và phát: hai nửa khung hình (camera đầu robot, góc nhìn thứ ba)
0:34	Mở khung **Chat**: "Hi, I'm VSS Agent"



Bước 12 · Chạy thật: một vòng qua giao diện web. Video này không gửi câu hỏi nào.

8 Phát hiện và việc cần làm

#	PHÁT HIỆN	VIỆC CẦN LÀM	AI LÀM
1	`dev-profile.sh --dry-run` vẫn ghi đè `generated.env` và đổi quyền `data-dir`	Chỉ dry-run trong bản clone tạm (bước 5)	Người chạy script
2	Gói `docker-ce`, `docker-ce-cli` hiện không bị khoá phiên bản; `apt upgrade` có thể đưa Docker ra ngoài khoảng đã kiểm định	`sudo apt-mark hold docker-ce docker-ce-cli`	Quản trị viên
3	Dịch vụ VLM bỏ qua `temperature 0` và lấy mẫu ở 1,0: cùng video cho đáp án khác nhau	Gửi `top_k: 1` ở cả ba cách chấm; chạy lại tóm tắt video và hỏi đáp với `top_k: 1`	Trưởng nhóm kỹ thuật
4	Con số 94,2 % đo sau khi đã biết kết quả tập giữ riêng; tập này không có lần robot ngã nào	Xác nhận bằng một lần chạy trên episode mới, có cả episode robot ngã	Trưởng nhóm kỹ thuật
5	Agent viết lại câu hỏi trong 46/99 lần; hỏi đáp gọi 31/63 episode thành công là thất bại	Cố định câu hỏi gửi tới VLM; định nghĩa rõ thế nào là "trong thùng" thay cho chữ "hẳn"	Trưởng nhóm kỹ thuật
6	Engine VLM đã chết từ 19/10 trong khi healthcheck vẫn báo `healthy`; đĩa đầy làm Kafka dừng và Elasticsearch chuyển sang chỉ đọc	Giám sát bằng một lần gọi VLM thật và theo dõi dung lượng đĩa, không chỉ dựa vào healthcheck	Quản trị viên

Kết luận: ở cấu hình hiện tại, tóm tắt video và hỏi đáp không hơn cách đoán "luôn thành công" (73,3 %). Cảnh báo thời gian thực với `top_k: 1` đạt 94,2 % trên 86 episode giữ riêng, nhanh (dưới 1 s sau mỗi đoạn 8 s) và ổn định: đây là hướng nên đi tiếp. Trước khi dùng VSS để thay hoặc bổ trợ cách chấm của Arena, cần một lần chạy xác nhận trên episode mới và chạy lại hai cách còn lại với `top_k: 1`.

9 Vận hành hằng ngày

Các lệnh chạy trong `vss/deploy/docker`, với `E=developer-profiles/dev-profile-lvs/generated.env`.

VIỆC	LỆNH
Xem log	`docker logs --tail 200 vss-lvs` (hoặc `vss-rtvi-vlm`, `vss-agent`)
Dừng / chạy lại cả hệ thống, giữ dữ liệu	`docker compose --env-file $E stop` / `start`
Giải phóng GPU	`docker stop vss-rtvi-vlm`; chạy lại bằng `docker start vss-rtvi-vlm`
Sau khi khởi động lại Docker	`docker start kafka phoenix` (hai container này không tự khởi động)
Đổi model LLM	Sửa `LLM_NAME` trong `$E`, rồi `docker compose --env-file $E up -d --no-deps --force-recreate lvs-server vss-agent`
Kiểm tra định kỳ	Gọi thử VLM (dòng VLM ở bảng card 4) và `df -h /`. Healthcheck không phát hiện được VLM đã chết; đĩa đầy làm Kafka dừng và Elasticsearch chỉ đọc (đã phải dọn 79 GB dữ liệu cũ, có phê duyệt)

Không chạy `dev-profile.sh up` hay `down` trên máy đã cài: xoá volume model 17,8 GB và cần sudo. **Không** `cat` `generated.env` hay `keys.env`, không chạy `docker compose config` hay `docker inspect` không kèm `grep`: các lệnh này in khoá ra màn hình.

V.01 Đánh giá tổng hợp

Ba kịch bản, một bức tranh chung.

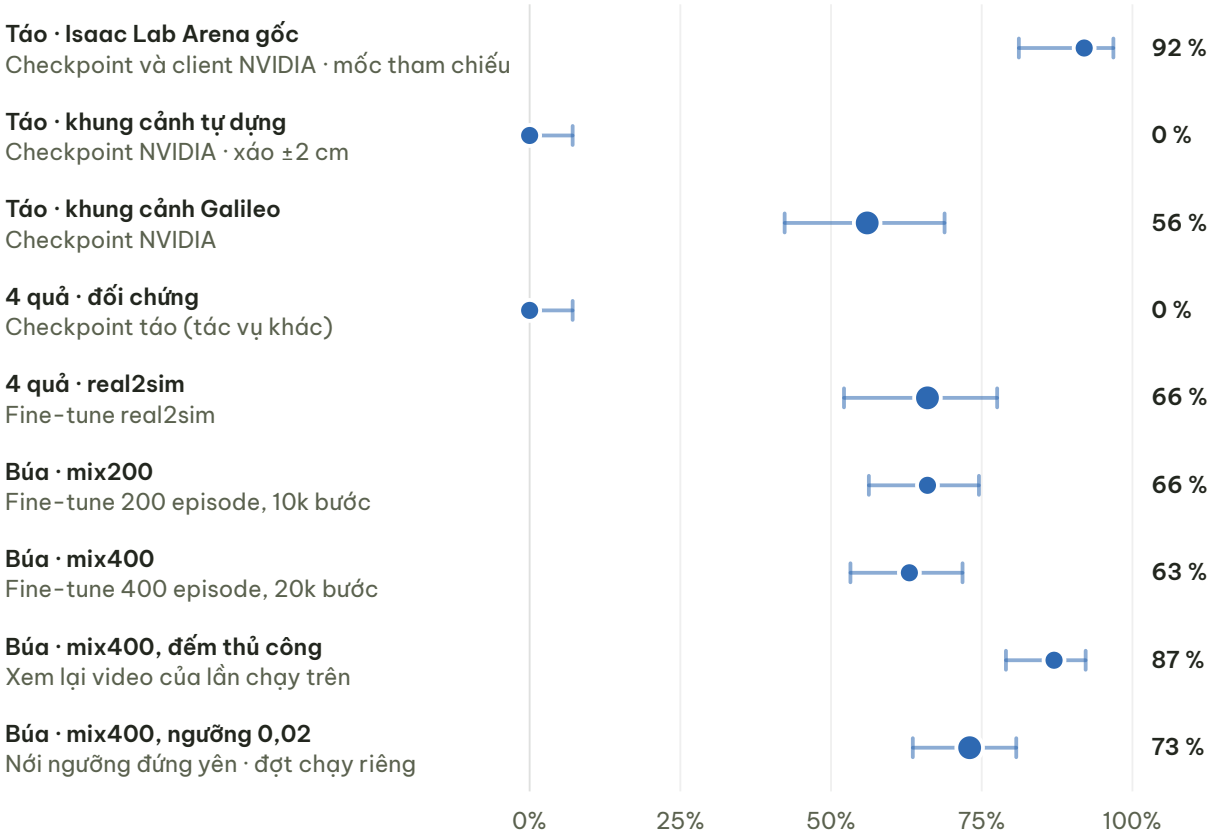
GR00T N1.7 điều khiển được G1 gắp–đặt vật trong mô phỏng: **56 %** với bài táo, **66 %** với 4 loại quả, **66–73 %** với bài búa. Kết quả phụ thuộc trước hết vào **hình ảnh khung cảnh** và **dữ liệu fine-tune**, sau đó mới tới cách chấm điểm.

1 Kết quả tổng hợp

KỊCH BẢN	TRƯỚC KHI XỬ LÝ	KẾT QUẢ TỐT NHẤT	YẾU TỐ QUYẾT ĐỊNH	NÚT THẮT CÒN LẠI
III.01 · Táo → đĩa	0 % (khung cảnh tự dựng)	**92 %** · 46/50 · Isaac Lab Arena gốc, client NVIDIA **56 %** · 28/50 · Galileo dựng lại	Khung cảnh giống lúc ghi dữ liệu	Pha nhìn và tiếp cận: khung cảnh lạ thì chỉ 22 % nhấc được táo
III.02 · 4 loại quả	0 % (checkpoint táo)	**66 %** · 33/50 · real2sim	Fine-tune trên 790 episode real2sim	Pha nắm: 11/17 thất bại là không nhấc được quả
III.03 · Búa → thùng	**0 % (chưa fine-tune)**	**66 % · 66/100 · mix200** **73 % · 73/100 · mix400, ngưỡng 0,02** **87 % · 87/100 · mix400, đếm thủ công từ video**	**Dữ liệu tự thu: teleop + sinh thêm**	**Pha thả và cách chấm "đứng yên"**

Tỉ lệ thành công và khoảng tin cậy Wilson 95 %

Chấm = tỉ lệ đo được; thanh ngang = khoảng tin cậy. Di chuột hoặc Tab để xem chi tiết.



Ghi chú: client NVIDIA trên môi trường Isaac Lab Arena gốc đạt 92 % (46/50) với bài táo; lần chạy đó là mốc tham chiếu (dòng đầu biểu đồ). Mọi lần đánh giá khác đều chạy vòng kín trên pipeline tự viết, Isaac Sim local.

Chấm bằng video (mục IV.05): thử cho NVIDIA VSS xem 99 video episode búa → thùng và tự kết luận. Trên 86 episode giữ riêng, tóm tắt video (60,5 %) và hỏi đáp (59,3 %) không hơn cách đoán "luôn thành công" (73,3 %). Sau khi sửa lỗi giải mã (`top_k: 1`), cảnh báo thời gian thực khớp với Arena 94,2 %; con số này đo sau khi đã biết kết quả nên cần xác nhận trên dữ liệu mới.

2 Nhận định chung

01 · HÌNH ẢNH

Khung cảnh quyết định nhiều hơn model

Cùng checkpoint và câu lệnh, bài táo đi từ 92 % (client NVIDIA, Arena gốc) xuống 56 % (chỉ lệch ánh sáng) rồi 0 % (lệch cả khung cảnh). Model fine-tune trên một khung cảnh không tổng quát hoá sang khung cảnh khác.

02 · FINE-TUNE

Tác vụ mới bắt buộc phải tune, và tune nhanh

Bài quả 0 % khi dùng lại checkpoint táo, bài búa 0 % khi chưa tune; cả hai đạt 66 % sau khi fine-tune. Bài búa chỉ cần 200 episode và ~1 giờ trên một H100; chi phí chính nằm ở dữ liệu.

03 · NGÔN NGỮ

Model làm đúng theo câu lệnh

0 lần gắp nhầm trong 50 episode bài quả, dù trên bàn luôn có quả khác và vật nhiễu. Phần ngôn ngữ học tốt từ dữ liệu real2sim.

04 · NÚT THẮT

Mỗi kịch bản hụt ở một pha khác nhau

Táo hụt ở pha nhìn và tiếp cận, quả hụt ở pha nắm, búa hụt ở pha thả. Hướng cải thiện vì thế phải đi theo từng tác vụ, không có một cách chung.

05 · DỮ LIỆU

Thêm dữ liệu cùng kiểu có giới hạn

Bài búa từ 200 lên 400 episode giúp đặt chính xác hơn (búa ngoài lòng thùng 19 → 5 lần) nhưng tỉ lệ chung không tăng (66 % → 63 %, trong sai số).

06 · CHẤM ĐIỂM

Cách chấm điểm ảnh hưởng tới con số

Bài búa: 22/100 lần búa đã nằm trong thùng, tay đã buông, vẫn bị chấm "chưa đứng yên" vì điều kiện chỉ xét vận tốc tức thời. Lần chạy nới ngưỡng lên 0,02 m/s đạt 73 %.

3 Bài học rút ra

01

Dữ liệu & môi trường quyết định nhiều hơn model

Cùng checkpoint: 0 % hoặc 56 % chỉ vì khung cảnh.

02

Câu lệnh là một phần "hợp đồng"

Quản lý câu lệnh như tham số cấu hình có kiểm soát.

03

Kiểm chứng theo tầng tiết kiệm thời gian

Open-loop → phát lại → vòng kín.

04

Điều kiện thành công cần kiểm thử

Điều kiện lỏng làm bẩn dữ liệu; điều kiện quá chặt làm thấp chỉ số. Thống nhất ngưỡng trước khi so sánh.

05

Đánh giá cần số lượng lớn

50–100 lần thử mỗi cấu hình, luôn kèm khoảng tin cậy.

06

Sinh thêm dữ liệu rẻ

133 lượt sinh → 100 demo đạt điều kiện chặt, không tốn thêm giờ teleop.

07

Fine-tune nhanh và rẻ

~1 giờ (200 episode) đến 2 giờ 15 (400 episode) trên 1 H100.

4 Định hướng tương lai

Giai đoạn này đã chứng minh GR00T N1.7 điều khiển được G1 gắp–đặt vật trong mô phỏng và dựng được quy trình dữ liệu → fine-tune → đánh giá cho tác vụ mới. Bước tiếp theo đi theo ba nhóm: **nghiên cứu mô hình và cảm biến**, **thu hẹp khoảng cách hình ảnh giữa mô phỏng và thực tế**, rồi **chuẩn bị đưa model lên robot thật** (giai đoạn 2).

01 · NGHIÊN CỨU · MÔ HÌNH

Nghiên cứu thêm các mô hình khác tốt hơn

Đánh giá các bản GR00T mới hơn và các mô hình VLA mã nguồn mở khác (ví dụ π0, OpenVLA) trên cùng ba kịch bản và cùng cách chấm, để biết GR00T N1.7 đang đứng ở đâu. Pipeline standalone giao tiếp với model qua server ZMQ, nên chỉ cần viết thêm server cho model mới; scene, runner và notebook đánh giá dùng lại nguyên vẹn.

02 · MÔ PHỎNG · HÌNH ẢNH

Domain randomization cho khung cảnh

Ngẫu nhiên hoá ánh sáng, vật liệu, nền, camera và vị trí vật khi tạo dữ liệu (phương pháp ở mục III.01), để model không phụ thuộc một khung cảnh: bài táo đang rơi từ 92 % xuống 56 % chỉ vì lệch ánh sáng. Đây là bước tiền đề cho sim2real.

03 · MÔ PHỎNG · TÁI TẠO

3D Gaussian Splatting để tái tạo môi trường thật

Quay ảnh hoặc video môi trường làm việc thật rồi dựng lại bằng 3D Gaussian Splatting, dùng làm nền render trong mô phỏng để ánh sáng và vật liệu giống thực tế; bài táo cho thấy chỉ lệch ánh sáng đã làm tỉ lệ rơi từ 92 % xuống 56 %. Kết hợp với real2sim, dữ liệu huấn luyện sẽ có cả chuyển động thật lẫn hình ảnh thật. Gaussian Splatting chỉ cho hình ảnh; vật lý vẫn cần mô hình va chạm riêng của bàn, kệ và vật thể.

04 · ĐÁNH GIÁ · VSS

Xác nhận cách chấm bằng video

Chạy lại cảnh báo thời gian thực với `top_k: 1` trên episode mới, có cả episode robot ngã, để xác nhận con số 94,2 %; chạy lại tóm tắt video và hỏi đáp với cùng bản sửa. Nếu đạt, đây là cách chấm tự động cho robot thật, nơi không có trạng thái mô phỏng (mục IV.05).

05 · ROBOT THẬT · DỮ LIỆU

Quy trình dữ liệu kết hợp

Teleop ít demo ngay tại môi trường làm việc → real2sim dựng lại khung cảnh → sinh thêm demo và lọc bằng mô phỏng → domain randomization → fine-tune, đánh giá trong mô phỏng rồi trên robot thật (mục II.04). Hai trong ba kỹ thuật đã được kiểm chứng ở giai đoạn này.

06 · NGHIÊN CỨU · CẢM BIẾN

Depth map giúp robot xác định khoảng cách tốt hơn

Model hiện chỉ nhận ảnh RGB từ camera đầu, trong khi nhiều thất bại nằm ở việc ước lượng khoảng cách: táo thả cách đĩa 9–20 cm, búa rơi ngoài thùng hoặc gác trên miệng thùng. Nghiên cứu dùng depth map từ camera RealSense D435 trên đầu G1 (hoặc ước lượng độ sâu từ ảnh) làm quan sát bổ sung cho model, hoặc để kiểm tra vị trí trước khi thả; cần thu dữ liệu và fine-tune có kèm độ sâu.

Thứ tự ưu tiên: 04 tốn ít công nhất và làm cho mọi con số sau này đáng tin hơn; 02 và 03 cùng nhắm vào khoảng cách hình ảnh, là điều kiện cần trước khi lên robot thật; 01 và 06 là hướng nghiên cứu để nâng năng lực model; 05 là nội dung chính của giai đoạn 2. Mọi lần đánh giá tiếp theo nên giữ cách làm của giai đoạn này: 50–100 lần thử mỗi cấu hình, kèm khoảng tin cậy.

Thuật ngữ & tham khảo.

Danh sách sản phẩm bàn giao xem mục IV.01.

A Thuật ngữ

VLA	Mô hình nhận ảnh và ngôn ngữ, xuất trực tiếp lệnh điều khiển robot
Foundation model	Mô hình pre-train trên dữ liệu lớn; fine-tune cho tác vụ cụ thể với ít dữ liệu
Embodiment	Một loại thân robot cụ thể (khớp, camera, bàn tay)
Action chunk	Chuỗi action model trả về trong một lần suy luận (40 bước = 0,8 s ở 50 Hz)
Flow matching / DiT	Kỹ thuật sinh dữ liệu liên tục bằng khử nhiễu dần, dùng để sinh chuỗi action
Teleop	Người điều khiển robot từ xa (kính VR) để tạo dữ liệu mẫu
Open-loop / closed-loop	Đánh giá trên dữ liệu ghi sẵn / model tự điều khiển liên tục
Real2sim	Chuyển dữ liệu từ thế giới thật vào mô phỏng
Domain gap	Khác biệt giữa môi trường huấn luyện và môi trường chạy
Visual gap	Model sụt tỉ lệ thành công khi hình ảnh khung cảnh lúc chạy khác lúc ghi dữ liệu (bài táo 92 → 56 → 0 %)
Domain randomization	Đổi ngẫu nhiên ánh sáng, vật liệu, nền, góc camera trong mô phỏng để model không học thuộc một khung cảnh
Demo generation	Biến đổi demo teleop có sẵn (thời gian, điểm xuất phát, gắp, thả) rồi chạy lại trong mô phỏng, chỉ giữ demo thành công (cách của MimicGen)
FK / IK	Động học thuận: từ góc khớp tính vị trí bàn tay; động học ngược: từ vị trí mong muốn tính ra góc khớp
Bộ điều khiển PD	Tỉ lệ – vi phân: mô-men = Kp × (góc mục tiêu − góc hiện tại) − Kd × vận tốc khớp; kéo khớp về mục tiêu như lò xo có giảm chấn
Rate limiter	Giới hạn mỗi bước điều khiển (20 ms) lệnh khớp chỉ được đổi tối đa một ngưỡng so với lệnh trước (tay 0,2 rad, eo 0,1 rad, bàn tay 1,5 rad)

WBC / AGILE	Bộ điều khiển toàn thân; AGILE giữ thăng bằng chân G1
Wilson 95 %	Khoảng tin cậy cho tỉ lệ thành công, phù hợp số lần thử nhỏ
MAE	Sai số tuyệt đối trung bình

B Tài liệu tham khảo

1. NVIDIA. *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots.* arXiv:2503.14734, 2025.
2. NVIDIA. Model card GR00T N1.7-3B và GN1x-Tuned-Arena-G1-Static-PickNPlace (Hugging Face).
3. Isaac Lab Arena — github.com/isaac-sim/IsaacLab-Arena
4. Isaac-GR00T — github.com/NVIDIA/Isaac-GR00T
5. NVIDIA. GR00T end-to-end workflow: fine-tuning on sim data, sim evaluation (docs.nvidia.com).
6. Dataset nvidia/Arena-G1-Static-PickNPlace-Task và nvidia/PhysicalAI-Robotics-GR00T-Teleop-G1 (CC-BY-4.0).
7. Black, K. et al. *π0: A Vision-Language-Action Flow Model for General Robot Control.* arXiv:2410.24164, 2024.
8. Liu, X., Gong, C. *Flow Straight and Fast: Rectified Flow.* ICLR 2023.